

Claude Code 学习手册

作者：月之眼

AI 辅助整理：ChatGPT

生成时间：2026-05-16 02:04

本手册面向已经安装 Claude Code、希望从基础使用逐步进阶到工程化协作的新手与程序员。内容采用案例驱动、边做边学的方式，覆盖 Markdown 工作流、Skill、SubAgent、Hook、MCP、多 Agent 协作、自动化工作流、GitHub / GitLab / CI、SDK、本地知识库、安全权限、沙箱与成本控制。

总目录

- **第一篇：入门地图 - 从 AI Agent 到 Claude Code**
- 第 0 章：AI Agent 与 LLM 原理预备（新手必读）
- 第 1 章：为什么 Claude Code 不只是聊天机器人
- 第 2 章：安装、登录、第一个任务
- 第 3 章：Claude Code 的工具系统全景
- 第 4 章：学习用项目准备（三语言基线）
- **第二篇：日常开发工作流 - 先把 Claude Code 用顺手**
- 第 5 章：代码理解与项目地图
- 第 6 章：Bug 定位与修复
- 第 7 章：重构、测试和文档
- 第 8 章：Git 工作流与提交质量
- 第 9 章：从一次性问答到可复用工作流

- **第三篇：Markdown、记忆和本地知识库**
 - 第 10 章：Markdown 不是文档格式，而是 Agent 控制层
 - 第 11 章：`CLAUDE.md`、Auto Memory 与导入机制
 - 第 12 章：项目规则、`AGENTS.md` 和规则文件夹
 - 第 13 章：本地知识库与 ADR
 - 第 14 章：记忆系统的反模式
- **第四篇：提示词、Opus 4.7 和模型驾驭**
 - 第 15 章：Claude Code 提示词基本功
 - 第 16 章：计划模式与探索模式
 - 第 17 章：驯服 Opus 4.7：模型、思考预算、Fast Mode 与目标驱动
 - 第 18 章：复杂工程任务的提示词编排
- **第五篇：上下文、Token 和成本控制**
 - 第 19 章：上下文窗口与文件选择
 - 第 20 章：压缩、检查点与长会话管理
 - 第 21 章：Token 节省与成本优化
- **第六篇：扩展系统 - Skills、Commands、SubAgents、Hooks、MCP、Plugins、个性化**
 - 第 22 章：Skills 基础
 - 第 23 章：自定义命令与任务型 Skill
 - 第 24 章：SubAgents 基础
 - 第 25 章：多 Agent 协作与 Agent Teams
 - 第 26 章：Hooks 事件驱动自动化
 - 第 27 章：MCP 基础与外部工具连接

- 第 28 章: Plugins 插件系统与市场
- 第 29 章: Tools 与 Rules 体系
- 第 30 章: Skill、Agent、MCP、Plugin 的组合模式
- 第 31 章: 扩展系统实战小项目
- 第 32 章: 个性化定制 - Output Styles、Status Line、Keybindings、Voice、Fullscreen
 - **第七篇: 多端、Web、桌面与遥控**
- 第 33 章: 桌面应用、Web 与 iOS
- 第 34 章: Remote Control、Dispatch 与跨设备会话
- 第 35 章: Channels - 把外部事件推到运行中的会话
- 第 36 章: Slack、Chrome、IDE 与外部工具集成
 - **第八篇: 自动化、Headless、CI 和云端规划**
- 第 37 章: Headless 模式与命令行自动化
- 第 38 章: Routines、`/loop` 与定时任务
- 第 39 章: GitHub / GitLab CI、Code Review 自动化
- 第 40 章: ultraplan、ultrareview 与 Worktrees
 - **第九篇: Agent SDK - 把 Claude Code 嵌进你自己的产品**
- 第 41 章: Agent SDK 入门
- 第 42 章: SDK 中的权限、工具与 Hooks
- 第 43 章: SDK 中的多 Agent、Skills、Commands 与 Plugins
- 第 44 章: SDK 部署、可观测性与迁移
 - **第十篇: 安全、权限、沙箱和风险控制**
- 第 45 章: 权限系统与 Permission Modes

- 第 46 章：沙箱、数据安全和提示注入
- 第 47 章：Git、安全发布和回滚
- 第 48 章：生产级 Claude Code 使用规范与可观测性
- **第十一篇：高级驾驭工程 - 从会用到会设计 Agent 工作系统**
- 第 49 章：Claude Code 内部工作机制导览
- 第 50 章：上下文工程、缓存和成本工程
- 第 51 章：从个人能力到团队 AI 编码体系
- **第十二篇：完整实战案例**
- 第 52 章：实战一 - Python 项目从 Bug 到 PR（个人完整链路）
- 第 53 章：实战二 - C# Web API + C++ 库混合项目的团队 Claude Code 工具包
- 第 54 章：实战三 - 用 Agent SDK 写一个自定义 Agent

入门地图 - 从 AI Agent 到 Claude Code

第 0 章：AI Agent 与 LLM 原理预备（新手必读）

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

本章介绍“AI Agent 与 LLM 原理预备（新手必读）”的核心概念，并通过可验证的小任务帮助读者建立实践基础。它不只解释“AI Agent 与 LLM 原理预备（新手必读）是什么”，还要求读者完成一个可复盘的小任务，把经验沉淀到 Markdown、Skill、SubAgent、Hook、MCP 或自动化脚本中。

学习目标

- 理解“AI Agent 与 LLM 原理预备（新手必读）”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为可交给 Claude Code 执行的小任务。
- 能用 PowerShell 完成主流程，并读懂 macOS/Linux 对照命令。
- 能识别本章能力的适用场景、风险和不适用场景。
- 能用检查清单判断任务是否真正完成。

先做一个小案例

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\enhanced"
claude "                                "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/enhanced
claude "                                "
```

案例中的最小代码对象如下：

```
from pathlib import Path

target = Path("README.md").resolve()
print({"target": str(target), "exists": target.exists()})
```

目录

- 0.1 LLM 是什么
 - 0.1.1 模型、参数与训练数据
 - 0.1.2 推理过程：从提示词到下一个 token
 - 0.1.3 温度、采样与确定性
- 0.2 Chatbot、Assistant、Agent 三者区别
 - 0.2.1 Chatbot：一次问答，无外部动作
 - 0.2.2 Assistant：能调用少量预设工具
 - 0.2.3 Agent：自主感知 - 规划 - 行动 - 观察循环
- 0.3 Tool Calling（工具调用）机制
 - 0.3.1 LLM 如何「使用」工具
 - 0.3.2 工具结果回到上下文意味着什么
 - 0.3.3 为什么工具描述比工具实现更影响效果
- 0.4 上下文窗口与 token 经济学入门
 - 0.4.1 输入、输出、工具结果都计费
 - 0.4.2 上下文窗口大小与「读太多」的代价
 - 0.4.3 估算一段文本的 token 数
- 0.5 模型家族：Opus 4.7 / Sonnet 4.6 / Haiku 4.5
 - 0.5.1 三档模型的能力与价格梯度

- 0.5.2 任务难度 → 模型选择决策表
- 0.5.3 Fast Mode 与思考预算先打个招呼
 - 0.6 幻觉、对齐与「Agent 错觉」
- 0.6.1 模型为什么会编造文件路径或函数名
- 0.6.2 对齐与系统提示词的作用
- 0.6.3 为什么后面要学权限、Hook 和验证

0.1 LLM 是什么

学习目标

- 理解「LLM 是什么」解决的具体问题。
- 能把「LLM 是什么」写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

「LLM 是什么」的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕

模型、参数与训练数据、推理过程：从提示词到下一个 token、温度、采样与确定性，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。

4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“LLM ”

练习

- 练习 1：把一个与`LLM 是什么`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-00.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

0.2 Chatbot、Assistant、Agent 三者区别

学习目标

- 理解`Chatbot、Assistant、Agent 三者区别`解决的具体问题。
- 能把`Chatbot、Assistant、Agent 三者区别`
 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

“Chatbot、Assistant、Agent 三者区别”

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕

Chatbot：一次问答，无外部动作、Assistant：能调用少量预设工具、

Agent：自主感知 - 规划 - 行动 - 观察循环，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 “.md”、Skill、命令或团队规则。

示例提示词

“Chatbot Assistant Agent ”

练习

- 练习 1：把一个与 “Chatbot、Assistant、Agent 三者区别” 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 “.notes/enhanced-00.md”。

检查清单

- [] 任务边界清楚。

- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

0.3 Tool Calling（工具调用）机制

学习目标

- 理解 `Tool Calling（工具调用）机制` 解决的具体问题。
- 能把 `Tool Calling（工具调用）机制` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Tool Calling（工具调用）机制`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 LLM 如何「使用」工具、工具结果回到上下文意味着什么、为什么工具描述比工具实现更影响效果，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

练习

- 练习 1: 把一个与 `Tool Calling`（工具调用）机制相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-00.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

0.4 上下文窗口与 token 经济学入门

学习目标

- 理解 `上下文窗口与 token 经济学入门` 解决的具体问题。
- 能把 `上下文窗口与 token 经济学入门` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`上下文窗口与 token 经济学入门`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕输入、输出、工具结果都计费、上下文窗口大小与「读太多」的代价、估算一段文本的 token 数，你要让 Claude Code 在可控范围内读取

资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“ token ”

练习

- 练习 1：把一个与`上下文窗口与 token 经济学入门`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-00.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

0.5 模型家族：Opus 4.7 / Sonnet 4.6 / Haiku 4.5

学习目标

- 理解 `模型家族：Opus 4.7 / Sonnet 4.6 / Haiku 4.5` 解决的具体问题。
- 能把 `模型家族：Opus 4.7 / Sonnet 4.6 / Haiku 4.5` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`模型家族：Opus 4.7 / Sonnet 4.6 / Haiku 4.5`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕三档模型的能力与价格梯度、任务难度 → 模型选择决策表、Fast Mode 与思考预算先打个招呼，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“ Opus 4.7 / Sonnet 4.6 / Haiku 4.5”

练习

- 练习 1: 把一个与 `模型家族: Opus 4.7 / Sonnet 4.6 / Haiku 4.5` 相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索, 并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-00.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

0.6 幻觉、对齐与「Agent 错觉」

学习目标

- 理解 `幻觉、对齐与「Agent 错觉」` 解决的具体问题。
- 能把 `幻觉、对齐与「Agent 错觉」` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用, 什么时候不要使用。

核心概念

`幻觉、对齐与「Agent 错觉」`

的重点不是记住术语, 而是理解它如何改变 Agent 的工作边界。围绕模型为什么会编造文件路径或函数名、对齐与系统提示词的作用、为什么后面要学权限、Hook 和验证, 你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据, 并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 ``md``、Skill、命令或团队规则。

示例提示词

“ Agent ”

练习

- 练习 1：把一个与 ``幻觉、对齐与「Agent 错觉」`` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 ``notes/enhanced-00.md``。

检查清单

- ☐ 任务边界清楚。
- ☐ 有可复现命令。
- ☐ 有证据和验证结果。
- ☐ 有风险与回滚说明。

常见坑

- 把新功能当成固定不变的事实，不复核官方文档。

- 没有限制工具权限，导致 Agent 执行范围过大。
- 只生成配置，不实际跑一次验证。
- 忽略成本、上下文污染和多人协作时的规则冲突。

本章小结

掌握“AI Agent 与 LLM 原理预备（新手必读）”的标准不是看懂定义，而是能把它放进你的真实开发流程：有输入、有边界、有工具、有验证、有沉淀。

第 1 章：为什么 Claude Code 不只是聊天机器人

> 所属篇章：第一篇：入门地图 - 从 AI Agent 到 Claude Code

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

帮助读者建立第一个正确心智模型：Claude Code 是能读取项目、调用工具、修改文件、运行命令并受权限约束的 AI 编码 Agent，不是只回答问题的聊天窗口。

本章先用一个简单函数分析任务展示聊天模型和 Claude Code 的区别，再解释 Claude Code 的核心闭环：理解目标、探索项目、使用工具、修改文件、验证结果、总结交付。重点强调“上下文不是魔法”“权限不是麻烦”“验证不是可选项”。读者应明白，后续所有高级功能都建立在这个闭环之上。

学习目标

- 理解“为什么 Claude Code 不只是聊天机器人”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-01"
claude "    "    Claude Code    "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-01
claude "    "    Claude Code    "    "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 1.1 AI Chat、AI Agent、AI 编码 Agent 的区别
 - 1.1.1 普通聊天模型能做什么
 - 1.1.2 AI Agent 多了哪些能力
 - 1.1.3 AI 编码 Agent 为什么需要文件系统、命令行和权限
- 1.2 Claude Code 的核心工作方式
 - 1.2.1 用户目标如何变成模型任务

- 1.2.2 模型如何选择读取、搜索、编辑、执行命令
- 1.2.3 工具结果如何反过来影响下一步决策
- 1.3 新手最容易误解的地方
- 1.3.1 以为模型天然知道整个项目
- 1.3.2 以为说一句“帮我修好”就足够
- 1.3.3 以为 AI 修改代码不需要验证
- 1.4 本书的学习方法
- 1.4.1 每章先做一个小任务
- 1.4.2 用检查清单判断是否真正掌握
- 1.4.3 把成功经验写回 Markdown 和规则文件

1.1 AI Chat、AI Agent、AI 编码 Agent 的区别

学习目标

- 理解 `AI Chat、AI Agent、AI 编码 Agent` 的区别` 在本章主题中的具体作用。
- 能把 `AI Chat、AI Agent、AI 编码 Agent` 的区别` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：普通聊天模型能做什么、AI Agent 多了哪些能力、AI 编码 Agent 为什么需要文件系统、命令行和权限。

核心概念

`AI Chat、AI Agent、AI 编码 Agent 的区别`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `1.1.1 普通聊天模型能做什么`：先解释它解决的问题，再给出一个可观察的操作。
- `1.1.2 AI Agent 多了哪些能力`：先解释它解决的问题，再给出一个可观察的操作。
- `1.1.3 AI 编码 Agent 为什么需要文件系统、命令行和权限`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-01"
claude " "AI Chat AI Agent AI Agent
" "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-01
claude " "AI Chat AI Agent AI Agent
" "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `AI Chat、AI Agent、AI 编码 Agent 的区别` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `AI Chat、AI Agent、AI 编码 Agent 的区别` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

1.2 Claude Code 的核心工作方式

学习目标

- 理解 `Claude Code 的核心工作方式` 在本章主题中的具体作用。
- 能把 `Claude Code 的核心工作方式` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：用户目标如何变成模型任务、模型如何选择读取、搜索、编辑、执行命令、工具结果如何反过来影响下一步决策。

核心概念

`Claude Code 的核心工作方式`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `1.2.1 用户目标如何变成模型任务`：先解释它解决的问题，再给出一个可观察的操作。
- `1.2.2 模型如何选择读取、搜索、编辑、执行命令`：先解释它解决的问题，再给出一个可观察的操作。
- `1.2.3 工具结果如何反过来影响下一步决策`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**Python**`。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-01"
claude "    "Claude Code                "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-01
claude "    "Claude Code                "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1: 把 `Claude Code 的核心工作方式` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2: 要求 Claude Code 先列计划，不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Claude Code 的核心工作方式` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

1.3 新手最容易误解的地方

学习目标

- 理解 `新手最容易误解的地方` 在本章主题中的具体作用。
- 能把 `新手最容易误解的地方` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：以为模型天然知道整个项目、以为说一句“帮我修好”就足够、以为 AI 修改代码不需要验证。

核心概念

`新手最容易误解的地方`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 1.3.1 以为模型天然知道整个项目：先解释它解决的问题，再给出一个可观察的操作。
- 1.3.2 以为说一句“帮我修好”就足够：先解释它解决的问题，再给出一个可观察的操作。
- 1.3.3 以为 AI 修改代码不需要验证：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-01"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-01
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
```

```
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `新手最容易误解的地方` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `新手最容易误解的地方` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

1.4 本书的学习方法

学习目标

- 理解 `本书的学习方法` 在本章主题中的具体作用。
- 能把 `本书的学习方法` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：每章先做一个小任务、用检查清单判断是否真正掌握、把成功经验写回 Markdown 和规则文件。

核心概念

本书的学习方法

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 1.4.1 每章先做一个小任务：先解释它解决的问题，再给出一个可观察的操作。
- 1.4.2 用检查清单判断是否真正掌握：先解释它解决的问题，再给出一个可观察的操作。
- 1.4.3 把成功经验写回 Markdown 和规则文件：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-01"
claude " " " "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-01
claude " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`本书的学习方法`改写成一个包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`本书的学习方法`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：同一个 Bug 让普通聊天模型和 Claude Code 分别分析，比较输出差异。
- 练习：写出 3 个 Claude Code 适合做、普通聊天模型不适合直接做的工程任务。
- 练习：画一张“用户、Claude Code、工具、项目文件、终端命令”的关系图。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `notes/01-agent-vs-chat.md`

- 一张 Claude Code 工作闭环图
- 5 条个人使用 Claude Code 的基础原则

本章检查清单

- [] 我已经完成第 1 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“为什么 Claude Code 不只是聊天机器人”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“环境、账号、模型和第一个任务”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 2 章：安装、登录、第一个任务

> 所属篇章：第一篇：入门地图 - 从 AI Agent 到 Claude Code

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者确认 Claude Code

能在自己的机器上正常工作，并完成第一个可验证的真实任务。

本章围绕“能不能正常使用”展开，不深入复杂功能。先确认安装、登录、模型和目录，再用一个小项目做第一个代码理解任务。重点不是让 Claude Code 给出漂亮总结，而是让读者学会要求它引用真实文件、说明探索过程，并能在环境出错时自己定位问题。

学习目标

- 理解“环境、账号、模型和第一个任务”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 `**C#**` 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。


```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-02"
claude " " " "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-02
claude " " " "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 2.1 安装与登录检查
 - 2.1.1 检查 Claude Code 是否安装成功
 - 2.1.2 检查账号、模型和网络状态
 - 2.1.3 记录本机配置路径和常用诊断命令
- 2.2 终端与工作目录基础
 - 2.2.1 什么是当前工作目录
 - 2.2.2 为什么在错误目录启动会影响结果
 - 2.2.3 Windows PowerShell 路径与常见路径错误
- 2.3 第一个真实任务

- 2.3.1 让 Claude Code 识别项目入口
- 2.3.2 让 Claude Code 输出模块关系
- 2.3.3 要求它给出证据文件路径
- 2.4 常用状态检查
- 2.4.1 出问题先检查什么
- 2.4.2 权限、模型、网络、目录四类常见问题
- 2.4.3 建立环境检查清单

2.1 安装与登录检查

学习目标

- 理解`安装与登录检查`在本章主题中的具体作用。
- 能把`安装与登录检查`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：检查 Claude Code 是否安装成功、检查账号、模型和网络状态、记录本机配置路径和常用诊断命令。

核心概念

`安装与登录检查`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `2.1.1 检查 Claude Code 是否安装成功`：先解释它解决的问题，再给出一个可观察的操作。

- `2.1.2 检查账号、模型和网络状态`：先解释它解决的问题，再给出一个可观察的操作。
- `2.1.3 记录本机配置路径和常用诊断命令`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-02"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-02
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
    }
}
```

```
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||  
Directory.Exists(fullPath)}");  
    }  
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `安装与登录检查` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `安装与登录检查` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

2.2 终端与工作目录基础

学习目标

- 理解 `终端与工作目录基础` 在本章主题中的具体作用。
- 能把 `终端与工作目录基础` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：什么是当前工作目录、为什么在错误目录启动会影响结果、Windows PowerShell 路径与常见路径错误。

核心概念

终端与工作目录基础

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 2.2.1 什么是当前工作目录：先解释它解决的问题，再给出一个可观察的操作。
- 2.2.2 为什么在错误目录启动会影响结果：先解释它解决的问题，再给出一个可观察的操作。
- 2.2.3 Windows PowerShell 路径与常见路径错误：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-02"
claude "    "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-02
claude " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `终端与工作目录基础` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `终端与工作目录基础` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

2.3 第一个真实任务

学习目标

- 理解 `第一个真实任务` 在本章主题中的具体作用。
- 能把 `第一个真实任务` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：让 Claude Code 识别项目入口、让 Claude Code 输出模块关系、要求它给出证据文件路径。

核心概念

`第一个真实任务`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `2.3.1 让 Claude Code 识别项目入口`：先解释它解决的问题，再给出一个可观察的操作。
- `2.3.2 让 Claude Code 输出模块关系`：先解释它解决的问题，再给出一个可观察的操作。
- `2.3.3 要求它给出证据文件路径`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-02"
claude "    "    "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-02
claude "    "    "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1: 把 `第一个真实任务` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2: 要求 Claude Code 先列计划，不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `第一个真实任务` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

2.4 常用状态检查

学习目标

- 理解 `常用状态检查` 在本章主题中的具体作用。
- 能把 `常用状态检查` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：出问题先检查什么、权限、模型、网络、目录四类常见问题、建立环境检查清单。

核心概念

`常用状态检查`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `2.4.1 出问题先检查什么`：先解释它解决的问题，再给出一个可观察的操作。
- `2.4.2 权限、模型、网络、目录四类常见问题`：先解释它解决的问题，再给出一个可观察的操作。
- `2.4.3 建立环境检查清单`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-02"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-02
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
```

```

    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `常用状态检查` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `常用状态检查` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |

在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |

要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：进入一个已有项目目录，让 Claude Code 总结项目入口、依赖、运行方式。
- 练习：分别在错误目录和正确目录提问，观察回答差异。
- 练习：写一份本机 Claude Code 环境检查清单。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `notes/02-environment-checklist.md`
- 第一个项目结构摘要
- 常用启动与诊断命令清单

本章检查清单

- [] 我已经完成第 2 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“环境、账号、模型和第一个任务”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Claude Code 的文件读写和工具系统”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 3 章：Claude Code 的工具系统全景

> 所属篇章：第一篇：入门地图 - 从 AI Agent 到 Claude Code

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者理解 Claude Code 为什么能处理工程任务：它不是只靠模型记忆，而是通过工具读取、搜索、编辑和执行命令。

本章以一次“小 Bug 修复”为主线，拆解 Claude Code 可能使用的工具链：搜索相关代码、读取关键文件、提出修改计划、应用补丁、运行测试、总结结果。重点讲“工具选择”而不是“工具百科”，让读者理解什么时候该让它搜索、什么时候该让它读取、什么时候必须运行命令验证。

学习目标

- 理解“Claude Code 的文件读写和工具系统”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C++** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-03"
claude "        "Claude Code        "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-03
claude "        "Claude Code        "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 3.1 文件读取工具
 - 3.1.1 读取单个文件与搜索文件的区别
 - 3.1.2 先搜索再阅读的效率优势
 - 3.1.3 如何要求 Claude Code 给出读取证据
- 3.2 编辑工具
 - 3.2.1 小范围修改与大范围重写
 - 3.2.2 修改前说明计划
 - 3.2.3 修改后查看 diff

- 3.3 Shell 命令工具
- 3.3.1 命令执行为什么重要
- 3.3.2 只读命令、写入命令和危险命令
- 3.3.3 运行测试、格式化和构建
- 3.4 Todo 与任务跟踪
- 3.4.1 为什么复杂任务需要任务列表
- 3.4.2 如何判断 Todo 是否真实推进
- 3.4.3 防止遗漏验证步骤

3.1 文件读取工具

学习目标

- 理解 `文件读取工具` 在本章主题中的具体作用。
- 能把 `文件读取工具` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：读取单个文件与搜索文件的区别、先搜索再阅读的效率优势、如何要求 Claude Code 给出读取证据。

核心概念

`文件读取工具`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `3.1.1 读取单个文件与搜索文件的区别`：先解释它解决的问题，再给出一个可观察的操作。
- `3.1.2 先搜索再阅读的效率优势`：先解释它解决的问题，再给出一个可观察的操作。
- `3.1.3 如何要求 Claude Code`
给出读取证据：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-03"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-03
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
```

```

";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `文件读取工具` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `文件读取工具` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

3.2 编辑工具

学习目标

- 理解 `编辑工具` 在本章主题中的具体作用。
- 能把 `编辑工具` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：小范围修改与大范围重写、修改前说明计划、修改后查看 diff。

核心概念

编辑工具

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 3.2.1 小范围修改与大范围重写：先解释它解决的问题，再给出一个可观察的操作。
- 3.2.2 修改前说明计划：先解释它解决的问题，再给出一个可观察的操作。
- 3.2.3 修改后查看
diff：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-03"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-03
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `编辑工具` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `编辑工具` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

3.3 Shell 命令工具

学习目标

- 理解 `Shell 命令工具` 在本章主题中的具体作用。

- 能把 `Shell 命令工具` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：命令执行为什么重要、只读命令、写入命令和危险命令、运行测试、格式化和构建。

核心概念

`Shell 命令工具`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `3.3.1 命令执行为什么重要`：先解释它解决的问题，再给出一个可观察的操作。
- `3.3.2 只读命令、写入命令和危险命令`：先解释它解决的问题，再给出一个可观察的操作。
- `3.3.3 运行测试、格式化和构建`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。

5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-03"
claude "    "Shell        "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-03
claude "    "Shell        "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Shell 命令工具` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Shell 命令工具` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。

- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

3.4 Todo 与任务跟踪

学习目标

- 理解 `Todo 与任务跟踪` 在本章主题中的具体作用。
- 能把 `Todo 与任务跟踪` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：为什么复杂任务需要任务列表、如何判断 Todo 是否真实推进、防止遗漏验证步骤。

核心概念

`Todo 与任务跟踪`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `3.4.1 为什么复杂任务需要任务列表`：先解释它解决的问题，再给出一个可观察的操作。
- `3.4.2 如何判断 Todo 是否真实推进`：先解释它解决的问题，再给出一个可观察的操作。
- `3.4.3 防止遗漏验证步骤`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-03"
claude "    "Todo    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-03
claude "    "Todo    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Todo` 与任务跟踪`
改写成包含目标、范围、限制、输出格式的提示词。

- 练习 2: 要求 Claude Code 先列计划, 不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令, 并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Todo 与任务跟踪` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围, 明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成, 但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件, 并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例: 让 Claude Code
用搜索定位一个函数, 不允许它全量阅读项目。
- 练习: 要求它在修改前列出计划和预计修改文件。
- 练习: 让它运行测试并解释失败输出。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `notes/03-tool-chain.md`
- 一份“读、搜、改、跑、验”的工具使用流程
- 危险命令识别清单

本章检查清单

- [] 我已经完成第 3 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“Claude Code 的文件读写和工具系统”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“学习用项目准备”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>

- Skills: <https://code.claude.com/docs/zh-CN/skills>

第 4 章：学习用项目准备（三语言基线）

> 所属篇章：第一篇：入门地图 - 从 AI Agent 到 Claude Code

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

为全书后续练习准备一个可反复使用的小项目，避免每章都从空白开始。

本章将读者从“看教程”带到“有项目可练”。项目选择不要求复杂，但必须有真实代码、真实测试和真实文档。后续的 Bug

修复、重构、记忆、Skill、SubAgent、MCP

和插件练习，都围绕这个学习项目逐步展开。

学习目标

- 理解“学习用项目准备”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 ->

获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-04"
claude "    "    "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-04
claude "    "    "    "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 4.1 选择练习项目
 - 4.1.1 项目不能太大也不能太假
 - 4.1.2 Python、C#、C++ 项目的选择原则
 - 4.1.3 必备结构：源码、测试、文档、Git
- 4.2 建立初始文档
 - 4.2.1 `README.md` 说明项目目的
 - 4.2.2 `docs/` 存放项目地图和规则
 - 4.2.3 `CHANGELOG.md` 记录学习过程中的变更

- 4.3 建立 Git 基线
 - 4.3.1 初始化仓库和首个提交
 - 4.3.2 建立学习分支
 - 4.3.3 用 diff 评估 AI 修改
- 4.4 学习记录规范
 - 4.4.1 每章记录学到的能力
 - 4.4.2 记录 Claude Code 犯过的错误
 - 4.4.3 记录可复用提示词和规则

4.1 选择练习项目

学习目标

- 理解 `选择练习项目` 在本章主题中的具体作用。
- 能把 `选择练习项目` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：项目不能太大也不能太假、Python、C#、C++ + 项目的选择原则、必备结构：源码、测试、文档、Git。

核心概念

`选择练习项目`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `4.1.1 项目不能太大也不能太假`：先解释它解决的问题，再给出一个可观察的操作。
- `4.1.2 Python、C#、C++ 项目的选择原则`：先解释它解决的问题，再给出一个可观察的操作。
- `4.1.3 必备结构：源码、测试、文档、Git`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-04"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-04
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
```

```
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `选择练习项目` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `选择练习项目` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

4.2 建立初始文档

学习目标

- 理解 `建立初始文档` 在本章主题中的具体作用。
- 能把 `建立初始文档` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：`README.md` 说明项目目的、`docs/` 存放项目地图和规则、`CHANGELOG.md` 记录学习过程中的变更。

核心概念

`建立初始文档`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `4.2.1 README.md`
说明项目目的：先解释它解决的问题，再给出一个可观察的操作。
- `4.2.2 docs/` 存放项目地图和规则：先解释它解决的问题，再给出一个可观察的操作。
- `4.2.3 CHANGELOG.md` 记录学习过程中的变更：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-04"
claude " " " "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-04
claude " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `建立初始文档` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `建立初始文档` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

4.3 建立 Git 基线

学习目标

- 理解 `建立 Git 基线` 在本章主题中的具体作用。
- 能把 `建立 Git 基线` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：初始化仓库和首个提交、建立学习分支、用 diff 评估 AI 修改。

核心概念

`建立 Git 基线`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `4.3.1 初始化仓库和首个提交`：先解释它解决的问题，再给出一个可观察的操作。
- `4.3.2 建立学习分支`：先解释它解决的问题，再给出一个可观察的操作。
- `4.3.3 用 diff 评估 AI 修改`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-04"
claude "    "    Git    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-04
claude "    "    Git    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1: 把 `建立 Git 基线` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2: 要求 Claude Code 先列计划，不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `建立 Git 基线` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

4.4 学习记录规范

学习目标

- 理解 `学习记录规范` 在本章主题中的具体作用。
- 能把 `学习记录规范` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：每章记录学到的能力、记录 Claude Code 犯过的错误、记录可复用提示词和规则。

核心概念

`学习记录规范`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `4.4.1 每章记录学到的能力`：先解释它解决的问题，再给出一个可观察的操作。
- `4.4.2 记录 Claude Code 犯过的错误`：先解释它解决的问题，再给出一个可观察的操作。
- `4.4.3 记录可复用提示词和规则`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-04"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-04
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
```

```

    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `学习记录规范` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `学习记录规范` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |

要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：准备一个小型命令行工具或 Web API 项目。
- 练习：让 Claude Code 为项目生成初始 `docs/code-map.md`。
- 练习：制造一个可控 Bug，用于后续章节修复。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- 一个学习用 Git 仓库
- `README.md`
- `docs/code-map.md`
- `notes/claude-code-learning.md`

本章检查清单

- [] 我已经完成第 4 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“学习用项目准备”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“代码理解与项目地图”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

日常开发工作流 - 先把 Claude Code 用顺手

第 5 章：代码理解与项目地图

> 所属篇章：第二篇：日常开发工作流 - 先把 Claude Code 用顺手

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

训练读者把 Claude Code

用作项目理解助手，但要求所有结论都能回到代码证据。

本章不让 Claude Code 直接写代码，而是训练“理解项目”的工作流。章节重点是探索顺序、输出格式和证据链。读者要学会把“这个项目是干什么的”改成“请只读文件，找出入口、关键模块、数据流、运行命令，并为每个结论附文件路径”。

学习目标

- 理解“代码理解与项目地图”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C#** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
```

```
Set-Location "D:\AI\claude code    \book\examples\chapter-05"
claude "    "    "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-05
claude "    "    "    "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 5.1 让 Claude Code 先探索
 - 5.1.1 先列探索计划
 - 5.1.2 先搜索入口再读取文件
 - 5.1.3 限制只读不改
- 5.2 结构化提问
 - 5.2.1 目标、范围、限制、输出格式
 - 5.2.2 模糊问题改写为工程任务
 - 5.2.3 输出中要求文件路径和证据
- 5.3 代码地图输出

- 5.3.1 项目入口
- 5.3.2 模块边界
- 5.3.3 数据流和风险点
- 5.4 反查与验证
- 5.4.1 抽查模型结论
- 5.4.2 要求引用源码
- 5.4.3 更新错误总结

5.1 让 Claude Code 先探索

学习目标

- 理解“让 Claude Code 先探索”在本章主题中的具体作用。
- 能把“让 Claude Code 先探索”转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：先列探索计划、先搜索入口再读取文件、限制只读不改。

核心概念

“让 Claude Code 先探索”

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 5.1.1
先列探索计划：先解释它解决的问题，再给出一个可观察的操作。

- `5.1.2 先搜索入口再读取文件`：先解释它解决的问题，再给出一个可观察的操作。

- `5.1.3

限制只读不改`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-05"
claude "    " Claude Code    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-05
claude "    " Claude Code    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
    }
}
```

```
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||  
Directory.Exists(fullPath)}");  
    }  
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `让 Claude Code 先探索` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `让 Claude Code 先探索` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

5.2 结构化提问

学习目标

- 理解 `结构化提问` 在本章主题中的具体作用。
- 能把 `结构化提问` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：目标、范围、限制、输出格式、模糊问题改写为工程任务、输出中要求文件路径和证据。

核心概念

结构化提问

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 5.2.1 目标、范围、限制、输出格式：先解释它解决的问题，再给出一个可观察的操作。
- 5.2.2 模糊问题改写为工程任务：先解释它解决的问题，再给出一个可观察的操作。
- 5.2.3 输出中要求文件路径和证据：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-05"
claude "    "

# macOS / Linux
```



```
cd ~/claude-code-handbook/book/examples/chapter-05
claude " " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `结构化提问` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `结构化提问` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

5.3 代码地图输出

学习目标

- 理解 `代码地图输出` 在本章主题中的具体作用。
- 能把 `代码地图输出` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：项目入口、模块边界、数据流和风险点。

核心概念

`代码地图输出`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `5.3.1
项目入口`：先解释它解决的问题，再给出一个可观察的操作。
- `5.3.2
模块边界`：先解释它解决的问题，再给出一个可观察的操作。
- `5.3.3 数据流和风险点`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。

2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-05"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-05
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `代码地图输出` 改写成包含目标、范围、限制、输出格式的提示词。

- 练习 2: 要求 Claude Code 先列计划, 不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令, 并记录验证结果。

检查清单

- [] 我能用自己的话解释`代码地图输出`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

5.4 反查与验证

学习目标

- 理解`反查与验证`在本章主题中的具体作用。
- 能把`反查与验证`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景, 并知道如何修正。
- 能说明本节三个重点: 抽查模型结论、要求引用源码、更新错误总结。

核心概念

`反查与验证`

的重点不是记住术语, 而是把它放回真实工程动作里理解。对于 Claude Code 来说, 一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解:

- `5.4.1`
抽查模型结论: 先解释它解决的问题, 再给出一个可观察的操作。

- 5.4.2

要求引用源码：先解释它解决的问题，再给出一个可观察的操作。

- 5.4.3

更新错误总结：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-05"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-05
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
    }
}
```

```

        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `反查与验证` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `反查与验证` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |

要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：生成 `docs/code-map.md`。
- 练习：抽查 5 个代码地图结论是否真实。
- 练习：把一次项目理解过程改写为可复用提示词。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `docs/code-map.md`
- `prompts/project-exploration.md`
- 项目理解检查清单

本章检查清单

- [] 我已经完成第 5 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“代码理解与项目地图”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Bug 定位与修复”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 6 章：Bug 定位与修复

> 所属篇章：第二篇：日常开发工作流 - 先把 Claude Code 用顺手

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

建立 Claude Code 修 Bug

的标准流程：先根据证据提出假设，再最小修改，最后用测试验证。

本章围绕“测试失败日志”展开。先让 Claude Code 读取错误、定位相关文件、提出多个假设，再要求它给出验证方法。修复阶段强调小步修改和测试先行。最后输出修复报告，为后续 Git 和 PR workflow 打基础。

学习目标

- 理解“Bug 定位与修复”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C++** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
```

```
Set-Location "D:\AI\claude code    \book\examples\chapter-06"
claude "    "Bug    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-06
claude "    "Bug    "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 6.1 从报错到假设
 - 6.1.1 如何提供错误日志
 - 6.1.2 要求模型列出可能原因
 - 6.1.3 为每个假设设计验证方法
- 6.2 最小修改原则
 - 6.2.1 不借 Bug 修复做大重构
 - 6.2.2 修改范围控制
 - 6.2.3 diff 审查
- 6.3 回归测试
 - 6.3.1 先写失败测试
 - 6.3.2 修复后重跑测试

- 6.3.3 防止只修表面现象
- 6.4 修复报告
- 6.4.1 Bug 原因
- 6.4.2 修改点
- 6.4.3 验证结果和风险

6.1 从报错到假设

学习目标

- 理解 `从报错到假设` 在本章主题中的具体作用。
- 能把 `从报错到假设` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：如何提供错误日志、要求模型列出可能原因、为每个假设设计验证方法。

核心概念

`从报错到假设`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `6.1.1 如何提供错误日志`：先解释它解决的问题，再给出一个可观察的操作。
- `6.1.2 要求模型列出可能原因`：先解释它解决的问题，再给出一个可观察的操作。

- 6.1.3 为每个假设设计验证方法：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-06"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-06
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `从报错到假设` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `从报错到假设` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

6.2 最小修改原则

学习目标

- 理解 `最小修改原则` 在本章主题中的具体作用。
- 能把 `最小修改原则` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：不借 Bug 修复、做大重构、修改范围控制、diff 审查。

核心概念

`最小修改原则`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于

Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 6.2.1 不借 Bug
修复做大重构：先解释它解决的问题，再给出一个可观察的操作。
- 6.2.2
修改范围控制：先解释它解决的问题，再给出一个可观察的操作。
- 6.2.3 diff 审查：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-06"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-06
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>
```

```
int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `最小修改原则` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `最小修改原则` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

6.3 回归测试

学习目标

- 理解 `回归测试` 在本章主题中的具体作用。
- 能把 `回归测试` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：先写失败测试、修复后重跑测试、防止只修表面现象。

核心概念

“回归测试”

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- “6.3.1
先写失败测试”：先解释它解决的问题，再给出一个可观察的操作。
- “6.3.2 修复后重跑测试”：先解释它解决的问题，再给出一个可观察的操作。
- “6.3.3 防止只修表面现象”：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-06"
claude " " " "

# macOS / Linux
```



```
cd ~/claude-code-handbook/book/examples/chapter-06
claude " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `回归测试` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `回归测试` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

6.4 修复报告

学习目标

- 理解 `修复报告` 在本章主题中的具体作用。
- 能把 `修复报告` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：Bug 原因、修改点、验证结果和风险。

核心概念

`修复报告`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `6.4.1 Bug`
原因：先解释它解决的问题，再给出一个可观察的操作。
- `6.4.2 修改点`：先解释它解决的问题，再给出一个可观察的操作。
- `6.4.3 验证结果和风险`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。

4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-06"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-06
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `修复报告` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `修复报告` 解决什么问题。

- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：修复一个空值、边界条件或类型转换 Bug。
- 练习：要求 Claude Code 在修复前添加失败测试。
- 练习：根据 diff 写修复报告。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- 一个带测试的 Bug 修复提交
- `notes/bugfix-report-template.md`

- Bug 定位提示词模板

本章检查清单

- ☐ 我已经完成第 6 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“Bug 定位与修复”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“重构、测试和文档”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 7 章：重构、测试和文档

> 所属篇章：第二篇：日常开发工作流 - 先把 Claude Code 用顺手

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者学会用 Claude Code

做中低风险重构，并用测试和文档控制质量。

本章把重构、测试和文档放在一起讲，因为三者在实际开发中通常不可分割。读者要学会让 Claude Code 先判断风险，再做小步修改，并在文档中留下行为和使用方式。代码审查部分为后续 SubAgent reviewer 做铺垫。

学习目标

- 理解“重构、测试和文档”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
```

```
Set-Location "D:\AI\claude code    \book\examples\chapter-07"
claude "    "    "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-07
claude "    "    "    "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 7.1 小步重构
 - 7.1.1 识别重复和复杂函数
 - 7.1.2 先建立保护性测试
 - 7.1.3 小提交、小 diff、小验证
- 7.2 测试完善
 - 7.2.1 正常路径测试
 - 7.2.2 异常路径测试
 - 7.2.3 边界条件测试
- 7.3 文档生成
 - 7.3.1 使用真实函数名和命令

- 7.3.2 写给新成员的运行文档
- 7.3.3 避免空泛文档
- 7.4 代码审查
- 7.4.1 按严重级别输出问题
- 7.4.2 关注行为变化和测试缺口
- 7.4.3 区分必须修复和建议优化

7.1 小步重构

学习目标

- 理解 `小步重构` 在本章主题中的具体作用。
- 能把 `小步重构` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：识别重复和复杂函数、先建立保护性测试、小提交、小 diff、小验证。

核心概念

`小步重构`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `7.1.1 识别重复和复杂函数`：先解释它解决的问题，再给出一个可观察的操作。
- `7.1.2 先建立保护性测试`：先解释它解决的问题，再给出一个可观察的操作。

- 7.1.3 小提交、小

diff、小验证：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-07"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-07
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
```

```
print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `小步重构` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `小步重构` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

7.2 测试完善

学习目标

- 理解 `测试完善` 在本章主题中的具体作用。
- 能把 `测试完善` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：正常路径测试、异常路径测试、边界条件测试。

核心概念

`测试完善`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于

Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 7.2.1

正常路径测试：先解释它解决的问题，再给出一个可观察的操作。

- 7.2.2

异常路径测试：先解释它解决的问题，再给出一个可观察的操作。

- 7.2.3

边界条件测试：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-07"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-07
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path
```

```
PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `测试完善` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `测试完善` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

7.3 文档生成

学习目标

- 理解 `文档生成` 在本章主题中的具体作用。
- 能把 `文档生成` 转化为一个可执行、可验证的 Claude Code 任务。

- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：使用真实函数名和命令、写给新成员的运行文档、避免空泛文档。

核心概念

“文档生成”

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- “7.3.1 使用真实函数名和命令”：先解释它解决的问题，再给出一个可观察的操作。
- “7.3.2 写给新成员的运行文档”：先解释它解决的问题，再给出一个可观察的操作。
- “7.3.3

避免空泛文档”：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" -Path "\book\examples\chapter-07"
claude " " "
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-07
claude "    "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `文档生成` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `文档生成` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

7.4 代码审查

学习目标

- 理解 `代码审查` 在本章主题中的具体作用。
- 能把 `代码审查` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：按严重级别输出问题、关注行为变化和测试缺口、区分必须修复和建议优化。

核心概念

`代码审查`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `7.4.1 按严重级别输出问题`：先解释它解决的问题，再给出一个可观察的操作。
- `7.4.2 关注行为变化和测试缺口`：先解释它解决的问题，再给出一个可观察的操作。
- `7.4.3 区分必须修复和建议优化`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。

2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-07"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-07
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `代码审查` 改写成包含目标、范围、限制、输出格式的提示词。

- 练习 2: 要求 Claude Code 先列计划, 不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令, 并记录验证结果。

检查清单

- [] 我能用自己的话解释`代码审查`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围, 明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成, 但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件, 并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例: 重构一个重复函数, 并保持测试通过。
- 练习: 让 Claude Code 为核心函数补 3 类测试。
- 练习: 对一次重构 diff 做审查。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- 一次小步重构提交
- 新增或改进的测试文件
- ``docs/module-usage.md``

本章检查清单

- ☐ 我已经完成第 7 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“重构、测试和文档”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Git 工作流与提交质量”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>

- Skills: <https://code.claude.com/docs/zh-CN/skills>

第 8 章：Git 工作流与提交质量

> 所属篇章：第二篇：日常开发工作流 - 先把 Claude Code 用顺手

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者学会用 Claude Code 辅助 Git 工作，但不让它破坏工作区。

本章强调 Claude Code 在 Git 中的正确角色：帮助理解 diff、生成提交说明、写 PR 描述和检查风险，而不是替人盲目执行破坏性命令。读者要建立“先看状态、再看 diff、再提交”的习惯。

学习目标

- 理解“Git 工作流与提交质量”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C#** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-08"
claude "Git"
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-08
claude "      "Git      "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 8.1 读懂 diff
 - 8.1.1 工作区状态检查
 - 8.1.2 按文件总结行为变化
 - 8.1.3 识别无关改动
- 8.2 提交信息
 - 8.2.1 Conventional Commit 基础
 - 8.2.2 从 diff 生成提交说明
 - 8.2.3 修改提交说明使其可审查
- 8.3 分支与 PR 描述
 - 8.3.1 分支命名

- 8.3.2 PR 背景、方案和验证
- 8.3.3 风险和回滚说明
- 8.4 避免 Git 灾难
- 8.4.1 不随意 reset
- 8.4.2 不覆盖用户未提交修改
- 8.4.3 高风险 Git 命令确认机制

8.1 读懂 diff

学习目标

- 理解 `读懂 diff` 在本章主题中的具体作用。
- 能把 `读懂 diff` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：工作区状态检查、按文件总结行为变化、识别无关改动。

核心概念

`读懂 diff`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `8.1.1 工作区状态检查`：先解释它解决的问题，再给出一个可观察的操作。
- `8.1.2 按文件总结行为变化`：先解释它解决的问题，再给出一个可观察的操作。

- 8.1.3

识别无关改动：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-08"
claude "    diff"

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-08
claude "    diff"
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `读懂 diff` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `读懂 diff` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

8.2 提交信息

学习目标

- 理解 `提交信息` 在本章主题中的具体作用。
- 能把 `提交信息` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：Conventional Commit 基础、从 diff 生成提交说明、修改提交说明使其可审查。

核心概念

`提交信息`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于

Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 8.2.1 Conventional Commit

基础：先解释它解决的问题，再给出一个可观察的操作。

- 8.2.2 从 diff

生成提交说明：先解释它解决的问题，再给出一个可观察的操作。

- 8.2.3 修改提交说明使其可审查：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-08"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-08
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
```

```
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `提交信息` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `提交信息` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

8.3 分支与 PR 描述

学习目标

- 理解 `分支与 PR 描述` 在本章主题中的具体作用。

- 能把`分支与 PR 描述`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：分支命名、PR 背景、方案和验证、风险和回滚说明。

核心概念

`分支与 PR 描述`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `8.3.1 分支命名`：先解释它解决的问题，再给出一个可观察的操作。
- `8.3.2 PR 背景、方案和验证`：先解释它解决的问题，再给出一个可观察的操作。
- `8.3.3 风险和回滚说明`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。

5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-08"
claude "    "    PR    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-08
claude "    "    PR    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `分支与 PR 描述` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `分支与 PR 描述` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

8.4 避免 Git 灾难

学习目标

- 理解 `避免 Git 灾难` 在本章主题中的具体作用。
- 能把 `避免 Git 灾难` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：不随意
reset、不覆盖用户未提交修改、高风险 Git 命令确认机制。

核心概念

`避免 Git 灾难`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `8.4.1 不随意
reset`：先解释它解决的问题，再给出一个可观察的操作。
- `8.4.2 不覆盖用户未提交修改`：先解释它解决的问题，再给出一个可观察的操作。
- `8.4.3 高风险 Git
命令确认机制`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-08"
claude "    "    Git    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-08
claude "    "    Git    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `避免 Git 灾难` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `避免 Git 灾难` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：让 Claude Code 根据当前 diff 生成提交说明和 PR 描述。
- 练习：找出 diff 中的无关格式化改动。
- 练习：列出 5 个不应随便授权的 Git 命令。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- ``docs/pr-template.md``
- ``prompts/git-diff-summary.md``
- Git 安全操作清单

本章检查清单

- ☐ 我已经完成第 8 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“Git 工作流与提交质量”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“从一次性问答到可复用工作流”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览: <https://code.claude.com/docs/zh-CN/overview>
- 记忆系统: <https://code.claude.com/docs/zh-CN/memory>
- Skills: <https://code.claude.com/docs/zh-CN/skills>

第 9 章：从一次性问答到可复用 workflow

> 所属篇章：第二篇：日常开发 workflow - 先把 Claude Code 用顺手

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

把前几章零散经验整理成可复用的个人 workflow。

本章让读者把“会问问题”升级为“有流程”。通过模板化提示词和固定输出格式，减少每次从头沟通的成本。重点是建立个人 workflow 文档，为后面的 `CLAUDE.md`、命令和 Skill 做基础。

学习目标

- 理解“从一次性问答到可复用 workflow”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 `**C++**` 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-09"
claude "        "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-09
claude "        "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 9.1 常见任务模板
 - 9.1.1 Bug 修复模板
 - 9.1.2 代码审查模板
 - 9.1.3 文档生成模板
- 9.2 输出格式控制
 - 9.2.1 表格输出
 - 9.2.2 清单输出
 - 9.2.3 JSON 或固定 Markdown 输出
- 9.3 验证优先
 - 9.3.1 验证命令必须出现
 - 9.3.2 失败时不粉饰结果

- 9.3.3 验证结果进入最终报告
- 9.4 建立个人工作流
- 9.4.1 日常开发流程
- 9.4.2 任务前检查
- 9.4.3 任务后复盘

9.1 常见任务模板

学习目标

- 理解 `常见任务模板` 在本章主题中的具体作用。
- 能把 `常见任务模板` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：Bug 修复模板、代码审查模板、文档生成模板。

核心概念

`常见任务模板`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `9.1.1 Bug 修复模板`：先解释它解决的问题，再给出一个可观察的操作。
- `9.1.2 代码审查模板`：先解释它解决的问题，再给出一个可观察的操作。

- 9.1.3

文档生成模板：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-09"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-09
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `常见任务模板` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `常见任务模板` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

9.2 输出格式控制

学习目标

- 理解 `输出格式控制` 在本章主题中的具体作用。
- 能把 `输出格式控制` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：表格输出、清单输出、JSON 或固定 Markdown 输出。

核心概念

`输出格式控制`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于

Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 9.2.1

表格输出：先解释它解决的问题，再给出一个可观察的操作。

- 9.2.2

清单输出：先解释它解决的问题，再给出一个可观察的操作。

- 9.2.3 JSON 或固定 Markdown

输出：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-09"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-09
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
```

```
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `输出格式控制` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `输出格式控制` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

9.3 验证优先

学习目标

- 理解 `验证优先` 在本章主题中的具体作用。
- 能把 `验证优先` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：验证命令必须出现、失败时不粉饰结果、验证结果进入最终报告。

核心概念

验证优先

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 9.3.1 验证命令必须出现：先解释它解决的问题，再给出一个可观察的操作。
- 9.3.2 失败时不粉饰结果：先解释它解决的问题，再给出一个可观察的操作。
- 9.3.3 验证结果进入最终报告：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-09"
claude " " " "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-09
claude " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `验证优先` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `验证优先` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

9.4 建立个人 workflow

学习目标

- 理解 `建立个人 workflow` 在本章主题中的具体作用。
- 能把 `建立个人 workflow` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：日常开发流程、任务前检查、任务后复盘。

核心概念

`建立个人 workflow`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `9.4.1`
日常开发流程：先解释它解决的问题，再给出一个可观察的操作。
- `9.4.2`
任务前检查：先解释它解决的问题，再给出一个可观察的操作。
- `9.4.3`
任务后复盘：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。

3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-09"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-09
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `建立个人工作流` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `建立个人工作流` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：把 Bug 修复流程写成可复用提示词模板。
- 练习：用同一模板完成 3 个不同任务。
- 练习：让 Claude Code 根据你的使用习惯改进模板。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `docs/my-claude-workflow.md`

- `prompts/bugfix.md`
- `prompts/review.md`
- `prompts/docs.md`

本章检查清单

- [] 我已经完成第 9 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“从一次性问答到可复用工作流”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Markdown 不是文档格式，而是 Agent 控制层”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

Markdown、记忆和本地知识库

第 10 章：Markdown

不是文档格式，而是 Agent 控制层

> 所属篇章：第三篇：Markdown、记忆和本地知识库

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者理解 Markdown 在 Claude Code

中不仅是给人看的文档，也是给 Agent 读取、遵守和执行的控制层。

本章以重写一段混乱项目说明为主线。读者先学习写给 Agent 的 Markdown 应该短、准、可执行，再把项目说明、规则、命令、文档索引分层。重点说明 Markdown 的质量会直接影响 Claude Code 的行为稳定性。

学习目标

- 理解“Markdown 不是文档格式，而是 Agent 控制层”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-10"
claude "      "Markdown          Agent    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-10
claude "      "Markdown          Agent    "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 10.1 Markdown 基础够用版
 - 10.1.1 标题层级
 - 10.1.2 列表、代码块和表格
 - 10.1.3 链接和文件路径
- 10.2 写给人和写给 Agent 的区别
 - 10.2.1 人类说明可以含糊
 - 10.2.2 Agent 规则必须可执行

- 10.2.3 必须、禁止、建议、例外
- 10.3 文档分层
 - 10.3.1 `README.md`
 - 10.3.2 `CLAUDE.md`
 - 10.3.3 `docs/rules/`
- 10.4 文档老化问题
 - 10.4.1 文档与代码不一致
 - 10.4.2 文档过长导致模型忽略重点
 - 10.4.3 建立文档更新机制

10.1 Markdown 基础够用版

学习目标

- 理解 `Markdown 基础够用版` 在本章主题中的具体作用。
- 能把 `Markdown 基础够用版` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：标题层级、列表、代码块和表格、链接和文件路径。

核心概念

`Markdown 基础够用版`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `10.1.1

标题层级：先解释它解决的问题，再给出一个可观察的操作。

- `10.1.2 列表、代码块和表格：先解释它解决的问题，再给出一个可观察的操作。

- `10.1.3 链接和文件路径：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-10"
claude "    "Markdown    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-10
claude "    "Markdown    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
```

```
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Markdown 基础够用版` 改写成一个包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Markdown 基础够用版` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

10.2 写给人和写给 Agent 的区别

学习目标

- 理解 `写给人和写给 Agent 的区别` 在本章主题中的具体作用。
- 能把 `写给人和写给 Agent 的区别` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：人类说明可以含糊、Agent 规则必须可执行、必须、禁止、建议、例外。

核心概念

“写给人和写给 Agent 的区别”

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- “10.2.1 人类说明可以含糊”：先解释它解决的问题，再给出一个可观察的操作。
- “10.2.2 Agent 规则必须可执行”：先解释它解决的问题，再给出一个可观察的操作。
- “10.2.3 必须、禁止、建议、例外”：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-10"
claude " " Agent " "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-10
claude " " Agent "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `写给人和写给 Agent 的区别` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `写给人和写给 Agent 的区别` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。

- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

10.3 文档分层

学习目标

- 理解 `文档分层` 在本章主题中的具体作用。
- 能把 `文档分层` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：`README.md`、`CLAUDE.md`、`docs/rules/`。

核心概念

`文档分层`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `10.3.1`
`README.md`：先解释它解决的问题，再给出一个可观察的操作。
- `10.3.2`
`CLAUDE.md`：先解释它解决的问题，再给出一个可观察的操作。
- `10.3.3`
`docs/rules/`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-10"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-10
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1: 把 `文档分层` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2: 要求 Claude Code 先列计划，不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `文档分层` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

10.4 文档老化问题

学习目标

- 理解 `文档老化问题` 在本章主题中的具体作用。
- 能把 `文档老化问题` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：文档与代码不一致、文档过长导致模型忽略重点、建立文档更新机制。

核心概念

`文档老化问题`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `10.4.1 文档与代码不一致`：先解释它解决的问题，再给出一个可观察的操作。
- `10.4.2 文档过长导致模型忽略重点`：先解释它解决的问题，再给出一个可观察的操作。
- `10.4.3 建立文档更新机制`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-10"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-10
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
```

```

    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `文档老化问题` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `文档老化问题` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |

要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：把一段口语化项目规范改成结构化 Markdown。
- 练习：将规则标注为“必须、禁止、建议、例外”。
- 练习：检查一个文档是否适合给 Agent 读取。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `docs/markdown-for-agent.md`
- Markdown 写作检查清单
- 一份改写后的项目规则文档

本章检查清单

- [] 我已经完成第 10 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“Markdown 不是文档格式，而是 Agent 控制层”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“`CLAUDE.md` 记忆系统”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 11 章：`CLAUDE.md`、Auto Memory 与导入机制

> 所属篇章：第三篇：Markdown、记忆和本地知识库

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者掌握 Claude Code 项目记忆的核心文件：`CLAUDE.md`。

本章从最小 `CLAUDE.md` 开始，只包含项目简介、运行命令、测试命令和基本风格。随后逐步加入个人偏好和项目规则，并展示臃肿规则如何降低效果。读者要学会让 Claude Code 读取并复述规则，验证它是否真的理解。

学习目标

- 理解“`CLAUDE.md`、Auto Memory 与导入机制”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 `**C#**` 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 ->

获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-11"
claude "    "`CLAUDE.md` Auto Memory    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-11
claude "    "`CLAUDE.md` Auto Memory    "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 11.1 `CLAUDE.md` 的作用
 - 11.1.1 项目目标
 - 11.1.2 常用命令
 - 11.1.3 编码和测试规则
- 11.2 个人偏好与项目规则
 - 11.2.1 个人习惯
 - 11.2.2 项目约束
 - 11.2.3 团队规范

- 11.3 好的 ``CLAUDE.md`` 写法
- 11.3.1 短而明确
- 11.3.2 能被执行
- 11.3.3 按任务场景组织
- 11.4 记忆维护
- 11.4.1 定期删除过期规则
- 11.4.2 用测试命令验证规则
- 11.4.3 记录规则变更原因

11.1 ``CLAUDE.md`` 的作用

学习目标

- 理解 ``CLAUDE.md`` 的作用` 在本章主题中的具体作用。
- 能把 ``CLAUDE.md`` 的作用` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：项目目标、常用命令、编码和测试规则。

核心概念

``CLAUDE.md`` 的作用`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `11.1.1
项目目标：先解释它解决的问题，再给出一个可观察的操作。

- 11.1.2

常用命令：先解释它解决的问题，再给出一个可观察的操作。

- 11.1.3 编码和测试规则：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-11"
claude "    ``CLAUDE.md`    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-11
claude "    ``CLAUDE.md`    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
    }
}
```

```
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||  
Directory.Exists(fullPath)}");  
    }  
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 ``CLAUDE.md` 的作用`
改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 ``CLAUDE.md` 的作用` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

11.2 个人偏好与项目规则

学习目标

- 理解 `个人偏好与项目规则` 在本章主题中的具体作用。
- 能把 `个人偏好与项目规则` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：个人习惯、项目约束、团队规范。

核心概念

个人偏好与项目规则

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 11.2.1

个人习惯：先解释它解决的问题，再给出一个可观察的操作。

- 11.2.2

项目约束：先解释它解决的问题，再给出一个可观察的操作。

- 11.2.3

团队规范：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-11"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-11
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `个人偏好与项目规则` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `个人偏好与项目规则` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

11.3 好的 `CLAUDE.md` 写法

学习目标

- 理解 `好的 `CLAUDE.md` 写法` 在本章主题中的具体作用。
- 能把 `好的 `CLAUDE.md` 写法` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：短而明确、能被执行、按任务场景组织。

核心概念

`好的 `CLAUDE.md` 写法`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `11.3.1
短而明确`：先解释它解决的问题，再给出一个可观察的操作。
- `11.3.2
能被执行`：先解释它解决的问题，再给出一个可观察的操作。
- `11.3.3 按任务场景组织`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。

3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-11"
claude "    " `CLAUDE.md`    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-11
claude "    " `CLAUDE.md`    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1: 把 `好的`CLAUDE.md` 写法`
改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2: 要求 Claude Code 先列计划，不直接修改文件。

- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `好的` `CLAUDE.md` 写法` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

11.4 记忆维护

学习目标

- 理解 `记忆维护` 在本章主题中的具体作用。
- 能把 `记忆维护` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：定期删除过期规则、用测试命令验证规则、记录规则变更原因。

核心概念

`记忆维护`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `11.4.1 定期删除过期规则`：先解释它解决的问题，再给出一个可观察的操作。
- `11.4.2 用测试命令验证规则`：先解释它解决的问题，再给出一个可观察的操作。

- `11.4.3 记录规则变更原因`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-11"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-11
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}
```



```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `记忆维护` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `记忆维护` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：为学习项目创建最小 `CLAUDE.md`。
- 练习：把个人偏好和项目规则拆开。
- 练习：让 Claude Code 审查 `CLAUDE.md` 是否空泛、重复、过期。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `CLAUDE.md`
- `notes/claude-memory-review.md`
- 个人规则与项目规则分层表

本章检查清单

- ☐ 我已经完成第 11 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“`CLAUDE.md`、Auto Memory 与导入机制”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“项目规则、`AGENTS.md`

和规则文件夹”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

深入专题：Auto Memory 与 `@imports`

掌握 `CLAUDE.md` 之后，还需要理解 Auto Memory 和导入机制。Auto Memory 的意义是把高频、稳定、跨任务都成立的偏好自动沉淀下来；`@imports` 的意义是把大型规则拆成多个小文件，避免一个 `CLAUDE.md` 变成不可维护的长文档。

建议做法：

- 项目根目录只保留最重要的 10-20 条规则。
- 语言、测试、发布、安全等规则拆入 `docs/claude/\*.md`。
- 在主规则里用导入方式组织，而不是复制粘贴。
- 每月审查一次记忆，删除过时规则。

检查清单：

- [] Auto Memory 中没有临时偏好。
- [] `CLAUDE.md` 不保存密钥、账号、内部 URL。
- [] 导入文件名称能表达用途。

第 12 章：项目规则、`AGENTS.md` 和规则文件夹

> 所属篇章：第三篇：Markdown、记忆和本地知识库

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

把单个 `CLAUDE.md` 扩展成多文件规则体系，并理解不同 Agent 工具之间规则文件的关系。

本章解决 `CLAUDE.md` 越写越大的问题。将测试、安全、Git、API、数据库等规则拆成独立文件，并通过索引让 Claude Code 按需读取。`AGENTS.md` 只做概念对照，重点仍是 Claude Code 的项目规则管理。

学习目标

- 理解“项目规则、`AGENTS.md` 和规则文件夹”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 ****C++**** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-12"
claude "        "        `AGENTS.md`        "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-12
claude "        "        `AGENTS.md`        "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 12.1 多文件规则架构
 - 12.1.1 为什么拆分规则
 - 12.1.2 `claude/rules/testing.md`
 - 12.1.3 `claude/rules/security.md`
- 12.2 条件化规则
 - 12.2.1 API 规则
 - 12.2.2 数据库规则
 - 12.2.3 前端或客户端规则

- 12.3 `AGENTS.md` 对照
- 12.3.1 共享规则
- 12.3.2 Claude Code 专用规则
- 12.3.3 多工具兼容写法
- 12.4 规则冲突处理
- 12.4.1 冲突来源
- 12.4.2 优先级说明
- 12.4.3 冲突测试

12.1 多文件规则架构

学习目标

- 理解 `多文件规则架构` 在本章主题中的具体作用。
- 能把 `多文件规则架构` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：为什么拆分规则、`.claude/rules/testing.md`、`.claude/rules/security.md`。

核心概念

`多文件规则架构`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `12.1.1 为什么拆分规则`：先解释它解决的问题，再给出一个可观察的操作。
- `12.1.2 .claude/rules/testing.md`：先解释它解决的问题，再给出一个可观察的操作。
- `12.1.3 .claude/rules/security.md`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-12"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-12
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
```

```
";  
    std::cout << "Exists: " << std::filesystem::exists(target) << "  
";  
    return 0;  
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`多文件规则架构`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`多文件规则架构`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

12.2 条件化规则

学习目标

- 理解`条件化规则`在本章主题中的具体作用。
- 能把`条件化规则`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：API 规则、数据库规则、前端或客户端规则。

核心概念

条件化规则

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 12.2.1 API

规则：先解释它解决的问题，再给出一个可观察的操作。

- 12.2.2

数据库规则：先解释它解决的问题，再给出一个可观察的操作。

- 12.2.3 前端或客户端规则：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-12"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-12
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `条件化规则` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `条件化规则` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

12.3 `AGENTS.md` 对照

学习目标

- 理解 ``AGENTS.md` 对照` 在本章主题中的具体作用。
- 能把 ``AGENTS.md` 对照` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：共享规则、Claude Code 专用规则、多工具兼容写法。

核心概念

``AGENTS.md` 对照`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `12.3.1
共享规则`：先解释它解决的问题，再给出一个可观察的操作。
- `12.3.2 Claude Code
专用规则`：先解释它解决的问题，再给出一个可观察的操作。
- `12.3.3 多工具兼容写法`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。

4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-12"
claude "    ``AGENTS.md`    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-12
claude "    ``AGENTS.md`    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 ``AGENTS.md` 对照` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 ``AGENTS.md` 对照` 解决什么问题。

- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

12.4 规则冲突处理

学习目标

- 理解 `规则冲突处理` 在本章主题中的具体作用。
- 能把 `规则冲突处理` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：冲突来源、优先级说明、冲突测试。

核心概念

`规则冲突处理`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `12.4.1`
冲突来源：先解释它解决的问题，再给出一个可观察的操作。
- `12.4.2`
优先级说明：先解释它解决的问题，再给出一个可观察的操作。
- `12.4.3`
冲突测试：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-12"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-12
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`规则冲突处理`改写成包含目标、范围、限制、输出格式的提示词。

- 练习 2: 要求 Claude Code 先列计划, 不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令, 并记录验证结果。

检查清单

- [] 我能用自己的话解释`规则冲突处理`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围, 明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成, 但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件, 并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例: 把一个 200 行规则文件拆成索引和多个规则文件。
- 练习: 设计 API、数据库、测试三类条件化规则。
- 练习: 制造两条冲突规则, 并写优先级解决方案。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `\.claude/rules/testing.md`
- `\.claude/rules/security.md`
- `\.claude/rules/git-workflow.md`
- 规则优先级说明

本章检查清单

- ☐ 我已经完成第 12 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“项目规则、`\AGENTS.md` 和规则文件夹”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“本地知识库”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>

- 记忆系统: <https://code.claude.com/docs/zh-CN/memory>
- Skills: <https://code.claude.com/docs/zh-CN/skills>

第 13 章：本地知识库与 ADR 与 ADR

> 所属篇章：第三篇：Markdown、记忆和本地知识库与 ADR

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

帮助读者建立给 Claude Code 使用的本地知识库与 ADR，把经验、决策和项目背景沉淀下来。

本章把本地知识库与 ADR 定位为“可搜索、可引用、可维护”的长期上下文，而不是随手堆文件。读者要学会为每篇知识写摘要、关键词、适用场景，并让 Claude Code 通过索引找到资料。

学习目标

- 理解“本地知识库与 ADR”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-13"
claude "        "        ADR"        "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-13
claude "        "        ADR"        "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 13.1 什么适合放进知识库
- 13.1.1 项目决策
- 13.1.2 排障经验
- 13.1.3 外部资料摘要
- 13.2 知识库索引
- 13.2.1 `knowledge/index.md`
- 13.2.2 摘要和关键词
- 13.2.3 适用场景说明
- 13.3 决策记录 ADR

- 13.3.1 背景
- 13.3.2 决策
- 13.3.3 后果和替代方案
- 13.4 从对话沉淀知识
- 13.4.1 成功任务复盘
- 13.4.2 Bug 排查手册
- 13.4.3 更新知识库的提示词

13.1 什么适合放进知识库

学习目标

- 理解 `什么适合放进知识库` 在本章主题中的具体作用。
- 能把 `什么适合放进知识库` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：项目决策、排障经验、外部资料摘要。

核心概念

`什么适合放进知识库`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `13.1.1
- 项目决策`：先解释它解决的问题，再给出一个可观察的操作。

- 13.1.2

故障经验：先解释它解决的问题，再给出一个可观察的操作。

- 13.1.3

外部资料摘要：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-13"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-13
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
```

```
        "suffix": target.suffix,
    }

    if __name__ == "__main__":
        print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `什么适合放进知识库` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `什么适合放进知识库` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

13.2 知识库索引

学习目标

- 理解 `知识库索引` 在本章主题中的具体作用。
- 能把 `知识库索引` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：`knowledge/index.md`、摘要和关键词、适用场景说明。

核心概念

知识库索引

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 13.2.1 knowledge/index.md：先解释它解决的问题，再给出一个可观察的操作。
- 13.2.2
摘要和关键词：先解释它解决的问题，再给出一个可观察的操作。
- 13.2.3
适用场景说明：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-13"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-13
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `知识库索引` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `知识库索引` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

13.3 决策记录 ADR

学习目标

- 理解 `决策记录 ADR` 在本章主题中的具体作用。
- 能把 `决策记录 ADR` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：背景、决策、后果和替代方案。

核心概念

`决策记录 ADR`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `13.3.1 背景`：先解释它解决的问题，再给出一个可观察的操作。
- `13.3.2 决策`：先解释它解决的问题，再给出一个可观察的操作。
- `13.3.3 后果和替代方案`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**Python**`。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。

4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-13"
claude "    "    ADR"    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-13
claude "    "    ADR"    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `决策记录 ADR` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `决策记录 ADR` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

13.4 从对话沉淀知识

学习目标

- 理解 `从对话沉淀知识` 在本章主题中的具体作用。
- 能把 `从对话沉淀知识` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：成功任务复盘、Bug 排查手册、更新知识库的提示词。

核心概念

`从对话沉淀知识`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `13.4.1 成功任务复盘`：先解释它解决的问题，再给出一个可观察的操作。
- `13.4.2 Bug 排查手册`：先解释它解决的问题，再给出一个可观察的操作。

- `13.4.3 更新知识库的提示词`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-13"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-13
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
```

```
print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `从对话沉淀知识` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `从对话沉淀知识` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：建立 `knowledge/` 目录和索引。
- 练习：写一份 ADR。
- 练习：把一次 Bug 修复过程沉淀为排查手册。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `knowledge/index.md`
- `knowledge/adr/0001-example.md`
- `knowledge/troubleshooting/bugfix-playbook.md`

本章检查清单

- ☐ 我已经完成第 13 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“本地知识库与 ADR”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“记忆系统的反模式”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

深入专题：ADR 如何进入本地知识库

ADR（Architecture Decision Record）适合记录“为什么这样设计”，而不是记录“代码现在长什么样”。Claude Code 读取 ADR 后，能更稳定地理解架构约束，避免在重构时反复提出已经被否定的方案。

推荐 ADR 模板：

```
# ADR-0001:

##
##
##
##
## Claude Code
```

练习：让 Claude Code 为一次技术选型生成 ADR 草稿，再要求它列出风险和反例。

第 14 章：记忆系统的反模式

> 所属篇章：第三篇：Markdown、记忆和本地知识库

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者知道记忆系统不是越多越好，错误记忆、过期规则和敏感信息都会降低 Claude Code 的可靠性。

本章以清理一份臃肿 `CLAUDE.md` 为案例。读者要学会删除“请写高质量代码”这类空泛规则，保留“运行测试命令是 X”“禁止修改 Y 目录”这类可执行规则。同时加入敏感信息扫描意识。

学习目标

- 理解“记忆系统的反模式”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 `**C#**` 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-14"
```



```
claude "      "      "      "
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-14
claude "      "      "      "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 14.1 臃肿记忆
 - 14.1.1 空泛规则
 - 14.1.2 重复规则
 - 14.1.3 与当前任务无关的规则
- 14.2 过期记忆
 - 14.2.1 文档与代码不一致
 - 14.2.2 命令已失效
 - 14.2.3 依赖和目录变化
- 14.3 隐含假设
 - 14.3.1 默认风格没有写清

- 14.3.2 验收标准没有写清
- 14.3.3 团队约定没有写清
- 14.4 记忆权限边界
- 14.4.1 不写密钥
- 14.4.2 不写私密 token
- 14.4.3 不写不可共享内部信息

14.1 臃肿记忆

学习目标

- 理解 `臃肿记忆` 在本章主题中的具体作用。
- 能把 `臃肿记忆` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：空泛规则、重复规则、与当前任务无关的规则。

核心概念

`臃肿记忆`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `14.1.1`
空泛规则：先解释它解决的问题，再给出一个可观察的操作。
- `14.1.2`
重复规则：先解释它解决的问题，再给出一个可观察的操作。

- 14.1.3 与当前任务无关的规则：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-14"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-14
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `臃肿记忆` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `臃肿记忆` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

14.2 过期记忆

学习目标

- 理解 `过期记忆` 在本章主题中的具体作用。
- 能把 `过期记忆` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：文档与代码不一致、命令已失效、依赖和目录变化。

核心概念

`过期记忆`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于

Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `14.2.1 文档与代码不一致`：先解释它解决的问题，再给出一个可观察的操作。
- `14.2.2 命令已失效`：先解释它解决的问题，再给出一个可观察的操作。
- `14.2.3 依赖和目录变化`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-14"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-14
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
```

```
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `过期记忆` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `过期记忆` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

14.3 隐含假设

学习目标

- 理解 `隐含假设` 在本章主题中的具体作用。

- 能把`隐含假设`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：默认风格没有写清、验收标准没有写清、团队约定没有写清。

核心概念

`隐含假设`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `14.3.1 默认风格没有写清`：先解释它解决的问题，再给出一个可观察的操作。
- `14.3.2 验收标准没有写清`：先解释它解决的问题，再给出一个可观察的操作。
- `14.3.3 团队约定没有写清`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
```

```
Set-Location "D:\AI\claude code    \book\examples\chapter-14"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-14
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `隐含假设` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `隐含假设` 解决什么问题。

- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

14.4 记忆权限边界

学习目标

- 理解 `记忆权限边界` 在本章主题中的具体作用。
- 能把 `记忆权限边界` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：不写密钥、不写私密 token、不写不可共享内部信息。

核心概念

`记忆权限边界`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `14.4.1 不写密钥`：先解释它解决的问题，再给出一个可观察的操作。
- `14.4.2 不写私密 token`：先解释它解决的问题，再给出一个可观察的操作。
- `14.4.3 不写不可共享内部信息`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-14"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-14
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `记忆权限边界` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `记忆权限边界` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：将臃肿 `CLAUDE.md` 压缩为清晰版本。
- 练习：找出 10 条无效或过期规则。
- 练习：检查知识库是否包含敏感信息。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `notes/memory-antipatterns.md`
- 清理后的 `CLAUDE.md`
- 记忆系统安全检查清单

本章检查清单

- ☐ 我已经完成第 14 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“记忆系统的反模式”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Claude Code

提示词基本功”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

提示词、Opus 4.7 和模型驾驭

第 15 章：Claude Code

提示词基本功

- > 所属篇章：第四篇：提示词、Opus 4.7 和模型驾驭
- > 主案例语言：C++
- > 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

建立 Claude Code 提示词的基本格式，让读者能稳定表达目标、范围、限制和输出要求。

本章把“提示词”从灵感表达变成工程规格。读者要学会把“帮我改一下”改成“目标是什么、只允许改哪里、不能做什么、输出什么、如何验证”。本章不讲复杂技巧，重点训练稳定表达。

学习目标

- 理解“Claude Code 提示词基本功”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C++** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 ->

获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-15"
claude "        "Claude Code        "        "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-15
claude "        "Claude Code        "        "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 15.1 任务说明四要素
 - 15.1.1 目标
 - 15.1.2 范围
 - 15.1.3 限制
 - 15.1.4 输出格式
- 15.2 约束和验收标准
 - 15.2.1 什么算完成
 - 15.2.2 什么不能做
 - 15.2.3 如何验证完成
- 15.3 让模型先问问题

- 15.3.1 需求不清时停止
- 15.3.2 关键问题优先
- 15.3.3 问完再计划
- 15.4 控制输出粒度
- 15.4.1 只要结论
- 15.4.2 结论加证据
- 15.4.3 完整方案

15.1 任务说明四要素

学习目标

- 理解 `任务说明四要素` 在本章主题中的具体作用。
- 能把 `任务说明四要素` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：目标、范围、限制。

核心概念

`任务说明四要素`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `15.1.1 目标`：先解释它解决的问题，再给出一个可观察的操作。
- `15.1.2 范围`：先解释它解决的问题，再给出一个可观察的操作。
- `15.1.3 限制`：先解释它解决的问题，再给出一个可观察的操作。

- 15.1.4

输出格式：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-15"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-15
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `任务说明四要素` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `任务说明四要素` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

15.2 约束和验收标准

学习目标

- 理解 `约束和验收标准` 在本章主题中的具体作用。
- 能把 `约束和验收标准` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：什么算完成、什么不能做、如何验证完成。

核心概念

`约束和验收标准`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资

料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 15.2.1

什么算完成：先解释它解决的问题，再给出一个可观察的操作。

- 15.2.2

什么不能做：先解释它解决的问题，再给出一个可观察的操作。

- 15.2.3

如何验证完成：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-15"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-15
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>
```

```
int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `约束和验收标准` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `约束和验收标准` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

15.3 让模型先问问题

学习目标

- 理解 `让模型先问问题` 在本章主题中的具体作用。
- 能把 `让模型先问问题` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：需求不清时停止、关键问题优先、问完再计划。

核心概念

让模型先问问题

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 15.3.1 需求不清时停止：先解释它解决的问题，再给出一个可观察的操作。
- 15.3.2 关键问题优先：先解释它解决的问题，再给出一个可观察的操作。
- 15.3.3 问完再计划：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-15"
claude " " " "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-15
claude " " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `让模型先问问题` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `让模型先问问题` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

15.4 控制输出粒度

学习目标

- 理解 `控制输出粒度` 在本章主题中的具体作用。
- 能把 `控制输出粒度` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：只要结论、结论加证据、完整方案。

核心概念

`控制输出粒度`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `15.4.1`
只要结论`：先解释它解决的问题，再给出一个可观察的操作。
- `15.4.2`
结论加证据`：先解释它解决的问题，再给出一个可观察的操作。
- `15.4.3`
完整方案`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。

3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-15"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-15
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `控制输出粒度` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`控制输出粒度`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：把 5 个模糊请求改成工程任务。
- 练习：为一个需求写验收标准。
- 练习：设计一个“信息不足时先提问”的提示词。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `prompts/task-brief-template.md`

- 验收标准模板
- 常见模糊提示改写表

本章检查清单

- [] 我已经完成第 15 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“Claude Code 提示词基本功”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“计划模式与探索模式”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 16 章：计划模式与探索模式

> 所属篇章：第四篇：提示词、Opus 4.7 和模型驾驭

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

教导者在复杂任务中先让 Claude Code 探索和计划，再决定是否执行。

本章用一个多文件重构任务说明计划模式价值。探索阶段只读不改，计划阶段列出修改范围和风险，执行阶段严格按计划推进。读者要学会在复杂任务中把“想法”变成“可审查计划”。

学习目标

- 理解“计划模式与探索模式”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 ****Python**** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-16"
```

```
claude "      "      "      "
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-16
claude "      "      "      "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 16.1 什么时候先计划
 - 16.1.1 多文件修改
 - 16.1.2 架构调整
 - 16.1.3 高风险命令或数据变更
- 16.2 探索而不是全量阅读
 - 16.2.1 先搜索关键词
 - 16.2.2 限定读取文件数量
 - 16.2.3 输出探索证据
- 16.3 计划评审
 - 16.3.1 风险点

- 16.3.2 修改文件
- 16.3.3 验证命令
- 16.4 计划到执行的交接
- 16.4.1 用编号追踪步骤
- 16.4.2 执行偏离时停下来
- 16.4.3 最终报告引用计划

16.1 什么时候先计划

学习目标

- 理解 `什么时候先计划` 在本章主题中的具体作用。
- 能把 `什么时候先计划` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：多文件修改、架构调整、高风险命令或数据变更。

核心概念

`什么时候先计划`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `16.1.1`
多文件修改：先解释它解决的问题，再给出一个可观察的操作。
- `16.1.2`
架构调整：先解释它解决的问题，再给出一个可观察的操作。

- `16.1.3 高风险命令或数据变更`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-16"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-16
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
```

```
print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `什么时候先计划` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `什么时候先计划` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

16.2 探索而不是全量阅读

学习目标

- 理解 `探索而不是全量阅读` 在本章主题中的具体作用。
- 能把 `探索而不是全量阅读` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：先搜索关键词、限定读取文件数量、输出探索证据。

核心概念

探索而不是全量阅读

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 16.2.1

先搜索关键词：先解释它解决的问题，再给出一个可观察的操作。

- 16.2.2 限定读取文件数量：先解释它解决的问题，再给出一个可观察的操作。

- 16.2.3

输出探索证据：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-16"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-16
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `探索而不是全量阅读` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `探索而不是全量阅读` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

16.3 计划评审

学习目标

- 理解 `计划评审` 在本章主题中的具体作用。
- 能把 `计划评审` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：风险点、修改文件、验证命令。

核心概念

`计划评审`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `16.3.1 风险点`：先解释它解决的问题，再给出一个可观察的操作。
- `16.3.2
修改文件`：先解释它解决的问题，再给出一个可观察的操作。
- `16.3.3
验证命令`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-16"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-16
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `计划评审` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `计划评审` 解决什么问题。

- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

16.4 计划到执行的交接

学习目标

- 理解 `计划到执行的交接` 在本章主题中的具体作用。
- 能把 `计划到执行的交接` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：用编号追踪步骤、执行偏离时停下来、最终报告引用计划。

核心概念

`计划到执行的交接`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `16.4.1 用编号追踪步骤`：先解释它解决的问题，再给出一个可观察的操作。
- `16.4.2 执行偏离时停下来`：先解释它解决的问题，再给出一个可观察的操作。
- `16.4.3 最终报告引用计划`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-16"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-16
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `计划到执行的交接` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `计划到执行的交接` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：让 Claude Code 为多文件重构写计划，但暂不执行。
- 练习：审查计划中的遗漏风险。
- 练习：执行时要求每步引用计划编号。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `prompts/plan-first.md`
- 一份多文件修改计划
- 计划评审清单

本章检查清单

- ☐ 我已经完成第 16 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“计划模式与探索模式”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“驯服 Opus

4.7: 模型、思考预算和思考预算”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览: <https://code.claude.com/docs/zh-CN/overview>
- 记忆系统: <https://code.claude.com/docs/zh-CN/memory>
- Skills: <https://code.claude.com/docs/zh-CN/skills>

第 17 章：驯服 Opus

4.7：模型、思考预算、Fast Mode 与目标驱动

> 所属篇章：第四篇：提示词、Opus 4.7 和模型驾驭

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

帮助读者理解什么时候使用更强模型、更高思考预算和深度思考提示，而不是无差别消耗高成本资源。

本章从“任务价值”和“失败成本”角度讲模型选择。Opus 4.7 的使用重点放在复杂架构、跨模块 Bug、安全风险和高价值决策上。读者要建立成本意识：简单文档修改不需要重型模型，复杂问题也不能只靠一句深度思考提示。

学习目标

- 理解“驯服 Opus 4.7：模型、思考预算和思考预算”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C#** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-17"
claude "    "    Opus 4.7    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-17
claude "    "    Opus 4.7    "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 17.1 模型选择
 - 17.1.1 简单任务与复杂任务
 - 17.1.2 高风险任务与低风险任务
 - 17.1.3 成本、速度和质量的权衡
- 17.2 Effort 与 Thinking
 - 17.2.1 思考预算的用途

- 17.2.2 思考预算与输出质量
- 17.2.3 何时不需要高思考
 - 17.3 `ultrathink` 类提示
- 17.3.1 适合复杂推理的问题
- 17.3.2 不适合简单重复任务
- 17.3.3 深度分析后的验证要求
 - 17.4 Opus 4.7 提示词建议
- 17.4.1 架构评审提示词
- 17.4.2 Bug 深挖提示词
- 17.4.3 重构计划提示词

17.1 模型选择

学习目标

- 理解 `模型选择` 在本章主题中的具体作用。
- 能把 `模型选择` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：简单任务与复杂任务、高风险任务与低风险任务、成本、速度和质量的权衡。

核心概念

`模型选择`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `17.1.1 简单任务与复杂任务`：先解释它解决的问题，再给出一个可观察的操作。
- `17.1.2 高风险任务与低风险任务`：先解释它解决的问题，再给出一个可观察的操作。
- `17.1.3 成本、速度和质量的权衡`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-17"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-17
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
```

```

    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `模型选择` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `模型选择` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

17.2 Effort 与 Thinking

学习目标

- 理解 `Effort 与 Thinking` 在本章主题中的具体作用。
- 能把 `Effort 与 Thinking` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：思考预算的用途、思考预算与输出质量、何时不需要高思考。

核心概念

`Effort 与 Thinking`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `17.2.1 思考预算的用途`：先解释它解决的问题，再给出一个可观察的操作。
- `17.2.2 思考预算与输出质量`：先解释它解决的问题，再给出一个可观察的操作。
- `17.2.3 何时不需要高思考`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-17"
claude "    "Effort    Thinking"    "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-17
claude "    "Effort    Thinking"    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Effort 与 Thinking` 改写成一个包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Effort 与 Thinking` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

17.3 `ultrathink` 类提示

学习目标

- 理解 ``ultrathink` 类提示` 在本章主题中的具体作用。
- 能把 ``ultrathink` 类提示` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：适合复杂推理的问题、不适合简单重复任务、深度分析后的验证要求。

核心概念

``ultrathink` 类提示`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `17.3.1 适合复杂推理的问题`：先解释它解决的问题，再给出一个可观察的操作。
- `17.3.2 不适合简单重复任务`：先解释它解决的问题，再给出一个可观察的操作。
- `17.3.3 深度分析后的验证要求`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-17"
claude "    "`ultrathink`    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-17
claude "    "`ultrathink`    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1: 把 ``ultrathink` 类提示` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2: 要求 Claude Code 先列计划，不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 ``ultrathink` 类提示` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

17.4 Opus 4.7 提示词建议

学习目标

- 理解 `Opus 4.7 提示词建议` 在本章主题中的具体作用。
- 能把 `Opus 4.7 提示词建议` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：架构评审提示词、Bug 深挖提示词、重构计划提示词。

核心概念

`Opus 4.7 提示词建议`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `17.4.1 架构评审提示词`：先解释它解决的问题，再给出一个可观察的操作。
- `17.4.2 Bug
深挖提示词`：先解释它解决的问题，再给出一个可观察的操作。
- `17.4.3 重构计划提示词`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-17"
claude "    "Opus 4.7    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-17
claude "    "Opus 4.7    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
```

```

    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Opus 4.7 提示词建议` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Opus 4.7 提示词建议` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |

在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |

要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：同一复杂 Bug 分别用普通提示和深度分析提示处理。
- 练习：为 5 类任务选择模型和思考预算。
- 练习：记录输出质量、耗时和成本感知差异。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `prompts/opus47-architecture-review.md`
- `prompts/opus47-deep-bug-analysis.md`
- 模型选择决策表

本章检查清单

- [] 我已经完成第 17 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“驯服 Opus 4.7：模型、思考预算和思考预算”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“复杂工程任务的提示词编排”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 18 章：复杂工程任务的提示词编排

- > 所属篇章：第四篇：提示词、Opus 4.7 和模型驾驭
- > 主案例语言：C++
- > 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者掌握把复杂任务拆成多阶段提示词的能力。

本章以一个小功能需求为例，把任务拆成澄清、探索、计划、实现、验证、总结六步。读者要学会让 Claude Code 在每个阶段产出可检查结果，而不是一次性完成所有动作。

学习目标

- 理解“复杂工程任务的提示词编排”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 ****C++**** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。


```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-18"
claude "        "        "        "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-18
claude "        "        "        "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 18.1 分阶段提示
 - 18.1.1 需求澄清
 - 18.1.2 项目探索
 - 18.1.3 方案计划
 - 18.1.4 执行验证
- 18.2 角色提示
 - 18.2.1 架构师视角
 - 18.2.2 测试工程师视角
 - 18.2.3 安全审计视角
- 18.3 反例和边界
 - 18.3.1 方案不适用场景

- 18.3.2 边界条件
- 18.3.3 风险清单
- 18.4 提示词版本管理
- 18.4.1 `prompts/` 目录
- 18.4.2 提示词适用场景
- 18.4.3 提示词版本和变更记录

18.1 分阶段提示

学习目标

- 理解 `分阶段提示` 在本章主题中的具体作用。
- 能把 `分阶段提示` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：需求澄清、项目探索、方案计划。

核心概念

`分阶段提示`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `18.1.1`
需求澄清：先解释它解决的问题，再给出一个可观察的操作。
- `18.1.2`
项目探索：先解释它解决的问题，再给出一个可观察的操作。

- 18.1.3

方案计划：先解释它解决的问题，再给出一个可观察的操作。

- 18.1.4

执行验证：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-18"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-18
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
```

```
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `分阶段提示` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `分阶段提示` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

18.2 角色提示

学习目标

- 理解 `角色提示` 在本章主题中的具体作用。
- 能把 `角色提示` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：架构师视角、测试工程师视角、安全审计视角。

核心概念

角色提示

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 18.2.1

架构师视角：先解释它解决的问题，再给出一个可观察的操作。

- 18.2.2 测试工程师视角：先解释它解决的问题，再给出一个可观察的操作。

- 18.2.3

安全审计视角：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-18"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-18
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `角色提示` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `角色提示` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

18.3 反例和边界

学习目标

- 理解 `反例和边界` 在本章主题中的具体作用。

- 能把`反例和边界`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：方案不适用场景、边界条件、风险清单。

核心概念

`反例和边界`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `18.3.1 方案不适用场景`：先解释它解决的问题，再给出一个可观察的操作。
- `18.3.2
边界条件`：先解释它解决的问题，再给出一个可观察的操作。
- `18.3.3
风险清单`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
```

```
Set-Location "D:\AI\claude code    \book\examples\chapter-18"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-18
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`反例和边界`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`反例和边界`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

18.4 提示词版本管理

学习目标

- 理解 `提示词版本管理` 在本章主题中的具体作用。
- 能把 `提示词版本管理` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：`prompts/` 目录、提示词适用场景、提示词版本和变更记录。

核心概念

`提示词版本管理`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `18.4.1 prompts/` 目录：先解释它解决的问题，再给出一个可观察的操作。
- `18.4.2 提示词适用场景`：先解释它解决的问题，再给出一个可观察的操作。
- `18.4.3 提示词版本和变更记录`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-18"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-18
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `提示词版本管理` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。

- 练习 3: 让 Claude Code 给出验证命令, 并记录验证结果。

检查清单

- [] 我能用自己的话解释 `提示词版本管理` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围, 明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成, 但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件, 并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例: 为一个新功能编排完整提示词链路。
- 练习: 让三个角色分别审查同一个方案。
- 练习: 为提示词模板建立版本记录。

练习答案不要求唯一, 但必须满足三个条件: 任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `prompts/feature-pipeline.md`
- `prompts/role-review.md`
- `prompts/CHANGELOG.md`

本章检查清单

- [] 我已经完成第 18 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“复杂工程任务的提示词编排”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“上下文窗口与文件选择”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

上下文、Token 和成本控制

第 19 章：上下文窗口与文件选择

> 所属篇章：第五篇：上下文、Token 和成本控制

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者理解 Claude Code

的回答质量受上下文质量影响，学会控制进入上下文的内容。

本章通过一个项目分析任务展示“读太多”和“读太少”都会出问题。读者要学会通过搜索、索引、摘要和交接文档控制上下文质量。

学习目标

- 理解“上下文窗口与文件选择”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-19
claude " " " "
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-19
claude "      "      "      "
```

案例中的最小代码对象如下:

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 19.1 上下文是什么
 - 19.1.1 对话内容
 - 19.1.2 文件内容
 - 19.1.3 工具输出
 - 19.1.4 规则和记忆
- 19.2 文件选择策略
 - 19.2.1 先搜索后读取
 - 19.2.2 只读相关文件
 - 19.2.3 大文件先摘要
- 19.3 上下文污染
 - 19.3.1 过时文档

- 19.3.2 无关日志
- 19.3.3 冲突规则
- 19.4 上下文交接
- 19.4.1 当前目标
- 19.4.2 已完成事项
- 19.4.3 关键文件和风险

19.1 上下文是什么

学习目标

- 理解`上下文是什么`在本章主题中的具体作用。
- 能把`上下文是什么`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：对话内容、文件内容、工具输出。

核心概念

`上下文是什么`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `19.1.1`
对话内容：先解释它解决的问题，再给出一个可观察的操作。
- `19.1.2`
文件内容：先解释它解决的问题，再给出一个可观察的操作。

- 19.1.3

工具输出：先解释它解决的问题，再给出一个可观察的操作。

- 19.1.4

规则和记忆：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-19"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-19
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
```

```
        "suffix": target.suffix,
    }

    if __name__ == "__main__":
        print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `上下文是什么` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `上下文是什么` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

19.2 文件选择策略

学习目标

- 理解 `文件选择策略` 在本章主题中的具体作用。
- 能把 `文件选择策略` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：先搜索后读取、只读相关文件、大文件先摘要。

核心概念

文件选择策略

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 19.2.1

先搜索后读取：先解释它解决的问题，再给出一个可观察的操作。

- 19.2.2

只读相关文件：先解释它解决的问题，再给出一个可观察的操作。

- 19.2.3

大文件先摘要：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-19"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-19
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`文件选择策略`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`文件选择策略`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

19.3 上下文污染

学习目标

- 理解`上下文污染`在本章主题中的具体作用。
- 能把`上下文污染`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：过时文档、无关日志、冲突规则。

核心概念

`上下文污染`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `19.3.1`
过时文档：先解释它解决的问题，再给出一个可观察的操作。
- `19.3.2`
无关日志：先解释它解决的问题，再给出一个可观察的操作。
- `19.3.3`
冲突规则：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。

3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-19"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-19
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `上下文污染` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。

- 练习 3: 让 Claude Code 给出验证命令, 并记录验证结果。

检查清单

- [] 我能用自己的话解释 `上下文污染` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

19.4 上下文交接

学习目标

- 理解 `上下文交接` 在本章主题中的具体作用。
- 能把 `上下文交接` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景, 并知道如何修正。
- 能说明本节三个重点: 当前目标、已完成事项、关键文件和风险。

核心概念

`上下文交接`

的重点不是记住术语, 而是把它放回真实工程动作里理解。对于 Claude Code 来说, 一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解:

- `19.4.1`
当前目标: 先解释它解决的问题, 再给出一个可观察的操作。
- `19.4.2`
已完成事项: 先解释它解决的问题, 再给出一个可观察的操作。

- `19.4.3 关键文件和风险`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-19"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-19
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
```



```
print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `上下文交接` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `上下文交接` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：要求 Claude Code 在最多读取 8 个文件内定位问题。
- 练习：清理一次对话中的无关上下文。
- 练习：写一份新会话交接摘要。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `notes/context-handoff.md`
- 文件选择策略清单
- 上下文污染排查表

本章检查清单

- ☐ 我已经完成第 19 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“上下文窗口与文件选择”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“压缩、恢复和长会话管理”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 20 章：压缩、检查点与长会话管理

> 所属篇章：第五篇：上下文、Token 和成本控制

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者掌握长任务中如何保存状态、减少信息丢失，并在新会话中继续工作。

本章重点讲长任务管理。读者要学会主动写 `handoff.md`，在任务阶段结束时保存状态，而不是等上下文丢失后再补救。长任务应拆成多个可验证阶段。

学习目标

- 理解“压缩、恢复和长会话管理”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 `**C#**` 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 ->

获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-20"
claude "    "    "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-20
claude "    "    "    "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 20.1 自动压缩
 - 20.1.1 长会话为什么会压缩
 - 20.1.2 压缩可能丢失什么
 - 20.1.3 如何检查当前状态
- 20.2 手动摘要
 - 20.2.1 任务目标摘要
 - 20.2.2 修改文件摘要
 - 20.2.3 待办和风险摘要

- 20.3 Checkpoint 思维
- 20.3.1 需求确认点
- 20.3.2 修改完成点
- 20.3.3 验证完成点
- 20.4 长任务拆分
- 20.4.1 按模块拆分
- 20.4.2 按 PR 拆分
- 20.4.3 按风险拆分

20.1 自动压缩

学习目标

- 理解 `自动压缩` 在本章主题中的具体作用。
- 能把 `自动压缩` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：长会话为什么会压缩、压缩可能丢失什么、如何检查当前状态。

核心概念

`自动压缩`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `20.1.1 长会话为什么会压缩`：先解释它解决的问题，再给出一个可观察的操作。

- `20.1.2 压缩可能丢失什么`：先解释它解决的问题，再给出一个可观察的操作。
- `20.1.3 如何检查当前状态`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-20"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-20
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
    }
}
```

```
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||  
Directory.Exists(fullPath)}");  
    }  
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `自动压缩` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `自动压缩` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

20.2 手动摘要

学习目标

- 理解 `手动摘要` 在本章主题中的具体作用。
- 能把 `手动摘要` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：任务目标摘要、修改文件摘要、待办和风险摘要。

核心概念

手动摘要

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 20.2.1

任务目标摘要：先解释它解决的问题，再给出一个可观察的操作。

- 20.2.2

修改文件摘要：先解释它解决的问题，再给出一个可观察的操作。

- 20.2.3 待办和风险摘要：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-20"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-20
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`手动摘要`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`手动摘要`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

20.3 Checkpoint 思维

学习目标

- 理解 `Checkpoint 思维` 在本章主题中的具体作用。
- 能把 `Checkpoint 思维` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：需求确认点、修改完成点、验证完成点。

核心概念

`Checkpoint 思维`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `20.3.1`
需求确认点：先解释它解决的问题，再给出一个可观察的操作。
- `20.3.2`
修改完成点：先解释它解决的问题，再给出一个可观察的操作。
- `20.3.3`
验证完成点：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。

3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-20"
claude "    "Checkpoint    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-20
claude "    "Checkpoint    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Checkpoint 思维` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。

- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Checkpoint 思维` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

20.4 长任务拆分

学习目标

- 理解 `长任务拆分` 在本章主题中的具体作用。
- 能把 `长任务拆分` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：按模块拆分、按 PR 拆分、按风险拆分。

核心概念

`长任务拆分`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `20.4.1
按模块拆分`：先解释它解决的问题，再给出一个可观察的操作。
- `20.4.2 按 PR
拆分`：先解释它解决的问题，再给出一个可观察的操作。

- 20.4.3

按风险拆分：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-20"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-20
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `长任务拆分` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `长任务拆分` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：完成一半任务后写交接摘要，再开启新会话继续。
- 练习：为一个大重构拆分 3 个阶段。
- 练习：检查压缩后是否保留关键文件和决策。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `notes/handoff-template.md`
- 长任务拆分计划
- Checkpoint 检查清单

本章检查清单

- ☐ 我已经完成第 20 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“压缩、恢复和长会话管理”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Token 节省与成本优化”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 21 章：Token 节省与成本优化

> 所属篇章：第五篇：上下文、Token 和成本控制

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者学会从文件读取、输出长度、模型选择和上下文结构上节省 token。

本章建立成本意识，但不把 token 优化变成省略必要信息。读者要掌握三个原则：少读但读对，少说但说准，稳定信息长期沉淀，临时信息短期使用。

学习目标

- 理解“Token 节省与成本优化”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C++** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-21"
```

```
claude "      "Token      "
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-21
claude "      "Token      "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 21.1 Token 花在哪里
 - 21.1.1 用户输入
 - 21.1.2 文件内容
 - 21.1.3 工具输出
 - 21.1.4 模型回答
- 21.2 少读文件
 - 21.2.1 用搜索定位
 - 21.2.2 用索引导航
 - 21.2.3 大日志先摘要
- 21.3 少输出废话
 - 21.3.1 只要结论
 - 21.3.2 限定格式

- 21.3.3 避免重复解释
- 21.4 缓存友好写法
- 21.4.1 稳定规则和临时任务分离
- 21.4.2 文档结构稳定
- 21.4.3 避免频繁改动大上下文

21.1 Token 花在哪里

学习目标

- 理解 `Token 花在哪里` 在本章主题中的具体作用。
- 能把 `Token 花在哪里` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：用户输入、文件内容、工具输出。

核心概念

`Token 花在哪里`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `21.1.1`
用户输入：先解释它解决的问题，再给出一个可观察的操作。
- `21.1.2`
文件内容：先解释它解决的问题，再给出一个可观察的操作。
- `21.1.3`
工具输出：先解释它解决的问题，再给出一个可观察的操作。

- 21.1.4

模型回答：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-21"
claude "    "Token    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-21
claude "    "Token    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Token 花在哪里` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Token 花在哪里` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

21.2 少读文件

学习目标

- 理解 `少读文件` 在本章主题中的具体作用。
- 能把 `少读文件` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：用搜索定位、用索引导航、大日志先摘要。

核心概念

`少读文件`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 21.2.1

用搜索定位：先解释它解决的问题，再给出一个可观察的操作。

- 21.2.2

用索引导航：先解释它解决的问题，再给出一个可观察的操作。

- 21.2.3

大日志先摘要：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-21"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-21
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
```

```

std::filesystem::path target{"README.md"};
std::cout << "Path: " << std::filesystem::absolute(target) << "
";
std::cout << "Exists: " << std::filesystem::exists(target) << "
";
return 0;
}

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `少读文件` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `少读文件` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

21.3 少输出废话

学习目标

- 理解 `少输出废话` 在本章主题中的具体作用。
- 能把 `少输出废话` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：只要结论、限定格式、避免重复解释。

核心概念

少输出废话

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 21.3.1

只要结论：先解释它解决的问题，再给出一个可观察的操作。

- 21.3.2

限定格式：先解释它解决的问题，再给出一个可观察的操作。

- 21.3.3

避免重复解释：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-21"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-21
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`少输出废话`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`少输出废话`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

21.4 缓存友好写法

学习目标

- 理解 `缓存友好写法` 在本章主题中的具体作用。
- 能把 `缓存友好写法` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：稳定规则和临时任务分离、文档结构稳定、避免频繁改动大上下文。

核心概念

`缓存友好写法`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `21.4.1 稳定规则和临时任务分离`：先解释它解决的问题，再给出一个可观察的操作。
- `21.4.2 文档结构稳定`：先解释它解决的问题，再给出一个可观察的操作。
- `21.4.3 避免频繁改动大上下文`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。

4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-21"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-21
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`缓存友好写法`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`缓存友好写法`解决什么问题。

- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：同一任务分别用“全量读取”和“搜索定位”执行，对比效率。
- 练习：把长日志压缩成结构化故障摘要。
- 练习：为常见任务制定模型和 token 策略。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `docs/token-budget.md`
- `prompts/concise-output.md`

- 成本优化检查清单

本章检查清单

- [] 我已经完成第 21 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“Token 节省与成本优化”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Skills 基础”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

扩展系统 - Skills、Commands、SubAgents、Hooks、MCP、Plugins 、个性化

第 22 章：Skills 基础

> 所属篇章：第六篇：扩展系统 -

Skills、Commands、SubAgents、Hooks、MCP、Plugins

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者掌握 Skill

的用途和最小结构，把重复知识和流程封装成可复用能力。

本章从一个最小 `SKILL.md` 开始，解释 Skill 的触发依赖描述，而不是靠用户每次手动复制长提示。随后加入引用资料和模板，展示渐进式披露如何节省 token。

学习目标

- 理解“Skills 基础”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
```



```
Set-Location "D:\AI\claude code    \book\examples\chapter-22"
claude "    "Skills    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-22
claude "    "Skills    "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 22.1 Skill 解决什么问题
- 22.1.1 重复任务
- 22.1.2 专业知识
- 22.1.3 团队标准
- 22.2 `SKILL.md` 结构
- 22.2.1 名称和描述
- 22.2.2 使用步骤
- 22.2.3 资源文件引用
- 22.3 渐进式披露
- 22.3.1 主文件保持短小

- 22.3.2 详细规则放入 `references/`
- 22.3.3 模板放入 `templates/`
- 22.4 Skill 案例
- 22.4.1 代码审查 Skill
- 22.4.2 API 文档 Skill
- 22.4.3 团队规范 Skill

22.1 Skill 解决什么问题

学习目标

- 理解 `Skill 解决什么问题` 在本章主题中的具体作用。
- 能把 `Skill 解决什么问题` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：重复任务、专业知识、团队标准。

核心概念

`Skill 解决什么问题`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `22.1.1`
重复任务：先解释它解决的问题，再给出一个可观察的操作。
- `22.1.2`
专业知识：先解释它解决的问题，再给出一个可观察的操作。

- 22.1.3

团队标准：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-22"
claude "    "Skill                "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-22
claude "    "Skill                "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
```

```
print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Skill 解决什么问题` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Skill 解决什么问题` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

22.2 `SKILL.md` 结构

学习目标

- 理解 ``SKILL.md` 结构` 在本章主题中的具体作用。
- 能把 ``SKILL.md` 结构` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：名称和描述、使用步骤、资源文件引用。

核心概念

``SKILL.md` 结构`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于

Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 22.2.1

名称和描述：先解释它解决的问题，再给出一个可观察的操作。

- 22.2.2

使用步骤：先解释它解决的问题，再给出一个可观察的操作。

- 22.2.3

资源文件引用：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-22"
claude "    "`SKILL.md`    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-22
claude "    "`SKILL.md`    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path
```

```
PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 ``SKILL.md` 结构`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 ``SKILL.md` 结构`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

22.3 渐进式披露

学习目标

- 理解 `渐进式披露` 在本章主题中的具体作用。

- 能把`渐进式披露`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：主文件保持短小、详细规则放入`references/`、模板放入`templates/`。

核心概念

`渐进式披露`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `22.3.1 主文件保持短小`：先解释它解决的问题，再给出一个可观察的操作。
- `22.3.2 详细规则放入`
`references/`：先解释它解决的问题，再给出一个可观察的操作。
- `22.3.3 模板放入`
`templates/`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。

5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-22"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-22
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `渐进式披露` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `渐进式披露` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

22.4 Skill 案例

学习目标

- 理解 `Skill 案例` 在本章主题中的具体作用。
- 能把 `Skill 案例` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：代码审查 Skill、API 文档 Skill、团队规范 Skill。

核心概念

`Skill 案例`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `22.4.1 代码审查`
Skill：先解释它解决的问题，再给出一个可观察的操作。
- `22.4.2 API 文档`
Skill：先解释它解决的问题，再给出一个可观察的操作。
- `22.4.3 团队规范`
Skill：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-22"
claude "    "Skill    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-22
claude "    "Skill    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Skill 案例` 改写成一个包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Skill 案例` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：创建 `code-reviewing/SKILL.md`。
- 练习：把长规则拆成 `references/`。
- 练习：用 Skill 生成一份 API 文档。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `.claude/skills/code-reviewing/SKILL.md`
- `.claude/skills/api-documenting/SKILL.md`
- Skill 设计检查清单

本章检查清单

- [] 我已经完成第 22 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“Skills 基础”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“自定义命令与任务型 Skill”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 23 章：自定义命令与任务型 Skill

> 所属篇章：第六篇：扩展系统 -

Skills、Commands、SubAgents、Hooks、MCP、Plugins

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者掌握 slash command 的使用场景，并理解命令与 Skill 的边界。

本章把常用任务从提示词模板升级为命令。读者要知道命令适合固定流程，比如审查 diff、生成提交说明、解释文件；Skill 适合需要长期知识、规范和模板的任务。

学习目标

- 理解“自定义命令与任务型 Skill”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C#** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
```

```
Set-Location "D:\AI\claude code    \book\examples\chapter-23"
claude "    "    Skill"    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-23
claude "    "    Skill"    "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 23.1 Slash Commands
 - 23.1.1 ``/review``
 - 23.1.2 ``/commit``
 - 23.1.3 ``/explain``
- 23.2 命令与 Skill 的区别
 - 23.2.1 固定流程用命令
 - 23.2.2 专业知识用 Skill
 - 23.2.3 复杂组合谨慎设计
- 23.3 参数化命令

- 23.3.1 文件路径参数
- 23.3.2 输出格式参数
- 23.3.3 默认行为和错误提示
- 23.4 命令库管理
- 23.4.1 ``.claude/commands/``
- 23.4.2 命令分组
- 23.4.3 命令索引文档

23.1 Slash Commands

学习目标

- 理解 ``Slash Commands`` 在本章主题中的具体作用。
- 能把 ``Slash Commands`` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：``/review``、``/commit``、``/explain``。

核心概念

``Slash Commands``

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- ``23.1.1``
 - ``/review``：先解释它解决的问题，再给出一个可观察的操作。

- 23.1.2

`/commit`：先解释它解决的问题，再给出一个可观察的操作。

- 23.1.3

`/explain`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-23"
claude "    "Slash Commands"    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-23
claude "    "Slash Commands"    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
    }
}
```

```
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||  
Directory.Exists(fullPath)}");  
    }  
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Slash Commands` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Slash Commands` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

23.2 命令与 Skill 的区别

学习目标

- 理解 `命令与 Skill 的区别` 在本章主题中的具体作用。
- 能把 `命令与 Skill 的区别` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：固定流程用命令、专业知识用 Skill、复杂组合谨慎设计。

核心概念

命令与 Skill 的区别

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 23.2.1 固定流程用命令：先解释它解决的问题，再给出一个可观察的操作。
- 23.2.2 专业知识用 Skill：先解释它解决的问题，再给出一个可观察的操作。
- 23.2.3 复杂组合谨慎设计：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-23"
claude "    " Skill    "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-23
claude "    " Skill "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `命令与 Skill 的区别` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `命令与 Skill 的区别` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

23.3 参数化命令

学习目标

- 理解 `参数化命令` 在本章主题中的具体作用。
- 能把 `参数化命令` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：文件路径参数、输出格式参数、默认行为和错误提示。

核心概念

`参数化命令`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `23.3.1
文件路径参数`：先解释它解决的问题，再给出一个可观察的操作。
- `23.3.2
输出格式参数`：先解释它解决的问题，再给出一个可观察的操作。
- `23.3.3 默认行为和错误提示`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-23"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-23
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1: 把 `参数化命令` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2: 要求 Claude Code 先列计划，不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `参数化命令` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

23.4 命令库管理

学习目标

- 理解 `命令库管理` 在本章主题中的具体作用。
- 能把 `命令库管理` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：`.claude/commands/`、命令分组、命令索引文档。

核心概念

`命令库管理`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 23.4.1 `.claude/commands/`：先解释它解决的问题，再给出一个可观察的操作。
- 23.4.2
命令分组：先解释它解决的问题，再给出一个可观察的操作。
- 23.4.3
命令索引文档：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-23"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-23
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
```



```

    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `命令库管理` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `命令库管理` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |

在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |

要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：创建 `/review`、`/commit`、`/todo` 三个命令。
- 练习：给 `/analyze` 添加路径参数。
- 练习：把一个复杂命令改造成 Skill。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `.claude/commands/review.md`
- `.claude/commands/commit.md`
- `.claude/commands/explain.md`
- 命令库索引

本章检查清单

- [] 我已经完成第 23 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。

- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“自定义命令与任务型 Skill”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“SubAgents 基础”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 24 章：SubAgents 基础

> 所属篇章：第六篇：扩展系统 -

Skills、Commands、SubAgents、Hooks、MCP、Plugins

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者理解子代理的价值：隔离上下文、限制权限、承担专职任务。

本章先创建一个只读 code-reviewer，再创建 test-runner 或 log-analyzer。重点是让读者理解子代理不只是“角色扮演”，而是权限、上下文和输出格式的工程边界。

学习目标

- 理解“SubAgents 基础”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C++** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-24"
```

```
claude "          "SubAgents          "
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-24
claude "          "SubAgents          "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 24.1 子代理解决什么问题
 - 24.1.1 专职角色
 - 24.1.2 上下文隔离
 - 24.1.3 权限边界
- 24.2 子代理配置
 - 24.2.1 名称和描述
 - 24.2.2 系统提示
 - 24.2.3 工具权限
- 24.3 高噪声任务隔离
 - 24.3.1 测试运行器
 - 24.3.2 日志分析器
 - 24.3.3 只返回结论

- 24.4 子代理反模式
- 24.4.1 简单任务不用拆
- 24.4.2 职责不能重叠
- 24.4.3 子代理输出必须可用

24.1 子代理解决什么问题

学习目标

- 理解`子代理解决什么问题`在本章主题中的具体作用。
- 能把`子代理解决什么问题`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：专职角色、上下文隔离、权限边界。

核心概念

`子代理解决什么问题`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `24.1.1
专职角色`：先解释它解决的问题，再给出一个可观察的操作。
- `24.1.2
上下文隔离`：先解释它解决的问题，再给出一个可观察的操作。
- `24.1.3
权限边界`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-24"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-24
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1: 把 `子代理` 解决什么问题` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2: 要求 Claude Code 先列计划，不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `子代理` 解决什么问题` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

24.2 子代理配置

学习目标

- 理解 `子代理配置` 在本章主题中的具体作用。
- 能把 `子代理配置` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：名称和描述、系统提示、工具权限。

核心概念

`子代理配置`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 24.2.1

名称和描述：先解释它解决的问题，再给出一个可观察的操作。

- 24.2.2

系统提示：先解释它解决的问题，再给出一个可观察的操作。

- 24.2.3

工具权限：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-24"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-24
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
```

```
";  
    std::cout << "Exists: " << std::filesystem::exists(target) << "  
";  
    return 0;  
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`子代理配置`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`子代理配置`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

24.3 高噪声任务隔离

学习目标

- 理解`高噪声任务隔离`在本章主题中的具体作用。
- 能把`高噪声任务隔离`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：测试运行器、日志分析器、只返回结论。

核心概念

高噪声任务隔离

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 24.3.1

测试运行器：先解释它解决的问题，再给出一个可观察的操作。

- 24.3.2

日志分析器：先解释它解决的问题，再给出一个可观察的操作。

- 24.3.3

只返回结论：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-24"
claude " "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-24
claude " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `高噪声任务隔离` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `高噪声任务隔离` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

24.4 子代理反模式

学习目标

- 理解 `子代理反模式` 在本章主题中的具体作用。

- 能把`子代理反模式`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：简单任务不用拆、职责不能重叠、子代理输出必须可用。

核心概念

`子代理反模式`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `24.4.1 简单任务不用拆`：先解释它解决的问题，再给出一个可观察的操作。
- `24.4.2 职责不能重叠`：先解释它解决的问题，再给出一个可观察的操作。
- `24.4.3 子代理输出必须可用`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。

5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-24"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-24
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`子代理反模式`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`子代理反模式`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。

- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：创建 ``.claude/agents/code-reviewer.md``。
- 练习：限制 code-reviewer 只读不改。
- 练习：用 log-analyzer 处理长日志。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- ``.claude/agents/code-reviewer.md``
- ``.claude/agents/test-runner.md``
- 子代理职责表

本章检查清单

- [] 我已经完成第 24 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“SubAgents 基础”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“多 Agent 协作”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 25 章：多 Agent 协作与 Agent Teams与 Agent Teams

> 所属篇章：第六篇：扩展系统 -

Skills、Commands、SubAgents、Hooks、MCP、Plugins

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者从单个子代理升级到并行探索、流水线和 Agent Teams。

本章展示多 Agent 协作与 Agent Teams不是越多越好。并行探索适合互不依赖的分析任务，流水线适合上下游明确的任务。读者要学会设计交接格式，让一个代理的输出能被下一个代理使用。

学习目标

- 理解“多 Agent 协作与 Agent Teams”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 ->

获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-25"
claude "        " Agent    Agent Teams"        "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-25
claude "        " Agent    Agent Teams"        "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 25.1 并行探索
 - 25.1.1 API 视角
 - 25.1.2 数据库视角
 - 25.1.3 认证或权限视角
- 25.2 流水线编排
 - 25.2.1 Bug 定位
 - 25.2.2 Bug 修复
 - 25.2.3 验证和报告

- 25.3 Agent Teams
 - 25.3.1 reviewer
 - 25.3.2 tester
 - 25.3.3 doc-writer
- 25.4 多 Agent 成本控制
 - 25.4.1 并行只用于独立任务
 - 25.4.2 输出必须短而结构化
 - 25.4.3 主会话负责整合

25.1 并行探索

学习目标

- 理解 `并行探索` 在本章主题中的具体作用。
- 能把 `并行探索` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：API 视角、数据库视角、认证或权限视角。

核心概念

`并行探索`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `25.1.1 API 视角`：先解释它解决的问题，再给出一个可观察的操作。

- 25.1.2

数据库视角：先解释它解决的问题，再给出一个可观察的操作。

- 25.1.3 认证或权限视角：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-25"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-25
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
```

```
        "suffix": target.suffix,
    }

    if __name__ == "__main__":
        print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `并行探索` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `并行探索` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

25.2 流水线编排

学习目标

- 理解 `流水线编排` 在本章主题中的具体作用。
- 能把 `流水线编排` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：Bug 定位、Bug 修复、验证和报告。

核心概念

流水线编排

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 25.2.1 Bug

定位：先解释它解决的问题，再给出一个可观察的操作。

- 25.2.2 Bug

修复：先解释它解决的问题，再给出一个可观察的操作。

- 25.2.3

验证和报告：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-25
claude " "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-25
claude " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `流水线编排` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `流水线编排` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

25.3 Agent Teams

学习目标

- 理解 `Agent Teams` 在本章主题中的具体作用。
- 能把 `Agent Teams` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：reviewer、tester、doc-writer。

核心概念

`Agent Teams`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `25.3.1`
reviewer：先解释它解决的问题，再给出一个可观察的操作。
- `25.3.2 tester`：先解释它解决的问题，再给出一个可观察的操作。
- `25.3.3`
doc-writer：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。

5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-25"
claude "    "Agent Teams"    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-25
claude "    "Agent Teams"    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Agent Teams` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Agent Teams` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

25.4 多 Agent 成本控制

学习目标

- 理解 `多 Agent 成本控制` 在本章主题中的具体作用。
- 能把 `多 Agent 成本控制` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：并行只用于独立任务、输出必须短而结构化、主会话负责整合。

核心概念

`多 Agent 成本控制`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `25.4.1 并行只用于独立任务`：先解释它解决的问题，再给出一个可观察的操作。
- `25.4.2 输出必须短而结构化`：先解释它解决的问题，再给出一个可观察的操作。
- `25.4.3 主会话负责整合`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-25"
claude "    " Agent    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-25
claude "    " Agent    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`多 Agent 成本控制`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`多 Agent 成本控制`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：三个探索代理分别分析 API、数据库、认证问题。
- 练习：设计 Bug 修复流水线。
- 练习：比较单 Agent 和多 Agent 的成本与效果。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `.claude/agents/api-explorer.md``
- `.claude/agents/db-explorer.md``
- `.claude/agents/auth-explorer.md``
- `.docs/agent-pipeline.md``

本章检查清单

- ☐ 我已经完成第 25 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“多 Agent 协作与 Agent Teams”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Hooks 事件驱动自动化”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

深入专题：Agent Teams

多 Agent 协作不是“多开几个模型一起想”，而是把任务拆成角色明确、输入输出契约明确的流水线。Agent Teams 的核心是固定团队角色，例如 planner、bug-locator、patch-writer、test-runner、reviewer。

最小团队示例：

- planner：只读需求，输出计划。
- bug-locator：只读日志和代码，输出嫌疑文件。
- patch-writer：只改允许文件。
- test-runner：只运行测试，输出失败摘要。
- reviewer：只审查 diff，不直接修改。

成本规则：只有任务之间可以并行且上下文不互相污染时，才使用多 Agent。

第 26 章：Hooks 事件驱动自动化

> 所属篇章：第六篇：扩展系统 -

Skills、Commands、SubAgents、Hooks、MCP、Plugins

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者学会用 Hook 在工具调用前后加入安全和质量控制。

本章将 Hook 定位为“工程护栏”。安全 Hook 防止危险操作，质量 Hook 提醒格式化和测试，Stop Hook 防止任务没有验证就结束。读者要理解 Hook 不应该过度复杂，否则会干扰正常开发。

学习目标

- 理解“Hooks 事件驱动自动化”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C#** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
```

```
Set-Location "D:\AI\claude code \book\examples\chapter-26"
claude "Hooks"

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-26
claude "Hooks"
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 26.1 Hook 是什么
 - 26.1.1 工具调用前
 - 26.1.2 工具调用后
 - 26.1.3 任务结束前
- 26.2 安全 Hook
 - 26.2.1 阻止危险删除
 - 26.2.2 阻止读取密钥
 - 26.2.3 阻止越权命令
- 26.3 质量 Hook

- 26.3.1 自动格式化
- 26.3.2 自动 lint
- 26.3.3 自动测试
- 26.4 Stop Hook
- 26.4.1 结束前检查验证结果
- 26.4.2 缺少测试时提醒
- 26.4.3 输出最终报告

26.1 Hook 是什么

学习目标

- 理解 `Hook 是什么` 在本章主题中的具体作用。
- 能把 `Hook 是什么` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：工具调用前、工具调用后、任务结束前。

核心概念

`Hook 是什么`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `26.1.1`
工具调用前：先解释它解决的问题，再给出一个可观察的操作。

- 26.1.2

工具调用后：先解释它解决的问题，再给出一个可观察的操作。

- 26.1.3

任务结束前：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-26"
claude "    "Hook    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-26
claude "    "Hook    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
    }
}
```

```
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||  
Directory.Exists(fullPath)}");  
    }  
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Hook 是什么` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Hook 是什么` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

26.2 安全 Hook

学习目标

- 理解 `安全 Hook` 在本章主题中的具体作用。
- 能把 `安全 Hook` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：阻止危险删除、阻止读取密钥、阻止越权命令。

核心概念

安全 Hook

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 26.2.1
阻止危险删除：先解释它解决的问题，再给出一个可观察的操作。
- 26.2.2
阻止读取密钥：先解释它解决的问题，再给出一个可观察的操作。
- 26.2.3
阻止越权命令：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-26"
claude " " Hook"
# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-26
claude "    " Hook"
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `安全 Hook` 改写成一个包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `安全 Hook` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

26.3 质量 Hook

学习目标

- 理解 `质量 Hook` 在本章主题中的具体作用。
- 能把 `质量 Hook` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：自动格式化、自动 lint、自动测试。

核心概念

`质量 Hook`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `26.3.1
自动格式化`：先解释它解决的问题，再给出一个可观察的操作。
- `26.3.2 自动
lint`：先解释它解决的问题，再给出一个可观察的操作。
- `26.3.3
自动测试`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。

2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-26"
claude "    "    Hook"    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-26
claude "    "    Hook"    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `质量 Hook` 改写成包含目标、范围、限制、输出格式的提示词。

- 练习 2: 要求 Claude Code 先列计划, 不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令, 并记录验证结果。

检查清单

- [] 我能用自己的话解释 `质量 Hook` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

26.4 Stop Hook

学习目标

- 理解 `Stop Hook` 在本章主题中的具体作用。
- 能把 `Stop Hook` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景, 并知道如何修正。
- 能说明本节三个重点: 结束前检查验证结果、缺少测试时提醒、输出最终报告。

核心概念

`Stop Hook`

的重点不是记住术语, 而是把它放回真实工程动作里理解。对于 Claude Code 来说, 一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解:

- `26.4.1 结束前检查验证结果`: 先解释它解决的问题, 再给出一个可观察的操作。

- `26.4.2 缺少测试时提醒`：先解释它解决的问题，再给出一个可观察的操作。

- `26.4.3

输出最终报告`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-26"
claude "    "Stop Hook"                "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-26
claude "    "Stop Hook"                "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
    }
}
```

```
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||  
Directory.Exists(fullPath)}");  
    }  
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Stop Hook` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Stop Hook` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |

要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：创建阻止删除关键目录的安全 Hook。
- 练习：创建结束前检查测试结果的 Stop Hook。
- 练习：故意跳过测试，观察 Hook 是否提醒。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `\.claude/hooks/safety-check.md`
- `\.claude/hooks/quality-check.md`
- Hook 风险控制表

本章检查清单

- [] 我已经完成第 26 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“Hooks 事件驱动自动化”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“MCP 基础与外部工具连接”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览: <https://code.claude.com/docs/zh-CN/overview>
- 记忆系统: <https://code.claude.com/docs/zh-CN/memory>
- Skills: <https://code.claude.com/docs/zh-CN/skills>

第 27 章：MCP 基础与外部工具连接

> 所属篇章：第六篇：扩展系统 -

Skills、Commands、SubAgents、Hooks、MCP、Plugins

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者理解 MCP

是连接外部系统的协议，并掌握基本配置和安全边界。

本章不把 MCP 当成神秘高级功能，而是从“Claude Code 需要访问外部资料”这个问题出发。读者要学会配置、检查、限制和审计 MCP。

学习目标

- 理解“MCP 基础与外部工具连接”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 `**C++**` 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
```

```
Set-Location "D:\AI\claude code    \book\examples\chapter-27"
claude "      "MCP      "      "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-27
claude "      "MCP      "      "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 27.1 MCP 解决什么问题
 - 27.1.1 本地工具
 - 27.1.2 数据库和 API
 - 27.1.3 知识库和 issue 系统
- 27.2 MCP 配置
 - 27.2.1 Server 启动方式
 - 27.2.2 配置文件位置
 - 27.2.3 工具可用性检查
- 27.3 MCP 安全
 - 27.3.1 最小权限
 - 27.3.2 只读优先

- 27.3.3 外部数据当作数据而非指令
- 27.4 MCP 案例
- 27.4.1 连接本地知识库
- 27.4.2 连接 issue 系统
- 27.4.3 基于外部资料生成计划

27.1 MCP 解决什么问题

学习目标

- 理解 `MCP 解决什么问题` 在本章主题中的具体作用。
- 能把 `MCP 解决什么问题` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：本地工具、数据库和 API、知识库和 issue 系统。

核心概念

`MCP 解决什么问题`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `27.1.1
本地工具`：先解释它解决的问题，再给出一个可观察的操作。
- `27.1.2 数据库和
API`：先解释它解决的问题，再给出一个可观察的操作。

- 27.1.3 知识库和 issue

系统：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-27"
claude "    "MCP    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-27
claude "    "MCP    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```


可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `MCP 解决什么问题` 改写成一个包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `MCP 解决什么问题` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

27.2 MCP 配置

学习目标

- 理解 `MCP 配置` 在本章主题中的具体作用。
- 能把 `MCP 配置` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：Server 启动方式、配置文件位置、工具可用性检查。

核心概念

`MCP 配置`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资

料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 27.2.1 Server

启动方式：先解释它解决的问题，再给出一个可观察的操作。

- 27.2.2

配置文件位置：先解释它解决的问题，再给出一个可观察的操作。

- 27.2.3 工具可用性检查：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-27"
claude "    "MCP    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-27
claude "    "MCP    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>
```

```
int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `MCP 配置` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `MCP 配置` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

27.3 MCP 安全

学习目标

- 理解 `MCP 安全` 在本章主题中的具体作用。
- 能把 `MCP 安全` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：最小权限、只读优先、外部数据当作数据而非指令。

核心概念

“MCP 安全”

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- “27.3.1
最小权限”：先解释它解决的问题，再给出一个可观察的操作。
- “27.3.2
只读优先”：先解释它解决的问题，再给出一个可观察的操作。
- “27.3.3 外部数据当作数据而非指令”：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-27"
claude " "MCP " "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-27
claude " "MCP "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `MCP 安全` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `MCP 安全` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

27.4 MCP 案例

学习目标

- 理解 `MCP 案例` 在本章主题中的具体作用。
- 能把 `MCP 案例` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：连接本地知识库、连接 issue 系统、基于外部资料生成计划。

核心概念

`MCP 案例`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `27.4.1 连接本地知识库`：先解释它解决的问题，再给出一个可观察的操作。
- `27.4.2 连接 issue 系统`：先解释它解决的问题，再给出一个可观察的操作。
- `27.4.3 基于外部资料生成计划`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。

3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-27"
claude "    "MCP    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-27
claude "    "MCP    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `MCP 案例` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `MCP 案例` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：配置一个本地知识库 MCP。
- 练习：让 Claude Code 查询外部资料后生成任务计划。
- 练习：写 MCP 权限边界说明。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `docs/mcp-config-notes.md`

- ``docs/mcp-security-policy.md``
- 一个可用的 MCP 示例配置

本章检查清单

- [] 我已经完成第 27 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“MCP 基础与外部工具连接”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Plugins 插件系统”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 28 章：Plugins

插件系统与市场与市场

> 所属篇章：第六篇：扩展系统 -

Skills、Commands、SubAgents、Hooks、MCP、Plugins

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者理解插件是团队能力打包和分发的机制。

本章从“我已经写了很多命令、Skill、Agent，如何给其他项目复用”出发，讲插件结构和团队工具包。重点是插件的边界、版本和维护，而不是只创建一个空目录。

学习目标

- 理解“Plugins 插件系统与市场”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-28"
claude "      "Plugins      "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-28
claude "      "Plugins      "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 28.1 插件的定位
 - 28.1.1 复用命令
 - 28.1.2 复用 Skill
 - 28.1.3 复用 Agent 和配置
- 28.2 插件结构
 - 28.2.1 插件元数据
 - 28.2.2 commands
 - 28.2.3 skills
 - 28.2.4 agents

- 28.3 团队工具包
- 28.3.1 review
- 28.3.2 test
- 28.3.3 deploy
- 28.3.4 security
- 28.4 插件维护
- 28.4.1 版本号
- 28.4.2 变更日志
- 28.4.3 兼容性说明

28.1 插件的定位

学习目标

- 理解 `插件的定位` 在本章主题中的具体作用。
- 能把 `插件的定位` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：复用命令、复用 Skill、复用 Agent 和配置。

核心概念

`插件的定位`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `28.1.1

复用命令`：先解释它解决的问题，再给出一个可观察的操作。

- `28.1.2 复用

Skill`：先解释它解决的问题，再给出一个可观察的操作。

- `28.1.3 复用 Agent

和配置`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-28"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-28
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
```

```
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `插件的定位` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `插件的定位` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

28.2 插件结构

学习目标

- 理解 `插件结构` 在本章主题中的具体作用。
- 能把 `插件结构` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：插件元数据、commands、skills。

核心概念

插件结构

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 28.2.1
插件元数据：先解释它解决的问题，再给出一个可观察的操作。
- 28.2.2
commands：先解释它解决的问题，再给出一个可观察的操作。
- 28.2.3 skills：先解释它解决的问题，再给出一个可观察的操作。
- 28.2.4 agents：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-28"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-28
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `插件结构` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `插件结构` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

28.3 团队工具包

学习目标

- 理解 `团队工具包` 在本章主题中的具体作用。
- 能把 `团队工具包` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：review、test、deploy。

核心概念

`团队工具包`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `28.3.1 review`：先解释它解决的问题，再给出一个可观察的操作。
- `28.3.2 test`：先解释它解决的问题，再给出一个可观察的操作。
- `28.3.3 deploy`：先解释它解决的问题，再给出一个可观察的操作。
- `28.3.4 security`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。

3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-28"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-28
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `团队工具包` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。

- 练习 3: 让 Claude Code 给出验证命令, 并记录验证结果。

检查清单

- [] 我能用自己的话解释 `团队工具包` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

28.4 插件维护

学习目标

- 理解 `插件维护` 在本章主题中的具体作用。
- 能把 `插件维护` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景, 并知道如何修正。
- 能说明本节三个重点: 版本号、变更日志、兼容性说明。

核心概念

`插件维护`

的重点不是记住术语, 而是把它放回真实工程动作里理解。对于 Claude Code 来说, 一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解:

- `28.4.1 版本号`: 先解释它解决的问题, 再给出一个可观察的操作。
- `28.4.2
变更日志`: 先解释它解决的问题, 再给出一个可观察的操作。
- `28.4.3
兼容性说明`: 先解释它解决的问题, 再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-28"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-28
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `插件维护` 改写成一个包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `插件维护` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：创建第一个本地插件。
- 练习：把 review、test、deploy 命令打包进插件。
- 练习：为插件写 v0.1 到 v0.2 的变更记录。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `plugins/team-toolkit/`
- 插件说明文档
- 插件版本策略

本章检查清单

- [] 我已经完成第 28 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“Plugins 插件系统与市场”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Tools 与 Rules 体系”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

深入专题：插件市场与团队分发

插件的价值不只是“本机装一个能力”，而是把 Skills、Commands、SubAgents、Hooks 和 MCP 配置打包成团队资产。团队内推荐把插件当作小产品维护：有 README、版本号、CHANGELOG、兼容版本和卸载说明。

发布前检查：

- [] 插件没有写死个人路径。
- [] 插件依赖清楚。
- [] 安全权限最小化。
- [] 有最小可运行示例。

第 29 章：Tools 与 Rules 体系

> 所属篇章：第六篇：扩展系统 -

Skills、Commands、SubAgents、Hooks、MCP、Plugins

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

把 Claude Code 的工具能力和规则约束放在同一张地图里，帮助读者理解“能做什么”和“该怎么做”的区别。

本章提供工具和规则的统一视角。工具决定 Claude Code 能做什么，规则决定它应该如何做，权限决定它被允许做什么。读者要建立自己的工具速查和权限边界文档。

学习目标

- 理解“Tools 与 Rules 体系”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C#** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
```



```
Set-Location "D:\AI\claude code    \book\examples\chapter-29"
claude "    "Tools    Rules    "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-29
claude "    "Tools    Rules    "    "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 29.1 工具系统全景
 - 29.1.1 读取类工具
 - 29.1.2 搜索类工具
 - 29.1.3 编辑类工具
 - 29.1.4 执行类工具
- 29.2 Rules 指令规则
 - 29.2.1 编码规则
 - 29.2.2 测试规则
 - 29.2.3 提交规则

- 29.3 权限规则
- 29.3.1 可读范围
- 29.3.2 可写范围
- 29.3.3 可执行命令
- 29.4 工具百科入口
- 29.4.1 常用工具速查
- 29.4.2 风险等级
- 29.4.3 示例命令

29.1 工具系统全景

学习目标

- 理解`工具系统全景`在本章主题中的具体作用。
- 能把`工具系统全景`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：读取类工具、搜索类工具、编辑类工具。

核心概念

`工具系统全景`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `29.1.1`
读取类工具：先解释它解决的问题，再给出一个可观察的操作。

- 29.1.2

搜索类工具：先解释它解决的问题，再给出一个可观察的操作。

- 29.1.3

编辑类工具：先解释它解决的问题，再给出一个可观察的操作。

- 29.1.4

执行类工具：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-29"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-29
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
```

```

    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `工具系统全景` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `工具系统全景` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

29.2 Rules 指令规则

学习目标

- 理解 `Rules 指令规则` 在本章主题中的具体作用。
- 能把 `Rules 指令规则` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：编码规则、测试规则、提交规则。

核心概念

Rules 指令规则

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 29.2.1
编码规则：先解释它解决的问题，再给出一个可观察的操作。
- 29.2.2
测试规则：先解释它解决的问题，再给出一个可观察的操作。
- 29.2.3
提交规则：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-29"
claude "    "Rules    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-29
```

```
claude " "Rules"
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Rules` 指令规则`改写成一个包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Rules` 指令规则`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

29.3 权限规则

学习目标

- 理解 `权限规则` 在本章主题中的具体作用。
- 能把 `权限规则` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：可读范围、可写范围、可执行命令。

核心概念

`权限规则`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `29.3.1`
可读范围：先解释它解决的问题，再给出一个可观察的操作。
- `29.3.2`
可写范围：先解释它解决的问题，再给出一个可观察的操作。
- `29.3.3`
可执行命令：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。

3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-29"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-29
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `权限规则` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。

- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `权限规则` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

29.4 工具百科入口

学习目标

- 理解 `工具百科入口` 在本章主题中的具体作用。
- 能把 `工具百科入口` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：常用工具速查、风险等级、示例命令。

核心概念

`工具百科入口`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `29.4.1
常用工具速查`：先解释它解决的问题，再给出一个可观察的操作。
- `29.4.2
风险等级`：先解释它解决的问题，再给出一个可观察的操作。

- 29.4.3

示例命令：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-29"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-29
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `工具百科入口` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `工具百科入口` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：记录一次任务中的工具调用链。
- 练习：把工具按读、写、执行、搜索分类。
- 练习：为项目写最小权限规则。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `docs/tool-reference.md`
- `docs/rules-reference.md`
- 工具风险等级表

本章检查清单

- ☐ 我已经完成第 29 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“Tools 与 Rules 体系”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Skill、Agent、MCP、Plugin 的组合模式”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 30

章：Skill、Agent、MCP、Plugin 的组合模式

> 所属篇章：第六篇：扩展系统 -

Skills、Commands、SubAgents、Hooks、MCP、Plugins

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

帮助读者在多个扩展机制之间做取舍，避免过度工程化。

本章用决策树方式讲“什么时候用什么”。如果只是一次性任务，用提示词；如果是固定流程，用命令；如果是可复用知识，用 Skill；如果需要专职角色，用 SubAgent；如果需要外部工具，用 MCP；如果要分发给团队，用 Plugin。

学习目标

- 理解“Skill、Agent、MCP、Plugin 的组合模式”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C++** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-30"
claude "    "Skill Agent MCP Plugin    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-30
claude "    "Skill Agent MCP Plugin    "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 30.1 组合选择
 - 30.1.1 只用提示词
 - 30.1.2 使用命令
 - 30.1.3 使用 Skill
 - 30.1.4 使用 Agent、MCP 或 Plugin
- 30.2 Skill + SubAgent
 - 30.2.1 专家角色
 - 30.2.2 专家知识

- 30.2.3 输出契约
- 30.3 MCP + Plugin
- 30.3.1 外部工具连接
- 30.3.2 团队分发
- 30.3.3 安装和配置说明
- 30.4 组合反模式
- 30.4.1 简单问题复杂化
- 30.4.2 职责重叠
- 30.4.3 维护成本失控

30.1 组合选择

学习目标

- 理解 `组合选择` 在本章主题中的具体作用。
- 能把 `组合选择` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：只用提示词、使用命令、使用 Skill。

核心概念

`组合选择`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `30.1.1`
只用提示词：先解释它解决的问题，再给出一个可观察的操作。

- `30.1.2

使用命令`：先解释它解决的问题，再给出一个可观察的操作。

- `30.1.3 使用

Skill`：先解释它解决的问题，再给出一个可观察的操作。

- `30.1.4 使用 Agent、MCP 或

Plugin`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-30"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-30
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
```

```

";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `组合选择` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `组合选择` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

30.2 Skill + SubAgent

学习目标

- 理解 `Skill + SubAgent` 在本章主题中的具体作用。
- 能把 `Skill + SubAgent` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：专家角色、专家知识、输出契约。

核心概念

`Skill + SubAgent`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `30.2.1

专家角色：先解释它解决的问题，再给出一个可观察的操作。

- `30.2.2

专家知识：先解释它解决的问题，再给出一个可观察的操作。

- `30.2.3

输出契约：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-30"
claude "    "Skill + SubAgent"    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-30
claude "    "Skill + SubAgent"    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Skill + SubAgent` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Skill + SubAgent` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

30.3 MCP + Plugin

学习目标

- 理解 `MCP + Plugin` 在本章主题中的具体作用。

- 能把 `MCP + Plugin` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：外部工具连接、团队分发、安装和配置说明。

核心概念

`MCP + Plugin`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `30.3.1`
外部工具连接：先解释它解决的问题，再给出一个可观察的操作。
- `30.3.2`
团队分发：先解释它解决的问题，再给出一个可观察的操作。
- `30.3.3 安装和配置说明`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。

5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-30"
claude "    "MCP + Plugin"    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-30
claude "    "MCP + Plugin"    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `MCP + Plugin` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `MCP + Plugin` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。

- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

30.4 组合反模式

学习目标

- 理解 `组合反模式` 在本章主题中的具体作用。
- 能把 `组合反模式` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：简单问题复杂化、职责重叠、维护成本失控。

核心概念

`组合反模式`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `30.4.1 简单问题复杂化`：先解释它解决的问题，再给出一个可观察的操作。
- `30.4.2 职责重叠`：先解释它解决的问题，再给出一个可观察的操作。
- `30.4.3 维护成本失控`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-30"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-30
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`组合反模式`

改写成包含目标、范围、限制、输出格式的提示词。

- 练习 2: 要求 Claude Code 先列计划, 不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令, 并记录验证结果。

检查清单

- [] 我能用自己的话解释 `组合反模式` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围, 明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成, 但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件, 并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例: 为 10 个需求选择扩展机制。
- 练习: 构建 API 文档子代理 + 文档 Skill 的组合。
- 练习: 把一个过度复杂的组合简化。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `docs/extension-decision-tree.md`
- Skill + Agent 组合示例
- 组合反模式清单

本章检查清单

- [] 我已经完成第 30 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“Skill、Agent、MCP、Plugin 的组合模式”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“扩展系统实战小项目”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>

- Skills: <https://code.claude.com/docs/zh-CN/skills>

第 31 章：扩展系统实战小项目

> 所属篇章：第六篇：扩展系统 -

Skills、Commands、SubAgents、Hooks、MCP、Plugins

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

把第 22-30 章的扩展能力整合成一个个人 Claude Code 工具箱。

本章是扩展系统阶段的小结项目。读者将命令、Skill、SubAgent 和 Hook 组合为个人工具箱，并用一个真实代码任务验证它们是否有用。

学习目标

- 理解“扩展系统实战小项目”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-31
claude " " " "
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-31
claude "      "      "      "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 31.1 项目目标
 - 31.1.1 工具箱解决哪些任务
 - 31.1.2 面向个人还是团队
 - 31.1.3 组件清单
- 31.2 实现基础能力
 - 31.2.1 review 能力
 - 31.2.2 bugfix 能力
 - 31.2.3 test 能力
- 31.3 加入自动化
 - 31.3.1 Hook 检查
 - 31.3.2 子代理验证

- 31.3.3 结束报告
- 31.4 打包和复盘
- 31.4.1 使用手册
- 31.4.2 适用场景
- 31.4.3 不适用场景

31.1 项目目标

学习目标

- 理解 `项目目标` 在本章主题中的具体作用。
- 能把 `项目目标` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：工具箱解决哪些任务、面向个人还是团队、组件清单。

核心概念

`项目目标`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `31.1.1 工具箱解决哪些任务`：先解释它解决的问题，再给出一个可观察的操作。
- `31.1.2 面向个人还是团队`：先解释它解决的问题，再给出一个可观察的操作。
- `31.1.3 组件清单`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-31"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-31
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `项目目标` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `项目目标` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

31.2 实现基础能力

学习目标

- 理解 `实现基础能力` 在本章主题中的具体作用。
- 能把 `实现基础能力` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：review 能力、bugfix 能力、test 能力。

核心概念

`实现基础能力`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资

料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `31.2.1 review

能力：先解释它解决的问题，再给出一个可观察的操作。

- `31.2.2 bugfix

能力：先解释它解决的问题，再给出一个可观察的操作。

- `31.2.3 test

能力：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-31"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-31
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()
```

```
def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `实现基础能力` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `实现基础能力` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

31.3 加入自动化

学习目标

- 理解 `加入自动化` 在本章主题中的具体作用。
- 能把 `加入自动化` 转化为一个可执行、可验证的 Claude Code 任务。

- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：Hook 检查、子代理验证、结束报告。

核心概念

“加入自动化”

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- “31.3.1 Hook
检查”：先解释它解决的问题，再给出一个可观察的操作。
- “31.3.2
子代理验证”：先解释它解决的问题，再给出一个可观察的操作。
- “31.3.3
结束报告”：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-31"
claude " " " "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-31
claude " " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `加入自动化` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `加入自动化` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

31.4 打包和复盘

学习目标

- 理解 `打包和复盘` 在本章主题中的具体作用。
- 能把 `打包和复盘` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：使用手册、适用场景、不适用场景。

核心概念

`打包和复盘`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `31.4.1`
使用手册：先解释它解决的问题，再给出一个可观察的操作。
- `31.4.2`
适用场景：先解释它解决的问题，再给出一个可观察的操作。
- `31.4.3`
不适用场景：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**Python**`。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。

2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-31"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-31
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `打包和复盘`

改写成包含目标、范围、限制、输出格式的提示词。

- 练习 2: 要求 Claude Code 先列计划, 不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令, 并记录验证结果。

检查清单

- [] 我能用自己的话解释 `打包和复盘` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围, 明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成, 但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件, 并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例: 构建个人 Claude Code 工具箱。
- 练习: 用工具箱完成一次 Bug 修复和代码审查。
- 练习: 写复盘说明哪些组件真正有价值。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `claude-toolbox/`
- `claude-toolbox/README.md`
- 工具箱复盘报告

本章检查清单

- [] 我已经完成第 31 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“扩展系统实战小项目”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Headless 模式与命令行自动化”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>

- 记忆系统: <https://code.claude.com/docs/zh-CN/memory>
- Skills: <https://code.claude.com/docs/zh-CN/skills>

第 32 章：个性化定制 - Output Styles、Status Line、Keybindings、Voice、Fullscreen

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

本章介绍“个性化定制 - Output Styles、Status Line、Keybindings、Voice、Fullscreen”的核心概念，并通过可验证的小任务帮助读者建立实践基础。它不只解释“个性化定制 - Output Styles、Status Line、Keybindings、Voice、Fullscreen是什么”，还要求读者完成一个可复盘的小任务，把经验沉淀到 Markdown、Skill、SubAgent、Hook、MCP 或自动化脚本中。

学习目标

- 理解“个性化定制 - Output Styles、Status Line、Keybindings、Voice、Fullscreen”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为可交给 Claude Code 执行的小任务。
- 能用 PowerShell 完成主流程，并读懂 macOS/Linux 对照命令。
- 能识别本章能力的适用场景、风险和不适用场景。
- 能用检查清单判断任务是否真正完成。

先做一个小案例

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\enhanced"
```

```
claude "  
# macOS / Linux  
cd ~/claude-code-handbook/book/examples/enhanced  
claude "
```

案例中的最小代码对象如下：

```
#include <filesystem>  
#include <iostream>  
  
int main() {  
    std::filesystem::path target{"README.md"};  
    std::cout << "Target: " << std::filesystem::absolute(target) <<  
    "\n";  
    std::cout << "Exists: " << std::filesystem::exists(target) << "\n";  
}
```

目录

- 32.1 Output Styles 输出样式
 - 32.1.1 Output Style 文件结构
 - 32.1.2 定义 concise / detailed / explanatory 三种
 - 32.1.3 按场景切换样式
- 32.2 Status Line 状态栏
 - 32.2.1 默认状态栏显示什么
 - 32.2.2 自定义脚本式 status line
 - 32.2.3 显示模型 / cwd / 分支 / token 用量
- 32.3 Keybindings 键位定制
 - 32.3.1 `~/claude/keybindings.json` 结构
 - 32.3.2 改写常用键位
 - 32.3.3 设计 chord（多键组合）快捷键
- 32.4 Voice Dictation 语音输入

- 32.4.1 何时语音比键盘高效
- 32.4.2 启用语音输入
- 32.4.3 长口述场景与代码相关场景的差异
- 32.5 Fullscreen / Interactive Mode
- 32.5.1 Fullscreen 模式适合什么任务
- 32.5.2 Interactive Mode 与普通模式差异
- 32.5.3 选择适合的交互模式

32.1 Output Styles 输出样式

学习目标

- 理解 `Output Styles 输出样式` 解决的具体问题。
- 能把 `Output Styles 输出样式`
写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Output Styles 输出样式` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 Output Style 文件结构、定义 concise / detailed / explanatory 三种、按场景切换样式，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。

4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“Output Styles ”

练习

- 练习 1：把一个与`Output Styles 输出样式`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-32.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

32.2 Status Line 状态栏

学习目标

- 理解`Status Line 状态栏`解决的具体问题。
- 能把`Status Line 状态栏`写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Status Line 状态栏`的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 默认状态栏显示什么、自定义脚本式 status line、显示模型 / cwd / 分支 / token 用量，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“Status Line ”

练习

- 练习 1：把一个与`Status Line 状态栏`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-32.md`。

检查清单

- [] 任务边界清楚。

- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

32.3 Keybindings 键位定制

学习目标

- 理解 `Keybindings 键位定制` 解决的具体问题。
- 能把 `Keybindings 键位定制` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Keybindings 键位定制` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 `~/ .claude/keybindings.json` 结构、改写常用键位、设计 chord（多键组合）快捷键，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Keybindings ”

练习

- 练习 1: 把一个与 `Keybindings` 键位定制` 相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-32.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

32.4 Voice Dictation 语音输入

学习目标

- 理解 `Voice Dictation 语音输入` 解决的具体问题。
- 能把 `Voice Dictation 语音输入` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Voice Dictation 语音输入` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 何时语音比键盘高效、启用语音输入、长口述场景与代码相关场景的差异，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“Voice Dictation ”

练习

- 练习 1：把一个与`Voice Dictation`语音输入`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-32.md`。

检查清单

- ☐ 任务边界清楚。
- ☐ 有可复现命令。
- ☐ 有证据和验证结果。
- ☐ 有风险与回滚说明。

32.5 Fullscreen / Interactive Mode

学习目标

- 理解 `Fullscreen / Interactive Mode` 解决的具体问题。
- 能把 `Fullscreen / Interactive Mode` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Fullscreen / Interactive Mode`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 Fullscreen 模式适合什么任务、Interactive Mode 与普通模式差异、选择适合的交互模式，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Fullscreen / Interactive Mode”

练习

- 练习 1：把一个与 `Fullscreen / Interactive Mode` 相关的模糊请求改写成完整任务。

- 练习 2: 让 Claude Code 执行一次只读探索, 并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-32.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

常见坑

- 把新功能当成固定不变的事实, 不复核官方文档。
- 没有限制工具权限, 导致 Agent 执行范围过大。
- 只生成配置, 不实际跑一次验证。
- 忽略成本、上下文污染和多人协作时的规则冲突。

本章小结

掌握“个性化定制 - Output Styles、Status Line、Keybindings、Voice、Fullscreen”的标准不是看懂定义, 而是能把它放进你的真实开发流程: 有输入、有边界、有工具、有验证、有沉淀。

多端、Web、桌面与遥控

第 33 章：桌面应用、Web 与 iOS

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

本章介绍“桌面应用、Web 与 iOS”的核心概念，并通过可验证的小任务帮助读者建立实践基础。它不只解释“桌面应用、Web 与 iOS 是什么”，还要求读者完成一个可复盘的小任务，把经验沉淀到 Markdown、Skill、SubAgent、Hook、MCP 或自动化脚本中。

学习目标

- 理解“桌面应用、Web 与 iOS”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为可交给 Claude Code 执行的小任务。
- 能用 PowerShell 完成主流程，并读懂 macOS/Linux 对照命令。
- 能识别本章能力的适用场景、风险和不适用场景。
- 能用检查清单判断任务是否真正完成。

先做一个小案例

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\enhanced"
claude "                                "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/enhanced
claude "                                "
```

案例中的最小代码对象如下：

```
from pathlib import Path

target = Path("README.md").resolve()
print({"target": str(target), "exists": target.exists()})
```

目录

- 33.1 桌面应用 Desktop App
 - 33.1.1 安装与首次启动
 - 33.1.2 桌面应用的可视 diff 视图
 - 33.1.3 并行会话与定期任务
 - 33.1.4 桌面适合的任务 vs 终端适合的任务
- 33.2 Web 版 / ``claude.ai/code``
 - 33.2.1 在浏览器里跑 Claude Code
 - 33.2.2 Web 端启动长任务、离开后回来看结果
 - 33.2.3 处理本地没有的仓库
 - 33.2.4 Web 与本地的差异（无本地文件、无本地命令）
- 33.3 iOS / 移动场景
 - 33.3.1 Claude iOS 应用入口
 - 33.3.2 在手机上接管一个 Web 启动的任务
 - 33.3.3 用手机做轻量审查与确认
 - 33.3.4 移动端做不了的事
- 33.4 ``claude --teleport`` 与 ``/desktop`` 切换
 - 33.4.1 把终端会话 teleport 到桌面
 - 33.4.2 用 ``/desktop`` 把终端会话交给桌面应用
 - 33.4.3 反向把 Web / 桌面会话拉回终端
 - 33.4.4 跨界面切换的状态同步

33.1 桌面应用 Desktop App

学习目标

- 理解 `桌面应用 Desktop App` 解决的具体问题。
- 能把 `桌面应用 Desktop App` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`桌面应用 Desktop App` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 安装与首次启动、桌面应用的可视 diff 视图、并行会话与定期任务、桌面适合的任务 vs 终端适合的任务，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“ Desktop App”

练习

- 练习 1: 把一个与 `桌面应用 Desktop App` 相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-33.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

33.2 Web 版 / `claude.ai/code`

学习目标

- 理解 `Web 版 / `claude.ai/code`` 解决的具体问题。
- 能把 `Web 版 / `claude.ai/code`` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Web 版 / `claude.ai/code`` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕在浏览器里跑 Claude Code、Web 端启动长任务、离开后回来看结果、处理本地没有的仓库、Web 与本地的差异（无本地文件、无本地命令），你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

`"Web / `claude.ai/code`"`

练习

- 练习 1：把一个与`Web 版 / `claude.ai/code``相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-33.md`。

检查清单

- ☐ 任务边界清楚。
- ☐ 有可复现命令。
- ☐ 有证据和验证结果。
- ☐ 有风险与回滚说明。

33.3 iOS / 移动场景

学习目标

- 理解`iOS / 移动场景`解决的具体问题。

- 能把 `iOS / 移动场景` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`iOS / 移动场景` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 Claude iOS 应用入口、在手机上接管一个 Web 启动的任务、用手机做轻量审查与确认、移动端做不了的事，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“iOS / ”

练习

- 练习 1：把一个与 `iOS / 移动场景` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。

- 练习 3: 把本节经验写入 `notes/enhanced-33.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

33.4 `claude --teleport` 与 `/desktop` 切换

学习目标

- 理解 `claude --teleport` 与 `/desktop` 切换` 解决的具体问题。
- 能把 `claude --teleport` 与 `/desktop` 切换`
 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`claude --teleport` 与 `/desktop` 切换`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕把终端会话 teleport 到桌面、用 `/desktop`

把终端会话交给桌面应用、反向把 Web /

桌面会话拉回终端、跨界面切换的状态同步，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。

3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

```
“`claude --teleport` 与 `/desktop` 切换”
```

练习

- 练习 1：把一个与`claude --teleport`与`/desktop`切换`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-33.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

常见坑

- 把新功能当成固定不变的事实，不复核官方文档。
- 没有限制工具权限，导致 Agent 执行范围过大。
- 只生成配置，不实际跑一次验证。
- 忽略成本、上下文污染和多人协作时的规则冲突。

本章小结

掌握“桌面应用、Web 与 iOS”的标准不是看懂定义，而是能把它放进你的真实开发流程：有输入、有边界、有工具、有验证、有沉淀。

第 34 章：Remote Control、Dispatch 与跨设备会话

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

本章介绍“Remote Control、Dispatch 与跨设备会话”的核心概念，并通过可验证的小任务帮助读者建立实践基础。它不只解释“Remote Control、Dispatch 与跨设备会话是什么”，还要求读者完成一个可复盘的小任务，把经验沉淀到 Markdown、Skill、SubAgent、Hook、MCP 或自动化脚本中。

学习目标

- 理解“Remote Control、Dispatch 与跨设备会话”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为可交给 Claude Code 执行的小任务。
- 能用 PowerShell 完成主流程，并读懂 macOS/Linux 对照命令。
- 能识别本章能力的适用场景、风险和不适用场景。
- 能用检查清单判断任务是否真正完成。

先做一个小案例

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\enhanced"
claude "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/enhanced
claude "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ClaudeTaskProbe
{
    public static void Main()
    {
        var path = Path.GetFullPath("README.md");
        Console.WriteLine($"Target: {path}");
        Console.WriteLine($"Exists: {File.Exists(path)}");
    }
}
```

目录

- 34.1 Remote Control 远程控制
 - 34.1.1 Remote Control 是什么、解决什么问题
 - 34.1.2 启用 Remote Control 的步骤
 - 34.1.3 在 iPhone 上接管 Windows 上的会话
 - 34.1.4 安全模型与会话授权
- 34.2 Dispatch
 - 34.2.1 Dispatch 与 Remote Control 的区别
 - 34.2.2 从手机把任务派给桌面会话
 - 34.2.3 把 issue 链接 dispatch 过去的工作流
 - 34.2.4 Dispatch 的典型使用场景
- 34.3 Deep Links 启动会话
 - 34.3.1 Deep Link 的格式与生成
 - 34.3.2 用 Deep Link 给同事分享一个特定会话
 - 34.3.3 用 Deep Link 复现一个 Bug 调查

- 34.3.4 Deep Link 的安全注意
- 34.4 Sessions 与 Checkpointing 协同
- 34.4.1 跨设备会话的状态同步
- 34.4.2 用 `claude --resume` + Checkpointing 跨设备恢复
- 34.4.3 设备断网后的恢复流程
- 34.4.4 多设备并发修改同一会话的处理

34.1 Remote Control 远程控制

学习目标

- 理解 `Remote Control 远程控制` 解决的具体问题。
- 能把 `Remote Control 远程控制` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Remote Control 远程控制`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 Remote Control 是什么、解决什么问题、启用 Remote Control 的步骤、在 iPhone 上接管 Windows 上的会话、安全模型与会话授权，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。

4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“Remote Control ”

练习

- 练习 1：把一个与`Remote Control 远程控制`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-34.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

34.2 Dispatch

学习目标

- 理解`Dispatch`解决的具体问题。
- 能把`Dispatch`写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Dispatch` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 Dispatch 与 Remote Control 的区别、从手机把任务派给桌面会话、把 issue 链接 dispatch 过去的工作流、Dispatch 的典型使用场景，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Dispatch”

练习

- 练习 1：把一个与 `Dispatch` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 `notes/enhanced-34.md`。

检查清单

- [] 任务边界清楚。

- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

34.3 Deep Links 启动会话

学习目标

- 理解 `Deep Links 启动会话` 解决的具体问题。
- 能把 `Deep Links 启动会话`
写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Deep Links 启动会话` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 Deep Link 的格式与生成、用 Deep Link 给同事分享一个特定会话、用 Deep Link 复现一个 Bug 调查、Deep Link 的安全注意，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

练习

- 练习 1: 把一个与 `Deep Links 启动会话` 相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-34.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

34.4 Sessions 与 Checkpointing 协同

学习目标

- 理解 `Sessions 与 Checkpointing 协同` 解决的具体问题。
- 能把 `Sessions 与 Checkpointing 协同` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Sessions 与 Checkpointing 协同`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕跨设备会话的状态同步、用 `claude --resume` + Checkpointing 跨设备恢复、设备断网后的恢复流程、多设备并发修改同一会话的处理，你

要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Sessions Checkpointing ”

练习

- 练习 1：把一个与 `Sessions` 与 `Checkpointing` 协同` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 `notes/enhanced-34.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

常见坑

- 把新功能当成固定不变的事实，不复核官方文档。
- 没有限制工具权限，导致 Agent 执行范围过大。
- 只生成配置，不实际跑一次验证。
- 忽略成本、上下文污染和多人协作时的规则冲突。

本章小结

掌握“Remote Control、Dispatch 与跨设备会话”的标准不是看懂定义，而是能把它放进你的真实开发流程：有输入、有边界、有工具、有验证、有沉淀。

第 35 章：Channels - 把外部事件推到运行中的会话

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

本章介绍“Channels - 把外部事件推到运行中的会话”的核心概念，并通过可验证的小任务帮助读者建立实践基础。它不只解释“Channels - 把外部事件推到运行中的会话是什么”，还要求读者完成一个可复盘的小任务，把经验沉淀到 Markdown、Skill、SubAgent、Hook、MCP 或自动化脚本中。

学习目标

- 理解“Channels - 把外部事件推到运行中的会话”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为可交给 Claude Code 执行的小任务。
- 能用 PowerShell 完成主流程，并读懂 macOS/Linux 对照命令。
- 能识别本章能力的适用场景、风险和不适用场景。
- 能用检查清单判断任务是否真正完成。

先做一个小案例

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\enhanced"
claude "                                "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/enhanced
claude "                                "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Target: " << std::filesystem::absolute(target) <<
    "\n";
    std::cout << "Exists: " << std::filesystem::exists(target) << "\n";
}
```

目录

- 35.1 Channels 概念
 - 35.1.1 为什么会话需要外部输入
 - 35.1.2 Channels 与命令、Hook、MCP 的差异
 - 35.1.3 Channels 架构：触发源 → 通道 → 运行中会话
- 35.2 Telegram / Discord / iMessage 集成
 - 35.2.1 用 Telegram 给 Claude Code 发任务
 - 35.2.2 用 Discord 频道当工单入口
 - 35.2.3 用 iMessage 接收任务结果通知
 - 35.2.4 聊天即工单的工作流
- 35.3 Webhook Channel
 - 35.3.1 用 webhook 把 CI 失败推送进会话
 - 35.3.2 失败 → Claude 自动分析 → 输出修复建议
 - 35.3.3 与第 39 章 GitHub Actions 的对比
 - 35.3.4 Webhook 频率控制与去重
- 35.4 Channels Reference 与权限

- 35.4.1 Channels 配置位置
- 35.4.2 给 channel 设置权限边界
- 35.4.3 模拟一次恶意 channel 输入
- 35.4.4 外部输入必须当数据处理（提示注入防御预告，第 46 章）

35.1 Channels 概念

学习目标

- 理解 `Channels 概念` 解决的具体问题。
- 能把 `Channels 概念` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Channels 概念` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 为什么会话需要外部输入、Channels 与命令、Hook、MCP 的差异、Channels 架构：触发源 → 通道 → 运行中会话，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Channels ”

练习

- 练习 1: 把一个与 `Channels 概念` 相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-35.md`。

检查清单

- ☐ 任务边界清楚。
- ☐ 有可复现命令。
- ☐ 有证据和验证结果。
- ☐ 有风险与回滚说明。

35.2 Telegram / Discord / iMessage 集成

学习目标

- 理解 `Telegram / Discord / iMessage 集成` 解决的具体问题。
- 能把 `Telegram / Discord / iMessage 集成` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Telegram / Discord / iMessage 集成`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕

用 Telegram 给 Claude Code 发任务、用 Discord 频道当工单入口、用 iMessage 接收任务结果通知、聊天即工单的工作流，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“Telegram / Discord / iMessage ”

练习

- 练习 1：把一个与`Telegram / Discord / iMessage`集成`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-35.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。

- [] 有风险与回滚说明。

35.3 Webhook Channel

学习目标

- 理解 `Webhook Channel` 解决的具体问题。
- 能把 `Webhook Channel` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Webhook Channel` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕用 webhook 把 CI 失败推送进会话、失败 → Claude 自动分析 → 输出修复建议、与第 39 章 GitHub Actions 的对比、Webhook 频率控制与去重，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Webhook Channel”

练习

- 练习 1: 把一个与 `Webhook Channel` 相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-35.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

35.4 Channels Reference 与权限

学习目标

- 理解 `Channels Reference 与权限` 解决的具体问题。
- 能把 `Channels Reference 与权限` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Channels Reference 与权限`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 Channels 配置位置、给 channel 设置权限边界、模拟一次恶意 channel 输入、外部输入必须当数据处理（提示注入防御预告，第 46 章），你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“Channels Reference ”

练习

- 练习 1：把一个与`Channels Reference`与权限`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-35.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

常见坑

- 把新功能当成固定不变的事实，不复核官方文档。

- 没有限制工具权限，导致 Agent 执行范围过大。
- 只生成配置，不实际跑一次验证。
- 忽略成本、上下文污染和多人协作时的规则冲突。

本章小结

掌握“Channels - 把外部事件推到运行中的会话”的标准不是看懂定义，而是能把它放进你的真实开发流程：有输入、有边界、有工具、有验证、有沉淀。

第 36 章：Slack、Chrome、IDE 与外部工具集成

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

本章介绍“Slack、Chrome、IDE 与外部工具集成”的核心概念，并通过可验证的小任务帮助读者建立实践基础。它不只解释“Slack、Chrome、IDE 与外部工具集成是什么”，还要求读者完成一个可复盘的小任务，把经验沉淀到 Markdown、Skill、SubAgent、Hook、MCP 或自动化脚本中。

学习目标

- 理解“Slack、Chrome、IDE 与外部工具集成”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为可交给 Claude Code 执行的小任务。
- 能用 PowerShell 完成主流程，并读懂 macOS/Linux 对照命令。
- 能识别本章能力的适用场景、风险和不适用场景。
- 能用检查清单判断任务是否真正完成。

先做一个小案例

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\enhanced"
claude "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/enhanced
claude "
```


案例中的最小代码对象如下：

```
from pathlib import Path

target = Path("README.md").resolve()
print({"target": str(target), "exists": target.exists()})
```

目录

- 36.1 Slack 集成
 - 36.1.1 在 Slack 里 @Claude 触发任务
 - 36.1.2 把 bug 报告 @Claude 让其开 PR
 - 36.1.3 配置 Slack 集成的权限
 - 36.1.4 团队 Slack 工作流模板
- 36.2 Chrome 集成 (beta)
 - 36.2.1 让 Claude Code 操作浏览器复现 UI 问题
 - 36.2.2 截图 + DOM 检查 + 控制台日志
 - 36.2.3 修复一个 UI 错误的完整链路
 - 36.2.4 Chrome 集成的稳定性边界
- 36.3 VS Code / JetBrains 插件深度使用
 - 36.3.1 编辑器内 diff 与 @-提及
 - 36.3.2 计划审查与对话历史
 - 36.3.3 IDE 与终端的分工
 - 36.3.4 何时该回到终端
- 36.4 Computer Use (计算机控制)
 - 36.4.1 让 Claude 通过 CLI 操作电脑
 - 36.4.2 打开程序、截图、自动化操作

- 36.4.3 Computer Use 的最小权限设置

- 36.4.4 风险与价值的权衡

36.1 Slack 集成

学习目标

- 理解 `Slack 集成` 解决的具体问题。
- 能把 `Slack 集成` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Slack 集成` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕在 Slack 里 @Claude 触发任务、把 bug 报告 @Claude 让其开 PR、配置 Slack 集成的权限、团队 Slack workflow 模板，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Slack ”

练习

- 练习 1: 把一个与 `Slack 集成` 相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-36.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

36.2 Chrome 集成 (beta)

学习目标

- 理解 `Chrome 集成 (beta)` 解决的具体问题。
- 能把 `Chrome 集成 (beta)` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Chrome 集成 (beta)` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 让 Claude Code 操作浏览器复现 UI 问题、截图 + DOM 检查 + 控制台日志、修复一个 UI 错误的完整链路、Chrome 集成的稳定性边界，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Chrome beta ”

练习

- 练习 1：把一个与 `Chrome 集成 (beta)` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 `notes/enhanced-36.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

36.3 VS Code / JetBrains 插件深度使用

学习目标

- 理解 `VS Code / JetBrains 插件深度使用` 解决的具体问题。
- 能把 `VS Code / JetBrains 插件深度使用` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`VS Code / JetBrains 插件深度使用`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕编辑器内 diff 与 @-提及、计划审查与对话历史、IDE 与终端的分工、何时该回到终端，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“VS Code / JetBrains ”

练习

- 练习 1：把一个与 `VS Code / JetBrains 插件深度使用` 相关的模糊请求改写成完整任务。

- 练习 2: 让 Claude Code 执行一次只读探索, 并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-36.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

36.4 Computer Use (计算机控制)

学习目标

- 理解 `Computer Use (计算机控制)` 解决的具体问题。
- 能把 `Computer Use (计算机控制)` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用, 什么时候不要使用。

核心概念

`Computer Use (计算机控制)`

的重点不是记住术语, 而是理解它如何改变 Agent 的工作边界。围绕让 Claude 通过 CLI

操作电脑、打开程序、截图、自动化操作、Computer Use

的最小权限设置、风险与价值的权衡, 你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据, 并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。

3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“Computer Use”

练习

- 练习 1：把一个与`Computer Use`（计算机控制）`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-36.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

常见坑

- 把新功能当成固定不变的事实，不复核官方文档。
- 没有限制工具权限，导致 Agent 执行范围过大。
- 只生成配置，不实际跑一次验证。
- 忽略成本、上下文污染和多人协作时的规则冲突。

本章小结

掌握“Slack、Chrome、IDE 与外部工具集成”的标准不是看懂定义，而是能把它放进你的真实开发流程：有输入、有边界、有工具、有验证、有沉淀。

自动化、Headless、CI 和云端规划

第 37 章：Headless 模式与命令行自动化

> 所属篇章：第七篇：自动化、GitHub、CI 和可编程 Agent

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者理解如何在非交互场景中调用 Claude Code 处理重复任务。

本章把 Claude Code 从“人坐在终端前对话”扩展到脚本化调用。重点是输出格式、权限边界和可重复运行，不鼓励无人值守直接修改生产代码。

学习目标

- 理解“Headless 模式与命令行自动化”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 `**C#**` 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-37"
claude "        "Headless        "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-37
claude "        "Headless        "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 32.1 Headless 是什么
 - 32.1.1 交互模式与非交互模式
 - 32.1.2 适合自动化的任务
 - 32.1.3 不适合自动化的任务
- 32.2 批处理任务
 - 32.2.1 批量检查文件
 - 32.2.2 批量生成摘要
 - 32.2.3 批量生成报告
- 32.3 输出格式

- 32.3.1 Markdown 报告
- 32.3.2 JSON 输出
- 32.3.3 可解析错误信息
- 32.4 自动化边界
- 32.4.1 只读自动化优先
- 32.4.2 写操作需要确认
- 32.4.3 自动化日志和审计

32.1 Headless 是什么

学习目标

- 理解 `Headless 是什么` 在本章主题中的具体作用。
- 能把 `Headless 是什么` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：交互模式与非交互模式、适合自动化的任务、不适合自动化的任务。

核心概念

`Headless 是什么`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `32.1.1 交互模式与非交互模式`：先解释它解决的问题，再给出一个可观察的操作。

- `32.1.2 适合自动化的任务`：先解释它解决的问题，再给出一个可观察的操作。
- `32.1.3 不适合自动化的任务`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-37"
claude "    "Headless    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-37
claude "    "Headless    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
    }
}
```

```
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||  
Directory.Exists(fullPath)}");  
    }  
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Headless 是什么` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Headless 是什么` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

32.2 批处理任务

学习目标

- 理解 `批处理任务` 在本章主题中的具体作用。
- 能把 `批处理任务` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：批量检查文件、批量生成摘要、批量生成报告。

核心概念

“批处理任务”

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- “32.2.1
批量检查文件”：先解释它解决的问题，再给出一个可观察的操作。
- “32.2.2
批量生成摘要”：先解释它解决的问题，再给出一个可观察的操作。
- “32.2.3
批量生成报告”：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-37"
claude " " " "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-37
claude " " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `批处理任务` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `批处理任务` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

32.3 输出格式

学习目标

- 理解 `输出格式` 在本章主题中的具体作用。
- 能把 `输出格式` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：Markdown 报告、JSON 输出、可解析错误信息。

核心概念

`输出格式`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `32.3.1 Markdown 报告`：先解释它解决的问题，再给出一个可观察的操作。
- `32.3.2 JSON 输出`：先解释它解决的问题，再给出一个可观察的操作。
- `32.3.3 可解析错误信息`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。

2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-37"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-37
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `输出格式` 改写成包含目标、范围、限制、输出格式的提示词。

- 练习 2: 要求 Claude Code 先列计划, 不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令, 并记录验证结果。

检查清单

- [] 我能用自己的话解释 `输出格式` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

32.4 自动化边界

学习目标

- 理解 `自动化边界` 在本章主题中的具体作用。
- 能把 `自动化边界` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景, 并知道如何修正。
- 能说明本节三个重点: 只读自动化优先、写操作需要确认、自动化日志和审计。

核心概念

`自动化边界`

的重点不是记住术语, 而是把它放回真实工程动作里理解。对于 Claude Code 来说, 一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解:

- `32.4.1 只读自动化优先`: 先解释它解决的问题, 再给出一个可观察的操作。

- `32.4.2 写操作需要确认`：先解释它解决的问题，再给出一个可观察的操作。
- `32.4.3 自动化日志和审计`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-37"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-37
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
    }
}
```

```
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||  
Directory.Exists(fullPath)}");  
    }  
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `自动化边界` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `自动化边界` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |

要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：批量为多个模块生成文档质量报告。
- 练习：要求输出 JSON 并验证可解析。
- 练习：设计只读自动化流程。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `scripts/claude-doc-check.md`
- 自动化输出格式规范
- Headless 使用边界清单

本章检查清单

- [] 我已经完成第 32 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“Headless 模式与命令行自动化”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Agent SDK 入门”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 38 章：Routines、`/loop` 与定时任务

> 所属篇章：第七篇：自动化、GitHub、CI 和可编程 Agent

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者把重复检查、总结和报告转化为可治理的例行自动化。

本章强调自动化需要治理。Routine 适合做周期性总结和检查，但不适合在没有审核的情况下做破坏性操作。读者要建立自动化清单，知道哪些任务在运行、何时运行、输出到哪里。

学习目标

- 理解“Routines、定时任务和自动化工作流”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。


```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-38"
claude "        "Routines        "        "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-38
claude "        "Routines        "        "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 34.1 Routine 思维
- 34.1.1 重复任务识别
- 34.1.2 固定输入和固定输出
- 34.1.3 触发频率
- 34.2 定时检查
- 34.2.1 依赖检查
- 34.2.2 测试状态检查
- 34.2.3 文档健康检查
- 34.3 目标与通道

- 34.3.1 输出到报告
- 34.3.2 输出到当前线程
- 34.3.3 不直接做高风险修改
- 34.4 自动化治理
- 34.4.1 自动化清单
- 34.4.2 暂停条件
- 34.4.3 复盘和调整

34.1 Routine 思维

学习目标

- 理解 `Routine 思维` 在本章主题中的具体作用。
- 能把 `Routine 思维` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：重复任务识别、固定输入和固定输出、触发频率。

核心概念

`Routine 思维`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `34.1.1`
重复任务识别：先解释它解决的问题，再给出一个可观察的操作。

- `34.1.2 固定输入和固定输出`：先解释它解决的问题，再给出一个可观察的操作。
- `34.1.3 触发频率`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-38"
claude "    "Routine    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-38
claude "    "Routine    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
```

```
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Routine 思维` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Routine 思维` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

34.2 定时检查

学习目标

- 理解 `定时检查` 在本章主题中的具体作用。
- 能把 `定时检查` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：依赖检查、测试状态检查、文档健康检查。

核心概念

`定时检查`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `34.2.1

依赖检查：先解释它解决的问题，再给出一个可观察的操作。

- `34.2.2

测试状态检查：先解释它解决的问题，再给出一个可观察的操作。

- `34.2.3

文档健康检查：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-38"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-38
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `定时检查` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `定时检查` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

34.3 目标与通道

学习目标

- 理解`目标与通道`在本章主题中的具体作用。
- 能把`目标与通道`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：输出到报告、输出到当前线程、不直接做高风险修改。

核心概念

`目标与通道`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `34.3.1 输出到报告`：先解释它解决的问题，再给出一个可观察的操作。
- `34.3.2 输出到当前线程`：先解释它解决的问题，再给出一个可观察的操作。
- `34.3.3 不直接做高风险修改`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**Python**`。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。

3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-38"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-38
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`目标与通道`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。

- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `目标与通道` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

34.4 自动化治理

学习目标

- 理解 `自动化治理` 在本章主题中的具体作用。
- 能把 `自动化治理` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：自动化清单、暂停条件、复盘和调整。

核心概念

`自动化治理`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `34.4.1
自动化清单`：先解释它解决的问题，再给出一个可观察的操作。
- `34.4.2
暂停条件`：先解释它解决的问题，再给出一个可观察的操作。

- 34.4.3

复盘和调整：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-38"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-38
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
```

```
print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`自动化治理`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`自动化治理`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：设计每周项目质量巡检。
- 练习：写一个每日待办总结任务。
- 练习：为自动化写暂停条件。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- ``docs/routines.md``
- 周期性质量巡检模板
- 自动化治理清单

本章检查清单

- ☐ 我已经完成第 34 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“Routines、定时任务和自动化工作流”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“GitHub PR / CI 工作流”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

深入专题：`/loop` 与 Routines 的区别

Routines 偏向“例行流程”，`/loop` 偏向“反复尝试直到满足条件”。使用 `/loop` 时必须设置停止条件，否则容易浪费 token 或重复执行风险命令。

示例停止条件：

- 测试全部通过。
- 只允许最多 3 轮修改。
- 每轮必须输出 diff 摘要和失败原因。
- 触及权限、安全、删除、发布动作时立即停止等待确认。

第 39 章：GitHub / GitLab CI、Code Review 自动化

> 所属篇章：第七篇：自动化、GitHub、CI 和可编程 Agent

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者掌握 Claude Code 在 PR 审查、CI 失败定位和修复流程中的使用方法。

本章把 Claude Code 放入 GitHub 协作流程中。读者要学会让 Claude Code 总结 PR、审查 diff、分析 CI

日志，并输出可执行修复计划。重点仍是人工确认和可验证。

学习目标

- 理解“GitHub PR / CI 工作流”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C#** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-39"
claude "      "GitHub PR / CI      "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-39
claude "      "GitHub PR / CI      "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 35.1 PR 分析
 - 35.1.1 读取 diff
 - 35.1.2 总结行为变化
 - 35.1.3 标注风险和测试
- 35.2 CI 失败定位
 - 35.2.1 读取失败日志
 - 35.2.2 定位相关文件
 - 35.2.3 输出修复假设
- 35.3 自动修复流程

- 35.3.1 失败日志到修复计划
- 35.3.2 人工确认修改
- 35.3.3 重跑检查
- 35.4 PR 质量门
- 35.4.1 测试是否充分
- 35.4.2 风险是否说明
- 35.4.3 文档是否更新

35.1 PR 分析

学习目标

- 理解 `PR 分析` 在本章主题中的具体作用。
- 能把 `PR 分析` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：读取 diff、总结行为变化、标注风险和测试。

核心概念

`PR 分析`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `35.1.1 读取 diff`：先解释它解决的问题，再给出一个可观察的操作。

- 35.1.2

总结行为变化：先解释它解决的问题，再给出一个可观察的操作。

- 35.1.3 标注风险和测试：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-39"
claude "    "PR    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-39
claude "    "PR    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
    }
}
```

```
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||  
Directory.Exists(fullPath)}");  
    }  
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `PR 分析` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `PR 分析` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

35.2 CI 失败定位

学习目标

- 理解 `CI 失败定位` 在本章主题中的具体作用。
- 能把 `CI 失败定位` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：读取失败日志、定位相关文件、输出修复假设。

核心概念

“CI 失败定位”

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- “35.2.1
读取失败日志”：先解释它解决的问题，再给出一个可观察的操作。
- “35.2.2
定位相关文件”：先解释它解决的问题，再给出一个可观察的操作。
- “35.2.3
输出修复假设”：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-39"
claude " " "CI" " "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-39
claude " "CI" "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `CI 失败定位` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `CI 失败定位` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

35.3 自动修复流程

学习目标

- 理解 `自动修复流程` 在本章主题中的具体作用。
- 能把 `自动修复流程` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：失败日志到修复计划、人工确认修改、重跑检查。

核心概念

`自动修复流程`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `35.3.1 失败日志到修复计划`：先解释它解决的问题，再给出一个可观察的操作。
- `35.3.2 人工确认修改`：先解释它解决的问题，再给出一个可观察的操作。
- `35.3.3 重跑检查`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-39"
claude "    "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-39
claude "    "    "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1: 把`自动修复流程`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2: 要求 Claude Code 先列计划，不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`自动修复流程`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

35.4 PR 质量门

学习目标

- 理解`PR 质量门`在本章主题中的具体作用。
- 能把`PR 质量门`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：测试是否充分、风险是否说明、文档是否更新。

核心概念

`PR 质量门`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 35.4.1

测试是否充分：先解释它解决的问题，再给出一个可观察的操作。

- 35.4.2

风险是否说明：先解释它解决的问题，再给出一个可观察的操作。

- 35.4.3

文档是否更新：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-39"
claude "    "PR    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-39
claude "    "PR    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
```



```

    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `PR 质量门` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `PR 质量门` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |

在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |

要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：分析一个失败的 GitHub Actions 日志。
- 练习：为一个 PR 写审查摘要。
- 练习：建立 PR 合并前质量门。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `docs/pr-review-checklist.md`
- `prompts/ci-failure-analysis.md`
- CI 修复流程模板

本章检查清单

- [] 我已经完成第 35 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“GitHub PR / CI 工作流”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“IDE、浏览器和外部开发工具集成”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览: <https://code.claude.com/docs/zh-CN/overview>
- 记忆系统: <https://code.claude.com/docs/zh-CN/memory>
- Skills: <https://code.claude.com/docs/zh-CN/skills>

深入专题：GitLab CI 与 Code Review 自动化

无论 GitHub 还是 GitLab，CI 与 Code Review 自动化的核心都不是“让 AI 自动合并”，而是让 AI 读取失败日志、定位嫌疑文件、给出修复建议和风险说明。

CI 自动审查输出建议固定为：

```
##  
##  
##  
##  
##
```

第 40 章：ultraplan、ultrareview 与 Worktrees

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

本章介绍“ultraplan、ultrareview 与 Worktrees”的核心概念，并通过可验证的小任务帮助读者建立实践基础。它不只解释“ultraplan、ultrareview 与 Worktrees是什么”，还要求读者完成一个可复盘的小任务，把经验沉淀到 Markdown、Skill、SubAgent、Hook、MCP 或自动化脚本中。

学习目标

- 理解“ultraplan、ultrareview 与 Worktrees”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为可交给 Claude Code 执行的小任务。
- 能用 PowerShell 完成主流程，并读懂 macOS/Linux 对照命令。
- 能识别本章能力的适用场景、风险和不适用场景。
- 能用检查清单判断任务是否真正完成。

先做一个小案例

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\enhanced"
claude "                                "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/enhanced
claude "                                "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ClaudeTaskProbe
{
    public static void Main()
    {
        var path = Path.GetFullPath("README.md");
        Console.WriteLine($"Target: {path}");
        Console.WriteLine($"Exists: {File.Exists(path)}");
    }
}
```

目录

- 40.1 ultraplan 云端规划
 - 40.1.1 ultraplan 是什么、和本地 Plan Mode 的区别
 - 40.1.2 用 ultraplan 计划一个跨模块重构
 - 40.1.3 何时上云规划、何时本地规划
 - 40.1.4 ultraplan 的成本与时间预期
- 40.2 ultrareview 云端审查
 - 40.2.1 多 Agent 云端审查当前分支或 PR
 - 40.2.2 用 `ultrareview` 审一次本地分支
 - 40.2.3 用 `ultrareview <PR#>` 审一个 GitHub PR
 - 40.2.4 ultrareview 与本地 review 的输出差异
 - 40.2.5 ultrareview 的成本与触发场景
- 40.3 Worktrees 并行会话
 - 40.3.1 Git worktree 概念回顾
 - 40.3.2 用 `EnterWorktree` 创建隔离环境

- 40.3.3 一个 Bug 修复 + 一个新功能并行开发
- 40.3.4 主仓与 worktree 的同步策略
- 40.4 Dev Containers 容器化开发
- 40.4.1 何时该容器化（隔离危险任务、复现环境）
- 40.4.2 写一个最小 devcontainer
- 40.4.3 Claude Code 在容器内的体验差异

40.1 ultraplan 云端规划

学习目标

- 理解 `ultraplan 云端规划` 解决的具体问题。
- 能把 `ultraplan 云端规划`
写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`ultraplan 云端规划` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 ultraplan 是什么、和本地 Plan Mode 的区别、用 ultraplan

计划一个跨模块重构、何时上云规划、何时本地规划、ultraplan 的成本与时间预期，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。

4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“ultraplan ”

练习

- 练习 1：把一个与`ultraplan` 云端规划`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-40.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

40.2 ultrareview 云端审查

学习目标

- 理解`ultrareview` 云端审查`解决的具体问题。
- 能把`ultrareview` 云端审查`写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`ultrareview` 云端审查` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 多 Agent 云端审查当前分支或 PR、用 `/ultrareview` 审一次本地分支、用 `/ultrareview <PR#>` 审一个 GitHub PR、ultrareview 与本地 review 的输出差异、ultrareview 的成本与触发场景，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“ultrareview ”

练习

- 练习 1：把一个与 `ultrareview` 云端审查` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 `notes/enhanced-40.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

40.3 Worktrees 并行会话

学习目标

- 理解 `Worktrees 并行会话` 解决的具体问题。
- 能把 `Worktrees 并行会话`
 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Worktrees 并行会话` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 Git worktree 概念回顾、用 `EnterWorktree` 创建隔离环境、一个 Bug 修复 + 一个新功能并行开发、主仓与 worktree 的同步策略，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Worktrees ”

练习

- 练习 1: 把一个与 `Worktrees` 并行会话` 相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-40.md`。

检查清单

- ☐ 任务边界清楚。
- ☐ 有可复现命令。
- ☐ 有证据和验证结果。
- ☐ 有风险与回滚说明。

40.4 Dev Containers 容器化开发

学习目标

- 理解 `Dev Containers 容器化开发` 解决的具体问题。
- 能把 `Dev Containers 容器化开发` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Dev Containers 容器化开发`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕

何时该容器化（隔离危险任务、复现环境）、写一个最小 devcontainer、Claude Code 在容器内的体验差异，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Dev Containers ”

练习

- 练习 1：把一个与 `Dev Containers` 容器化开发` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 `notes/enhanced-40.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。

- [] 有风险与回滚说明。

常见坑

- 把新功能当成固定不变的事实，不复核官方文档。
- 没有限制工具权限，导致 Agent 执行范围过大。
- 只生成配置，不实际跑一次验证。
- 忽略成本、上下文污染和多人协作时的规则冲突。

本章小结

掌握“ultraplan、ultrareview 与 Worktrees”的标准不是看懂定义，而是能把它放进你的真实开发流程：有输入、有边界、有工具、有验证、有沉淀。

Agent SDK - 把 Claude Code 嵌进你自己的产品

第 41 章：Agent SDK 入门

> 所属篇章：第七篇：自动化、GitHub、CI 和可编程 Agent

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者了解如何从程序中调用 Claude Code 能力，构建自己的轻量 Agent 工具。

本章定位为入门，不展开复杂 SDK 工程。读者要理解 SDK 的核心价值是把 Claude Code 变成程序可调用能力，并通过 options 控制模型、权限、目录和输出。

学习目标

- 理解“Agent SDK 入门”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C++** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-41"
```

```
claude "      "Agent SDK      "
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-41
claude "      "Agent SDK      "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 33.1 SDK 解决什么问题
- 33.1.1 把 Agent 能力嵌入程序
- 33.1.2 固定任务自动化
- 33.1.3 与现有工具链集成
- 33.2 Options 与权限
- 33.2.1 模型选择
- 33.2.2 工具权限
- 33.2.3 工作目录和输出控制
- 33.3 流式与会话
- 33.3.1 实时输出
- 33.3.2 保存状态
- 33.3.3 错误处理

- 33.4 SDK 案例
- 33.4.1 读取失败测试
- 33.4.2 生成修复建议
- 33.4.3 限制不直接越权修改

33.1 SDK 解决什么问题

学习目标

- 理解 `SDK 解决什么问题` 在本章主题中的具体作用。
- 能把 `SDK 解决什么问题` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：把 Agent 能力嵌入程序、固定任务自动化、与现有工具链集成。

核心概念

`SDK 解决什么问题`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `33.1.1 把 Agent 能力嵌入程序`：先解释它解决的问题，再给出一个可观察的操作。
- `33.1.2 固定任务自动化`：先解释它解决的问题，再给出一个可观察的操作。
- `33.1.3 与现有工具链集成`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-41"
claude "    "SDK    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-41
claude "    "SDK    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1: 把 `SDK 解决什么问题` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2: 要求 Claude Code 先列计划，不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `SDK 解决什么问题` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

33.2 Options 与权限

学习目标

- 理解 `Options 与权限` 在本章主题中的具体作用。
- 能把 `Options 与权限` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：模型选择、工具权限、工作目录和输出控制。

核心概念

`Options 与权限`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `33.2.1

模型选择：先解释它解决的问题，再给出一个可观察的操作。

- `33.2.2

工具权限：先解释它解决的问题，再给出一个可观察的操作。

- `33.2.3 工作目录和输出控制：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-41"
claude "    "Options    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-41
claude "    "Options    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
```

```

";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Options 与权限` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Options 与权限` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

33.3 流式与会话

学习目标

- 理解 `流式与会话` 在本章主题中的具体作用。
- 能把 `流式与会话` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：实时输出、保存状态、错误处理。

核心概念

流式与会话

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 33.3.1

实时输出：先解释它解决的问题，再给出一个可观察的操作。

- 33.3.2

保存状态：先解释它解决的问题，再给出一个可观察的操作。

- 33.3.3

错误处理：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-41"
claude " " " "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-41
claude " " " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `流式与会话` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `流式与会话` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

33.4 SDK 案例

学习目标

- 理解 `SDK 案例` 在本章主题中的具体作用。

- 能把 `SDK 案例` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：读取失败测试、生成修复建议、限制不直接越权修改。

核心概念

`SDK 案例`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `33.4.1
读取失败测试`：先解释它解决的问题，再给出一个可观察的操作。
- `33.4.2
生成修复建议`：先解释它解决的问题，再给出一个可观察的操作。
- `33.4.3 限制不直接越权修改`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

Windows PowerShell

```
Set-Location "D:\AI\claude code    \book\examples\chapter-41"
claude "    "SDK    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-41
claude "    "SDK    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `SDK 案例` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `SDK 案例` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：写一个代码摘要小脚本。
- 练习：设置只读权限，观察输出差异。
- 练习：构建测试失败分析助手。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `scripts/agent-sdk-code-summary.\*`
- `scripts/test-failure-analyzer.\*`
- SDK 调用边界说明

本章检查清单

- [] 我已经完成第 33 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“Agent SDK 入门”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Routines、定时任务和自动化工作流”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

深入专题：为什么 SDK 要拆成 4 章

SDK 入门不只解决“能不能从程序里调用 Agent”。真正落地时还要同时解决权限、工具、Hooks、多 Agent、Skills、Commands、Plugins、MCP、部署和可观测性。

读者应该把第 41 章当作入口：先跑通最小 Agent，再进入第 42-44 章逐步加护栏、生态复用和生产化能力。

第 42 章：SDK 中的权限、工具与 Hooks

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

本章介绍“SDK 中的权限、工具与 Hooks”的核心概念，并通过可验证的小任务帮助读者建立实践基础。它不只解释“SDK 中的权限、工具与 Hooks 是什么”，还要求读者完成一个可复盘的小任务，把经验沉淀到 Markdown、Skill、SubAgent、Hook、MCP 或自动化脚本中。

学习目标

- 理解“SDK 中的权限、工具与 Hooks”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为可交给 Claude Code 执行的小任务。
- 能用 PowerShell 完成主流程，并读懂 macOS/Linux 对照命令。
- 能识别本章能力的适用场景、风险和不适用场景。
- 能用检查清单判断任务是否真正完成。

先做一个小案例

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\enhanced"
claude "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/enhanced
claude "
```

案例中的最小代码对象如下：

```
from pathlib import Path

target = Path("README.md").resolve()
print({"target": str(target), "exists": target.exists()})
```

目录

- 42.1 Options 与权限模式
 - 42.1.1 SDK Options 总览 (model / cwd / allowed_tools / permission_mode)
 - 42.1.2 配置只读分析任务 (disallow Write/Edit/Bash 写入)
 - 42.1.3 改变权限观察行为差异
 - 42.1.4 SDK 调用边界的最小权限原则
- 42.2 Custom Tools 自定义工具
 - 42.2.1 给 Claude 加你自己的工具 (如 `query\_db`、`call\_internal\_api`)
 - 42.2.2 工具描述写法 (影响模型何时调用)
 - 42.2.3 让 Agent 必须用工具回答 (避免幻觉)
 - 42.2.4 错误处理与重试
- 42.3 Tool Search
 - 42.3.1 工具数量超过上下文容量怎么办
 - 42.3.2 启用 tool search 索引 100+ 工具
 - 42.3.3 开关 tool search 的对比测试
- 42.4 SDK 中的 Hooks
 - 42.4.1 SDK Hooks 与 CLI Hooks 概念互通
 - 42.4.2 用 Hook 拦截某些 Bash 命令

- 42.4.3 在 Hook 中记录审计日志
- 42.4.4 Hook 修改工具调用参数的写法
- 42.5 Handle Approvals & User Input
- 42.5.1 实现一个 approval 流程（程序中弹窗 / Slack 询问）
- 42.5.2 模拟用户拒绝
- 42.5.3 危险操作不会偷跑

42.1 Options 与权限模式

学习目标

- 理解 `Options 与权限模式` 解决的具体问题。
- 能把 `Options 与权限模式`
写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Options 与权限模式` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 SDK Options 总览（model / cwd / allowed_tools / permission_mode）、配置只读分析任务（disallow Write/Edit/Bash 写入）、改变权限观察行为差异、SDK 调用边界的最小权限原则，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。

4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“Options ”

练习

- 练习 1：把一个与`Options`与权限模式`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-42.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

42.2 Custom Tools 自定义工具

学习目标

- 理解`Custom Tools`自定义工具`解决的具体问题。
- 能把`Custom Tools`自定义工具`写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Custom Tools 自定义工具`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕给 Claude 加你自己的工具（如 `query\_db`、`call\_internal\_api`）、工具描述写法（影响模型何时调用）、让 Agent 必须用工具回答（避免幻觉）、错误处理与重试，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Custom Tools ”

练习

- 练习 1：把一个与 `Custom Tools 自定义工具` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 `notes/enhanced-42.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

42.3 Tool Search

学习目标

- 理解 `Tool Search` 解决的具体问题。
- 能把 `Tool Search` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Tool Search` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 工具数量超过上下文容量怎么办、启用 tool search 索引 100+ 工具、开关 tool search 的对比测试，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Tool Search”

练习

- 练习 1: 把一个与 `Tool Search` 相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-42.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

42.4 SDK 中的 Hooks

学习目标

- 理解 `SDK 中的 Hooks` 解决的具体问题。
- 能把 `SDK 中的 Hooks` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`SDK 中的 Hooks` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 SDK Hooks 与 CLI Hooks 概念互通、用 Hook 拦截某些 Bash 命令、在 Hook 中记录审计日志、Hook

修改工具调用参数的写法，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“SDK Hooks”

练习

- 练习 1：把一个与 `SDK 中的 Hooks` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 `notes/enhanced-42.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

42.5 Handle Approvals & User Input

学习目标

- 理解 `Handle Approvals & User Input` 解决的具体问题。
- 能把 `Handle Approvals & User Input` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Handle Approvals & User Input`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕实现一个 approval 流程（程序中弹窗 / Slack 询问）、模拟用户拒绝、危险操作不会偷跑，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Handle Approvals & User Input”

练习

- 练习 1: 把一个与 `Handle Approvals & User Input` 相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-42.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

常见坑

- 把新功能当成固定不变的事实，不复核官方文档。
- 没有限制工具权限，导致 Agent 执行范围过大。
- 只生成配置，不实际跑一次验证。
- 忽略成本、上下文污染和多人协作时的规则冲突。

本章小结

掌握“SDK 中的权限、工具与 Hooks”的标准不是看懂定义，而是能把它放进你的真实开发流程：有输入、有边界、有工具、有验证、有沉淀。

第 43 章：SDK 中的多 Agent、Skills、Commands 与 Plugins

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

本章介绍“SDK 中的多 Agent、Skills、Commands 与 Plugins”的核心概念，并通过可验证的小任务帮助读者建立实践基础。它不只解释“SDK 中的多 Agent、Skills、Commands 与 Plugins 是什么”，还要求读者完成一个可复盘的小任务，把经验沉淀到 Markdown、Skill、SubAgent、Hook、MCP 或自动化脚本中。

学习目标

- 理解“SDK 中的多 Agent、Skills、Commands 与 Plugins”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为可交给 Claude Code 执行的小任务。
- 能用 PowerShell 完成主流程，并读懂 macOS/Linux 对照命令。
- 能识别本章能力的适用场景、风险和不适用场景。
- 能用检查清单判断任务是否真正完成。

先做一个小案例

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\enhanced"
claude "
"

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/enhanced
claude "
```

案例中的最小代码对象如下:

```
using System;
using System.IO;

public static class ClaudeTaskProbe
{
    public static void Main()
    {
        var path = Path.GetFullPath("README.md");
        Console.WriteLine($"Target: {path}");
        Console.WriteLine($"Exists: {File.Exists(path)}");
    }
}
```

目录

- 43.1 Subagents in SDK
 - 43.1.1 在程序里派生子 Agent
 - 43.1.2 主 Agent 调用 reviewer subagent
 - 43.1.3 隔离子 Agent 上下文, 主上下文不被污染
- 43.2 Skills in SDK
 - 43.2.1 把 Skill 注入 SDK 会话
 - 43.2.2 让 SDK Agent 用一个团队 Skill
 - 43.2.3 调试 Skill 不触发的问题
- 43.3 Slash Commands in SDK
 - 43.3.1 在程序中暴露 / 命令
 - 43.3.2 给最终用户提供 `/diagnose`、`/analyze` 等命令
 - 43.3.3 写一个面向终端用户的 SDK CLI
- 43.4 Plugins in SDK

- 43.4.1 在程序中加载插件
- 43.4.2 用同一个团队插件包驱动 SDK
- 43.4.3 解决 SDK 里的插件依赖
- 43.5 File Checkpointing in SDK
- 43.5.1 程序中安全编辑文件
- 43.5.2 在程序里开启 checkpoint, 失败后 rollback
- 43.5.3 Checkpoint 与数据库事务的类比
- 43.6 MCP in SDK
- 43.6.1 在程序中连接外部 MCP
- 43.6.2 用 SDK 连接一个数据库 MCP
- 43.6.3 控制 MCP 权限

43.1 Subagents in SDK

学习目标

- 理解 `Subagents in SDK` 解决的具体问题。
- 能把 `Subagents in SDK`
写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用, 什么时候不要使用。

核心概念

`Subagents in SDK` 的重点不是记住术语, 而是理解它如何改变 Agent 的工作边界。围绕 在程序里派生子 Agent、主 Agent 调用 reviewer subagent、隔离子 Agent 上下文, 主上下文不被污染, 你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据, 并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“Subagents in SDK”

练习

- 练习 1：把一个与`Subagents in SDK`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-43.md`。

检查清单

- ☐ 任务边界清楚。
- ☐ 有可复现命令。
- ☐ 有证据和验证结果。
- ☐ 有风险与回滚说明。

43.2 Skills in SDK

学习目标

- 理解 `Skills in SDK` 解决的具体问题。
- 能把 `Skills in SDK` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Skills in SDK` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 把 Skill 注入 SDK 会话、让 SDK Agent 用一个团队 Skill、调试 Skill 不触发的问题，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Skills in SDK”

练习

- 练习 1：把一个与 `Skills in SDK` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 `notes/enhanced-43.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

43.3 Slash Commands in SDK

学习目标

- 理解 `Slash Commands in SDK` 解决的具体问题。
- 能把 `Slash Commands in SDK`
 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Slash Commands in SDK` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 在程序中暴露 / 命令、给最终用户提供 `/diagnose`、`/analyze` 等命令、写一个面向终端用户的 SDK CLI，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。

5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Slash Commands in SDK”

练习

- 练习 1: 把一个与 `Slash Commands in SDK` 相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-43.md`。

检查清单

- ☐ 任务边界清楚。
- ☐ 有可复现命令。
- ☐ 有证据和验证结果。
- ☐ 有风险与回滚说明。

43.4 Plugins in SDK

学习目标

- 理解 `Plugins in SDK` 解决的具体问题。
- 能把 `Plugins in SDK` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Plugins in SDK` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 在程序中加载插件、用同一个团队插件包驱动 SDK、解决 SDK 里的插件依赖，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Plugins in SDK”

练习

- 练习 1：把一个与 `Plugins in SDK` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 `notes/enhanced-43.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。

- [] 有风险与回滚说明。

43.5 File Checkpointing in SDK

学习目标

- 理解 `File Checkpointing in SDK` 解决的具体问题。
- 能把 `File Checkpointing in SDK` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`File Checkpointing in SDK` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 程序中安全编辑文件、在程序里开启 checkpoint，失败后 rollback、Checkpoint 与数据库事务的类比，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“File Checkpointing in SDK”

练习

- 练习 1: 把一个与 `File Checkpointing in SDK` 相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-43.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

43.6 MCP in SDK

学习目标

- 理解 `MCP in SDK` 解决的具体问题。
- 能把 `MCP in SDK` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`MCP in SDK` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 在程序中连接外部 MCP、用 SDK 连接一个数据库 MCP、控制 MCP 权限，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。

2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“MCP in SDK”

练习

- 练习 1: 把一个与`MCP in SDK`相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入`notes/enhanced-43.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

常见坑

- 把新功能当成固定不变的事实，不复核官方文档。
- 没有限制工具权限，导致 Agent 执行范围过大。
- 只生成配置，不实际跑一次验证。
- 忽略成本、上下文污染和多人协作时的规则冲突。

本章小结

掌握“SDK 中的多 Agent、Skills、Commands 与 Plugins”的标准不是看懂定义，而是能把它放进你的真实开发流程：有输入、有边界、有工具、有验证、有沉淀。

第 44 章：SDK 部署、可观测性与迁移

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

本章介绍“SDK 部署、可观测性与迁移”的核心概念，并通过可验证的小任务帮助读者建立实践基础。它不只解释“SDK 部署、可观测性与迁移是什么”，还要求读者完成一个可复盘的小任务，把经验沉淀到 Markdown、Skill、SubAgent、Hook、MCP 或自动化脚本中。

学习目标

- 理解“SDK 部署、可观测性与迁移”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为可交给 Claude Code 执行的小任务。
- 能用 PowerShell 完成主流程，并读懂 macOS/Linux 对照命令。
- 能识别本章能力的适用场景、风险和不适用场景。
- 能用检查清单判断任务是否真正完成。

先做一个小案例

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\enhanced"
claude "
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/enhanced
claude "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Target: " << std::filesystem::absolute(target) <<
"\n";
    std::cout << "Exists: " << std::filesystem::exists(target) << "\n";
}
```

目录

- 44.1 Hosting the SDK
 - 44.1.1 Agent SDK 的部署形态（脚本 / 服务 / 容器 / Serverless）
 - 44.1.2 用容器部署最小 Agent
 - 44.1.3 加一个健康检查端点
 - 44.1.4 Agent 可被远程访问的接口设计
- 44.2 Securely Deploying AI Agents
 - 44.2.1 部署期的安全清单
 - 44.2.2 密钥管理 / 网络隔离 / 输入校验
 - 44.2.3 用 secret manager 而非 `.env`
 - 44.2.4 部署不留尾巴
- 44.3 Cost Tracking
 - 44.3.1 程序级成本追踪 API
 - 44.3.2 上报每次调用的 token
 - 44.3.3 给团队画一张成本图
 - 44.3.4 知道哪种用户最贵
- 44.4 Observability with OpenTelemetry

- 44.4.1 接 OTLP collector / Jaeger
- 44.4.2 跟踪一次失败任务到具体工具调用
- 44.4.3 度量 / 追踪 / 日志三类信号
- 44.4.4 SLO 与告警阈值
- 44.5 Modifying System Prompts
- 44.5.1 自定义系统提示词的边界
- 44.5.2 替换 vs 追加 vs 合并
- 44.5.3 比较不同系统提示效果
- 44.5.4 不破坏官方安全约束
- 44.6 Migrate to Claude Agent SDK
- 44.6.1 从旧 SDK / 类似框架迁移
- 44.6.2 跟着 migration guide 升级一次
- 44.6.3 列出 breaking change 清单
- 44.6.4 升级路径与回滚计划

44.1 Hosting the SDK

学习目标

- 理解 `Hosting the SDK` 解决的具体问题。
- 能把 `Hosting the SDK`
 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Hosting the SDK` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 Agent SDK 的部署形态（脚本 / 服务 / 容器 / Serverless）、用容器部署最小 Agent、加一个健康检查端点、Agent 可被远程访问的接口设计，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Hosting the SDK”

练习

- 练习 1：把一个与 `Hosting the SDK` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 `notes/enhanced-44.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。

- [] 有证据和验证结果。
- [] 有风险与回滚说明。

44.2 Securely Deploying AI Agents

学习目标

- 理解 `Securely Deploying AI Agents` 解决的具体问题。
- 能把 `Securely Deploying AI Agents` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Securely Deploying AI Agents`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕部署期的安全清单、密钥管理 / 网络隔离 / 输入校验、用 secret manager 而非 `.env`、部署不留尾巴，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Securely Deploying AI Agents”

练习

- 练习 1: 把一个与 `Securely Deploying AI Agents` 相关的模糊请求改写成完整任务。
- 练习 2: 让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-44.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

44.3 Cost Tracking

学习目标

- 理解 `Cost Tracking` 解决的具体问题。
- 能把 `Cost Tracking` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Cost Tracking` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 程序级成本追踪 API、上报每次调用的 token、给团队画一张成本图、知道哪种用户最贵，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“Cost Tracking”

练习

- 练习 1：把一个与`Cost Tracking`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-44.md`。

检查清单

- ☐ 任务边界清楚。
- ☐ 有可复现命令。
- ☐ 有证据和验证结果。
- ☐ 有风险与回滚说明。

44.4 Observability with OpenTelemetry

学习目标

- 理解 `Observability with OpenTelemetry` 解决的具体问题。
- 能把 `Observability with OpenTelemetry` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`Observability with OpenTelemetry`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕接 OTLP collector / Jaeger、跟踪一次失败任务到具体工具调用、度量 / 追踪 / 日志三类信号、SLO 与告警阈值，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Observability with OpenTelemetry”

练习

- 练习 1：把一个与 `Observability with OpenTelemetry` 相关的模糊请求改写成完整任务。

- 练习 2: 让 Claude Code 执行一次只读探索, 并输出证据。
- 练习 3: 把本节经验写入 `notes/enhanced-44.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

44.5 Modifying System Prompts

学习目标

- 理解 `Modifying System Prompts` 解决的具体问题。
- 能把 `Modifying System Prompts` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用, 什么时候不要使用。

核心概念

`Modifying System Prompts`

的重点不是记住术语, 而是理解它如何改变 Agent 的工作边界。围绕自定义系统提示词的边界、替换 vs 追加 vs 合并、比较不同系统提示效果、不破坏官方安全约束, 你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据, 并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。

3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“Modifying System Prompts”

练习

- 练习 1：把一个与`Modifying System Prompts`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-44.md`。

检查清单

- ☐ 任务边界清楚。
- ☐ 有可复现命令。
- ☐ 有证据和验证结果。
- ☐ 有风险与回滚说明。

44.6 Migrate to Claude Agent SDK

学习目标

- 理解`Migrate to Claude Agent SDK`解决的具体问题。
- 能把`Migrate to Claude Agent SDK`写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。

- 能说明什么时候使用，什么时候不要使用。

核心概念

`Migrate to Claude Agent SDK`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕从旧 SDK / 类似框架迁移、跟着 migration guide 升级一次、列出 breaking change 清单、升级路径与回滚计划，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“Migrate to Claude Agent SDK”

练习

- 练习 1：把一个与 `Migrate to Claude Agent SDK` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 `notes/enhanced-44.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

常见坑

- 把新功能当成固定不变的事实，不复核官方文档。
- 没有限制工具权限，导致 Agent 执行范围过大。
- 只生成配置，不实际跑一次验证。
- 忽略成本、上下文污染和多人协作时的规则冲突。

本章小结

掌握“SDK 部署、可观测性与迁移”的标准不是看懂定义，而是能把它放进你的真实开发流程：有输入、有边界、有工具、有验证、有沉淀。

安全、权限、沙箱和风险控制

第 45 章：权限系统与 Permission Modes

- > 所属篇章：第八篇：安全、权限、沙箱和风险控制
- > 主案例语言：Python
- > 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者建立 Claude Code 使用中的最小权限意识。

本章以风险分级为主线。读者要知道不是所有权限都应该授权，尤其是删除、重写、网络、依赖、发布相关操作。团队使用时必须有明确权限规范。

学习目标

- 理解“权限系统与 Permission Modes”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-45"
claude "        "    Permission Modes"    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-45
claude "        "    Permission Modes"    "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 37.1 权限基础
 - 37.1.1 读取权限
 - 37.1.2 写入权限
 - 37.1.3 命令执行权限
- 37.2 人工确认
 - 37.2.1 删除文件
 - 37.2.2 修改依赖
 - 37.2.3 发起网络请求
- 37.3 自动模式边界

- 37.3.1 低风险任务
- 37.3.2 中风险任务
- 37.3.3 高风险任务
- 37.4 团队权限规范
- 37.4.1 默认禁止什么
- 37.4.2 什么时候允许
- 37.4.3 谁负责审核

37.1 权限基础

学习目标

- 理解 `权限基础` 在本章主题中的具体作用。
- 能把 `权限基础` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：读取权限、写入权限、命令执行权限。

核心概念

`权限基础`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `37.1.1`
读取权限：先解释它解决的问题，再给出一个可观察的操作。
- `37.1.2`
写入权限：先解释它解决的问题，再给出一个可观察的操作。

- 37.1.3

命令执行权限：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-45"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-45
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
```

```
print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `权限基础` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `权限基础` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

37.2 人工确认

学习目标

- 理解 `人工确认` 在本章主题中的具体作用。
- 能把 `人工确认` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：删除文件、修改依赖、发起网络请求。

核心概念

`人工确认`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资

料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 37.2.1

删除文件：先解释它解决的问题，再给出一个可观察的操作。

- 37.2.2

修改依赖：先解释它解决的问题，再给出一个可观察的操作。

- 37.2.3

发起网络请求：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-45"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-45
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()
```

```
def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`人工确认`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`人工确认`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

37.3 自动模式边界

学习目标

- 理解`自动模式边界`在本章主题中的具体作用。
- 能把`自动模式边界`转化为一个可执行、可验证的 Claude Code 任务。

- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：低风险任务、中风险任务、高风险任务。

核心概念

“自动模式边界”

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- “37.3.1
低风险任务”：先解释它解决的问题，再给出一个可观察的操作。
- “37.3.2
中风险任务”：先解释它解决的问题，再给出一个可观察的操作。
- “37.3.3
高风险任务”：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-45"
claude " " " "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-45
claude " " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `自动模式边界` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `自动模式边界` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

37.4 团队权限规范

学习目标

- 理解 `团队权限规范` 在本章主题中的具体作用。
- 能把 `团队权限规范` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：默认禁止什么、什么时候允许、谁负责审核。

核心概念

`团队权限规范`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `37.4.1`
默认禁止什么：先解释它解决的问题，再给出一个可观察的操作。
- `37.4.2`
什么时候允许：先解释它解决的问题，再给出一个可观察的操作。
- `37.4.3`
谁负责审核：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-45"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-45
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1: 把 `团队权限规范` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2: 要求 Claude Code 先列计划，不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `团队权限规范` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例: 对 10 个 Claude Code 操作做风险分级。
- 练习: 写最小权限策略。

- 练习：设计高风险命令确认提示。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `docs/permission-policy.md`
- 高风险操作确认模板
- 权限分级表

本章检查清单

- [] 我已经完成第 37 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“权限系统与 Permission Modes”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“沙箱、数据安全和提示注入”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>

- 记忆系统: <https://code.claude.com/docs/zh-CN/memory>
- Skills: <https://code.claude.com/docs/zh-CN/skills>

深入专题: Permission Modes 与 Auto Mode

权限模式决定 Claude Code 能不能自动读写、执行命令或请求确认。新手最容易犯的错误是为了省事直接开高权限。正确做法是按任务风险选择模式:

| 场景 | 推荐模式 |

| --- | --- |

| 只读理解项目 | plan / default |

| 小范围改文档 | acceptEdits |

| 自动化脚本 | 明确 allow/deny |

| 删除、发布、迁移 | 必须人工确认 |

Auto Mode 可以提高效率, 但不能替代安全边界。团队项目应把 deny 规则写进配置, 而不是靠每个人临场判断。

第 46 章：沙箱、数据安全和提示注入

- > 所属篇章：第八篇：安全、权限、沙箱和风险控制
- > 主案例语言：C#
- > 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者理解 Claude Code 的隔离、安全边界和提示注入风险。

本章把安全问题落到具体场景：仓库中的密钥、外部文档里的恶意指令、MCP 连接的外部系统权限。读者要学会把外部内容当作数据，不让其覆盖系统规则和项目规则。

学习目标

- 理解“沙箱、数据安全和提示注入”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 `**C#**` 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code \book\examples\chapter-46"
claude " " " "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-46
claude " " " "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 38.1 沙箱基本概念
 - 38.1.1 可读范围
 - 38.1.2 可写范围
 - 38.1.3 网络限制
- 38.2 敏感数据保护
 - 38.2.1 `.env`
 - 38.2.2 API key
 - 38.2.3 token 和凭据
- 38.3 提示注入

- 38.3.1 外部文档中的恶意指令
- 38.3.2 数据和指令的边界
- 38.3.3 Unicode 和隐藏文本风险
- 38.4 MCP 和外部工具风险
- 38.4.1 外部系统只读优先
- 38.4.2 数据访问审计
- 38.4.3 工具授权最小化

38.1 沙箱基本概念

学习目标

- 理解`沙箱基本概念`在本章主题中的具体作用。
- 能把`沙箱基本概念`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：可读范围、可写范围、网络限制。

核心概念

`沙箱基本概念`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `38.1.1`
可读范围：先解释它解决的问题，再给出一个可观察的操作。

- 38.1.2

可写范围：先解释它解决的问题，再给出一个可观察的操作。

- 38.1.3

网络限制：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-46"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-46
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
    }
}
```



```
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||  
Directory.Exists(fullPath)}");  
    }  
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `沙箱基本概念` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `沙箱基本概念` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

38.2 敏感数据保护

学习目标

- 理解 `敏感数据保护` 在本章主题中的具体作用。
- 能把 `敏感数据保护` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：`.env`、API key、token 和凭据。

核心概念

敏感数据保护

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 38.2.1 .env：先解释它解决的问题，再给出一个可观察的操作。
- 38.2.2 API key：先解释它解决的问题，再给出一个可观察的操作。
- 38.2.3 token

和凭据：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-46"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-46
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `敏感数据保护` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `敏感数据保护` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

38.3 提示注入

学习目标

- 理解 `提示注入` 在本章主题中的具体作用。
- 能把 `提示注入` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：外部文档中的恶意指令、数据和指令的边界、Unicode 和隐藏文本风险。

核心概念

`提示注入`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `38.3.1 外部文档中的恶意指令`：先解释它解决的问题，再给出一个可观察的操作。
- `38.3.2 数据和指令的边界`：先解释它解决的问题，再给出一个可观察的操作。
- `38.3.3 Unicode 和隐藏文本风险`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。

4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-46"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-46
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `提示注入` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `提示注入` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

38.4 MCP 和外部工具风险

学习目标

- 理解 `MCP 和外部工具风险` 在本章主题中的具体作用。
- 能把 `MCP 和外部工具风险` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：外部系统只读优先、数据访问审计、工具授权最小化。

核心概念

`MCP 和外部工具风险`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `38.4.1 外部系统只读优先`：先解释它解决的问题，再给出一个可观察的操作。
- `38.4.2
数据访问审计`：先解释它解决的问题，再给出一个可观察的操作。

- `38.4.3 工具授权最小化`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-46"
claude "    "MCP                "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-46
claude "    "MCP                "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `MCP 和外部工具风险` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `MCP 和外部工具风险` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：扫描项目中的敏感信息模式。
- 练习：写“外部内容不覆盖系统规则”的项目规范。
- 练习：为一个 MCP Server 写安全审核表。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- ``docs/security-checklist.md``
- ``docs/prompt-injection-policy.md``
- MCP 安全审核表

本章检查清单

- ☐ 我已经完成第 38 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“沙箱、数据安全和提示注入”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Git、安全发布和回滚”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 47 章：Git、安全发布和回滚

> 所属篇章：第八篇：安全、权限、沙箱和风险控制

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让读者掌握 AI 辅助开发中的发布前检查、回滚和事故复盘。

本章强调 AI

参与开发后更需要可回滚、可审查、可复盘。读者要学会让 Claude Code 帮助做发布检查和复盘，但最终发布判断仍由人负责。

学习目标

- 理解“Git、安全发布和回滚”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C++** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-47"
claude "    "Git    "
```

```
# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-47
claude "      "Git      "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 39.1 安全提交
 - 39.1.1 小步提交
 - 39.1.2 不混入无关文件
 - 39.1.3 提交说明可审查
- 39.2 发布前检查
 - 39.2.1 测试
 - 39.2.2 文档
 - 39.2.3 版本和 changelog
- 39.3 回滚策略
 - 39.3.1 回滚命令
 - 39.3.2 数据兼容
 - 39.3.3 用户影响
- 39.4 事故复盘

- 39.4.1 发生了什么
- 39.4.2 为什么没被提前发现
- 39.4.3 更新测试、规则或 Hook

39.1 安全提交

学习目标

- 理解 `安全提交` 在本章主题中的具体作用。
- 能把 `安全提交` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：小步提交、不混入无关文件、提交说明可审查。

核心概念

`安全提交`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `39.1.1 小步提交`：先解释它解决的问题，再给出一个可观察的操作。
- `39.1.2 不混入无关文件`：先解释它解决的问题，再给出一个可观察的操作。
- `39.1.3 提交说明可审查`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-47"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-47
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`安全提交`改写成包含目标、范围、限制、输出格式的提示词。

- 练习 2: 要求 Claude Code 先列计划, 不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令, 并记录验证结果。

检查清单

- [] 我能用自己的话解释 `安全提交` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

39.2 发布前检查

学习目标

- 理解 `发布前检查` 在本章主题中的具体作用。
- 能把 `发布前检查` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景, 并知道如何修正。
- 能说明本节三个重点: 测试、文档、版本和 changelog。

核心概念

`发布前检查`

的重点不是记住术语, 而是把它放回真实工程动作里理解。对于 Claude Code 来说, 一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解:

- `39.2.1 测试`: 先解释它解决的问题, 再给出一个可观察的操作。
- `39.2.2 文档`: 先解释它解决的问题, 再给出一个可观察的操作。

- 39.2.3 版本和

changelog：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-47"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-47
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```


可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `发布前检查` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `发布前检查` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

39.3 回滚策略

学习目标

- 理解 `回滚策略` 在本章主题中的具体作用。
- 能把 `回滚策略` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：回滚命令、数据兼容、用户影响。

核心概念

`回滚策略`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 39.3.1

回滚命令：先解释它解决的问题，再给出一个可观察的操作。

- 39.3.2

数据兼容：先解释它解决的问题，再给出一个可观察的操作。

- 39.3.3

用户影响：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-47"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-47
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
```

```

std::filesystem::path target{"README.md"};
std::cout << "Path: " << std::filesystem::absolute(target) << "
";
std::cout << "Exists: " << std::filesystem::exists(target) << "
";
return 0;
}

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `回滚策略` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `回滚策略` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

39.4 事故复盘

学习目标

- 理解 `事故复盘` 在本章主题中的具体作用。
- 能把 `事故复盘` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：发生了什么、为什么没被提前发现、更新测试、规则或 Hook。

核心概念

‘事故复盘’

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- ‘39.4.1 发生了什么’：先解释它解决的问题，再给出一个可观察的操作。
- ‘39.4.2 为什么没被提前发现’：先解释它解决的问题，再给出一个可观察的操作。
- ‘39.4.3 更新测试、规则或 Hook’：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-47"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-47
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `事故复盘` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `事故复盘` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：为一次改动写发布前检查和回滚说明。
- 练习：模拟一次失败发布的复盘。
- 练习：将事故复盘结果转化为新测试或 Hook。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `docs/release-checklist.md`
- `docs/rollback-plan-template.md`
- `docs/incident-review-template.md`

本章检查清单

- [] 我已经完成第 39 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。

- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“Git、安全发布和回滚”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“生产级 Claude Code 使用规范”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 48 章：生产级 Claude Code 使用规范与可观测性与可观测性

- > 所属篇章：第八篇：安全、权限、沙箱和风险控制
- > 主案例语言：Python
- > 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

把个人和团队的 Claude Code 使用经验整理成正式规范。

本章是安全篇的小结。读者要把分散的规则、模板、权限、Hook、PR 检查和复盘机制整理成一份生产级使用规范，并用成熟度模型评估自己当前阶段。

学习目标

- 理解“生产级 Claude Code 使用规范与可观测性”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。


```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-48"
claude "    "    Claude Code    "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-48
claude "    "    Claude Code    "    "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 40.1 个人规范
 - 40.1.1 任务前检查
 - 40.1.2 修改中审查
 - 40.1.3 完成后验证
- 40.2 团队规范
 - 40.2.1 统一规则文件
 - 40.2.2 统一 PR 模板
 - 40.2.3 统一安全边界
- 40.3 审计与可观测性

- 40.3.1 任务日志
- 40.3.2 命令日志
- 40.3.3 变更记录
- 40.4 成熟度模型
- 40.4.1 入门
- 40.4.2 熟练
- 40.4.3 高级
- 40.4.4 工程化

40.1 个人规范

学习目标

- 理解`个人规范`在本章主题中的具体作用。
- 能把`个人规范`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：任务前检查、修改中审查、完成后验证。

核心概念

`个人规范`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `40.1.1`
任务前检查：先解释它解决的问题，再给出一个可观察的操作。

- 40.1.2

修改中审查：先解释它解决的问题，再给出一个可观察的操作。

- 40.1.3

完成后验证：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-48"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-48
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
```

```
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `个人规范` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `个人规范` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

40.2 团队规范

学习目标

- 理解 `团队规范` 在本章主题中的具体作用。
- 能把 `团队规范` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：统一规则文件、统一 PR 模板、统一安全边界。

核心概念

团队规范

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 40.2.1

统一规则文件：先解释它解决的问题，再给出一个可观察的操作。

- 40.2.2 统一 PR

模板：先解释它解决的问题，再给出一个可观察的操作。

- 40.2.3

统一安全边界：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-48"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-48
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `团队规范` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `团队规范` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

40.3 审计与可观测性

学习目标

- 理解 `审计与可观测性` 在本章主题中的具体作用。
- 能把 `审计与可观测性` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：任务日志、命令日志、变更记录。

核心概念

`审计与可观测性`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `40.3.1`
任务日志：先解释它解决的问题，再给出一个可观察的操作。
- `40.3.2`
命令日志：先解释它解决的问题，再给出一个可观察的操作。
- `40.3.3`
变更记录：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。

4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-48"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-48
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `审计与可观测性` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `审计与可观测性` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

40.4 成熟度模型

学习目标

- 理解 `成熟度模型` 在本章主题中的具体作用。
- 能把 `成熟度模型` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：入门、熟练、高级。

核心概念

`成熟度模型`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `40.4.1 入门`：先解释它解决的问题，再给出一个可观察的操作。
- `40.4.2 熟练`：先解释它解决的问题，再给出一个可观察的操作。
- `40.4.3 高级`：先解释它解决的问题，再给出一个可观察的操作。
- `40.4.4 工程化`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-48"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-48
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `成熟度模型` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `成熟度模型` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：编写团队 Claude Code 使用规范。
- 练习：用成熟度模型给自己评分。
- 练习：审查规范是否可执行。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `docs/claude-code-team-guide.md`
- `docs/claude-code-maturity-model.md`
- 团队规范检查清单

本章检查清单

- ☐ 我已经完成第 40 章的先导案例。
- ☐ 我已经记录本章至少 3 条可复用经验。
- ☐ 我能解释本章每个小节解决的问题。
- ☐ 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- ☐ 我知道本章方法什么时候不适用。

本章小结

本章围绕“生产级 Claude Code 使用规范与可观测性”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“Claude Code 内部工作机制导览”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览: <https://code.claude.com/docs/zh-CN/overview>
- 记忆系统: <https://code.claude.com/docs/zh-CN/memory>
- Skills: <https://code.claude.com/docs/zh-CN/skills>

深入专题：生产级可观测性

生产级 Claude Code 使用必须能回答三个问题：花了多少钱、做了什么、哪里失败。至少需要记录任务目标、模型、工具调用、读写文件、命令输出摘要、token 或成本、最终验证结果。

最小日志字段：

- task_id
- user_goal
- model
- tools_used
- files_changed
- tests_run
- result
- risk_notes

高级驾驭工程 - 从会用到会设计 Agent 工作系统

第 49 章：Claude Code

内部工作机制导览

> 所属篇章：第九篇：高级驾驭工程 - 从会用到会设计 Agent 工作系统

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

让高级读者理解 Claude Code 的内部机制如何影响实际使用策略。

本章从源码和架构视角解释 Agent Loop、工具执行、权限和系统提示词。目标不是让读者成为 Claude Code 源码专家，而是让读者理解为什么前面章节强调探索、权限、验证和上下文管理。

学习目标

- 理解“Claude Code 内部工作机制导览”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C#** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-49"
claude "    "Claude Code    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-49
claude "    "Claude Code    "
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 41.1 Agent Loop
 - 41.1.1 用户输入
 - 41.1.2 模型推理
 - 41.1.3 工具调用
 - 41.1.4 观察结果并继续
- 41.2 工具执行编排

- 41.2.1 权限确认
- 41.2.2 并发和中断
- 41.2.3 流式输出
- 41.3 系统提示词作为控制面
- 41.3.1 系统规则
- 41.3.2 项目规则
- 41.3.3 用户当前指令
- 41.4 对 Claude Code 源码研究的使用方式
- 41.4.1 从源码提炼原则
- 41.4.2 不依赖未发布细节
- 41.4.3 将原理转化为实践规则

41.1 Agent Loop

学习目标

- 理解 `Agent Loop` 在本章主题中的具体作用。
- 能把 `Agent Loop` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：用户输入、模型推理、工具调用。

核心概念

`Agent Loop`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 41.1.1
用户输入：先解释它解决的问题，再给出一个可观察的操作。
- 41.1.2
模型推理：先解释它解决的问题，再给出一个可观察的操作。
- 41.1.3
工具调用：先解释它解决的问题，再给出一个可观察的操作。
- 41.1.4 观察结果并继续：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-49"
claude "    "Agent Loop"    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-49
claude "    "Agent Loop"    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
```

```
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Agent Loop` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Agent Loop` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

41.2 工具执行编排

学习目标

- 理解 `工具执行编排` 在本章主题中的具体作用。

- 能把`工具执行编排`转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：权限确认、并发和中断、流式输出。

核心概念

`工具执行编排`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `41.2.1`
权限确认：先解释它解决的问题，再给出一个可观察的操作。
- `41.2.2`
并发和中断：先解释它解决的问题，再给出一个可观察的操作。
- `41.2.3`
流式输出：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
```

```
Set-Location "D:\AI\claude code    \book\examples\chapter-49"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-49
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `工具执行编排` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `工具执行编排` 解决什么问题。

- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

41.3 系统提示词作为控制面

学习目标

- 理解 `系统提示词作为控制面` 在本章主题中的具体作用。
- 能把 `系统提示词作为控制面` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：系统规则、项目规则、用户当前指令。

核心概念

`系统提示词作为控制面`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `41.3.1`
系统规则：先解释它解决的问题，再给出一个可观察的操作。
- `41.3.2`
项目规则：先解释它解决的问题，再给出一个可观察的操作。
- `41.3.3`
用户当前指令：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-49"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-49
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1: 把 `系统提示词作为控制面` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2: 要求 Claude Code 先列计划，不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `系统提示词作为控制面` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

41.4 对 Claude Code 源码研究的使用方式

学习目标

- 理解 `对 Claude Code 源码研究的使用方式` 在本章主题中的具体作用。
- 能把 `对 Claude Code 源码研究的使用方式` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：从源码提炼原则、不依赖未发布细节、将原理转化为实践规则。

核心概念

`对 Claude Code 源码研究的使用方式`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资

料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `41.4.1 从源码提炼原则`：先解释它解决的问题，再给出一个可观察的操作。
- `41.4.2 不依赖未发布细节`：先解释它解决的问题，再给出一个可观察的操作。
- `41.4.3 将原理转化为实践规则`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C#**`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-49"
claude "    " Claude Code                "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-49
claude "    " Claude Code                "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;
```

```
public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}
```

```
ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把`对 Claude Code 源码研究的使用方式`改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释`对 Claude Code 源码研究的使用方式`解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：追踪一次“读取文件 -> 修改 -> 测试 -> 总结”的 Agent Loop。
- 练习：写出每轮循环的输入、工具和观察结果。
- 练习：把一个源码启发转化为项目规则。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `notes/agent-loop-trace.md`
- `docs/source-inspired-principles.md`
- 指令优先级说明

本章检查清单

- [] 我已经完成第 41 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。

- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“Claude Code 内部工作机制导览”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“上下文工程、缓存和成本工程”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 50 章：上下文工程、缓存和成本工程

> 所属篇章：第九篇：高级驾驭工程 - 从会用到会设计 Agent 工作系统

> 主案例语言：C++

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

把上下文、缓存和成本从技巧提升为系统设计能力。

本章从系统层面总结前面的 token 和上下文内容。读者要把项目文档、规则、知识库、提示词和日志都看成上下文工程的一部分。缓存友好不是单个技巧，而是稳定资料结构和减少无效变动的结果。

学习目标

- 理解“上下文工程、缓存和成本工程”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C++** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 ->

获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-50"
claude "        "        "        "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-50
claude "        "        "        "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 42.1 上下文工程原则
 - 42.1.1 输入质量
 - 42.1.2 结构化资料
 - 42.1.3 按需加载
- 42.2 微压缩
 - 42.2.1 长日志压缩
 - 42.2.2 长文档压缩
 - 42.2.3 长对话压缩
- 42.3 缓存友好项目结构
 - 42.3.1 稳定规则文件

- 42.3.2 任务临时文件
- 42.3.3 少改大文件
- 42.4 成本决策
- 42.4.1 任务价值
- 42.4.2 失败成本
- 42.4.3 模型和流程选择

42.1 上下文工程原则

学习目标

- 理解 `上下文工程原则` 在本章主题中的具体作用。
- 能把 `上下文工程原则` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：输入质量、结构化资料、按需加载。

核心概念

`上下文工程原则`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `42.1.1
输入质量`：先解释它解决的问题，再给出一个可观察的操作。
- `42.1.2
结构化资料`：先解释它解决的问题，再给出一个可观察的操作。

- 42.1.3

按需加载：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-50"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-50
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```


可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `上下文工程原则` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `上下文工程原则` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

42.2 微压缩

学习目标

- 理解 `微压缩` 在本章主题中的具体作用。
- 能把 `微压缩` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：长日志压缩、长文档压缩、长对话压缩。

核心概念

`微压缩`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 42.2.1

长日志压缩：先解释它解决的问题，再给出一个可观察的操作。

- 42.2.2

长文档压缩：先解释它解决的问题，再给出一个可观察的操作。

- 42.2.3

长对话压缩：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-50"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-50
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
```

```

std::filesystem::path target{"README.md"};
std::cout << "Path: " << std::filesystem::absolute(target) << "
";
std::cout << "Exists: " << std::filesystem::exists(target) << "
";
return 0;
}

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `微压缩` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `微压缩` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

42.3 缓存友好项目结构

学习目标

- 理解 `缓存友好项目结构` 在本章主题中的具体作用。
- 能把 `缓存友好项目结构` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：稳定规则文件、任务临时文件、少改大文件。

核心概念

缓存友好项目结构

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 42.3.1
稳定规则文件：先解释它解决的问题，再给出一个可观察的操作。
- 42.3.2
任务临时文件：先解释它解决的问题，再给出一个可观察的操作。
- 42.3.3
少改大文件：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" && cd \book\examples\chapter-50
claude " " " "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-50
claude " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `缓存友好项目结构` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `缓存友好项目结构` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

42.4 成本决策

学习目标

- 理解 `成本决策` 在本章主题中的具体作用。
- 能把 `成本决策` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：任务价值、失败成本、模型和流程选择。

核心概念

`成本决策`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `42.4.1
任务价值`：先解释它解决的问题，再给出一个可观察的操作。
- `42.4.2
失败成本`：先解释它解决的问题，再给出一个可观察的操作。
- `42.4.3 模型和流程选择`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。

4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-50"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-50
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `成本决策` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `成本决策` 解决什么问题。

- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：将一个混乱项目文档重构为缓存友好结构。
- 练习：把长日志压缩为故障摘要。
- 练习：为 5 类任务制定成本策略。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `docs/context-engineering-guide.md`
- `docs/cache-friendly-structure.md`

- 成本决策表

本章检查清单

- [] 我已经完成第 42 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“上下文工程、缓存和成本工程”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“从个人能力到团队 AI 编码体系”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

第 51 章：从个人能力到团队 AI 编码体系

> 所属篇章：第九篇：高级驾驭工程 - 从会用到会设计 Agent 工作系统

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

帮助读者把个人 Claude Code 使用经验升级为团队可复用体系。

本章是高级篇总结。读者要从“我会用 Claude Code”升级为“我能设计一套 Claude Code 工作系统”。重点是可复制、可培训、可审计、可持续改进。

学习目标

- 理解“从个人能力到团队 AI 编码体系”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 Python 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **Python** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 ->

获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-51"
claude "          AI          "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-51
claude "          AI          "
```

案例中的最小代码对象如下：

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

目录

- 43.1 个人 AI 操作系统
 - 43.1.1 规则
 - 43.1.2 提示词
 - 43.1.3 Skill 和命令
 - 43.1.4 自动化
- 43.2 团队资产沉淀
 - 43.2.1 模板
 - 43.2.2 插件

- 43.2.3 规范和培训材料
- 43.3 持续改进
 - 43.3.1 失败复盘
 - 43.3.2 规则更新
 - 43.3.3 工具包迭代
- 43.4 高级用户画像
 - 43.4.1 能设计流程
 - 43.4.2 能控制风险
 - 43.4.3 能沉淀团队能力

43.1 个人 AI 操作系统

学习目标

- 理解 `个人 AI 操作系统` 在本章主题中的具体作用。
- 能把 `个人 AI 操作系统` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：规则、提示词、Skill 和命令。

核心概念

`个人 AI 操作系统`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `43.1.1 规则`：先解释它解决的问题，再给出一个可观察的操作。

- `43.1.2 提示词`：先解释它解决的问题，再给出一个可观察的操作。
- `43.1.3 Skill
和命令`：先解释它解决的问题，再给出一个可观察的操作。
- `43.1.4 自动化`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-51"
claude "    "    AI    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-51
claude "    "    AI    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
```

```
        "suffix": target.suffix,
    }

    if __name__ == "__main__":
        print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `个人 AI 操作系统` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `个人 AI 操作系统` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

43.2 团队资产沉淀

学习目标

- 理解 `团队资产沉淀` 在本章主题中的具体作用。
- 能把 `团队资产沉淀` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：模板、插件、规范和培训材料。

核心概念

团队资产沉淀

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 43.2.1 模板：先解释它解决的问题，再给出一个可观察的操作。
- 43.2.2 插件：先解释它解决的问题，再给出一个可观察的操作。
- 43.2.3 规范和培训材料：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-51"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-51
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path
```

```
PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `团队资产沉淀` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `团队资产沉淀` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

43.3 持续改进

学习目标

- 理解 `持续改进` 在本章主题中的具体作用。
- 能把 `持续改进` 转化为一个可执行、可验证的 Claude Code 任务。

- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：失败复盘、规则更新、工具包迭代。

核心概念

“持续改进”

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 43.3.1
失败复盘：先解释它解决的问题，再给出一个可观察的操作。
- 43.3.2
规则更新：先解释它解决的问题，再给出一个可观察的操作。
- 43.3.3
工具包迭代：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" \book\examples\chapter-51"
claude " " " "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-51
claude " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `持续改进` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `持续改进` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

43.4 高级用户画像

学习目标

- 理解 `高级用户画像` 在本章主题中的具体作用。
- 能把 `高级用户画像` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：能设计流程、能控制风险、能沉淀团队能力。

核心概念

`高级用户画像`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `43.4.1 能设计流程`：先解释它解决的问题，再给出一个可观察的操作。
- `43.4.2 能控制风险`：先解释它解决的问题，再给出一个可观察的操作。
- `43.4.3 能沉淀团队能力`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **Python**。用一个小型 Python CLI 或脚本项目作为观察对象，重点展示文件读取、测试和自动化流程。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-51"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-51
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()

def summarize_target(path: str) -> dict:
    target = PROJECT_ROOT / path
    return {
        "path": str(target),
        "exists": target.exists(),
        "suffix": target.suffix,
    }

if __name__ == "__main__":
    print(summarize_target("README.md"))
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1: 把 `高级用户画像` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2: 要求 Claude Code 先列计划，不直接修改文件。
- 练习 3: 让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `高级用户画像` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：建立个人 Claude Code starter kit。
- 练习：把个人 starter kit 改造成团队 toolkit。

- 练习：用成熟度模型制定下一阶段学习计划。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `starter-kit/`
- `team-toolkit/`
- `docs/next-stage-plan.md`

本章检查清单

- [] 我已经完成第 43 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“从个人能力到团队 AI 编码体系”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“实战一 - 个人 Python 项目从 Bug 到 PR”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>

- 记忆系统: <https://code.claude.com/docs/zh-CN/memory>
- Skills: <https://code.claude.com/docs/zh-CN/skills>

完整实战案例

第 52 章：实战一 - Python 项目从 Bug 到 PR（个人完整链路）

> 所属篇章：第十篇：完整实战案例

> 主案例语言：C#

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

通过完整个人项目，把前面学到的代码理解、Bug 修复、测试、记忆、Git 和 PR 流程串起来。

本章是个人实战案例，必须完整闭环。读者从项目初始化开始，编写规则和记忆，修复真实 Bug，补充测试，生成 PR 描述，最后把成功流程沉淀为模板。

学习目标

- 理解“实战一 - 个人 Python 项目从 Bug 到 PR”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C# 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 `**C#**` 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 ->

获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-52"
claude "      " - Python Bug PR"

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-52
claude "      " - Python Bug PR"
```

案例中的最小代码对象如下：

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

目录

- 44.1 项目初始化
 - 44.1.1 创建或选择 Python 项目
 - 44.1.2 添加测试和文档
 - 44.1.3 制造可控 Bug
- 44.2 规则与记忆
 - 44.2.1 编写 `CLAUDE.md`
 - 44.2.2 添加测试规则
 - 44.2.3 添加 Git 规则

- 44.3 Bug 定位与修复
 - 44.3.1 读取失败日志
 - 44.3.2 建立假设
 - 44.3.3 先写测试再修复
 - 44.3.4 运行验证
- 44.4 PR 与复盘
 - 44.4.1 生成 PR 描述
 - 44.4.2 更新知识库
 - 44.4.3 将流程沉淀为命令或 Skill

44.1 项目初始化

学习目标

- 理解 `项目初始化` 在本章主题中的具体作用。
- 能把 `项目初始化` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：创建或选择 Python 项目、添加测试和文档、制造可控 Bug。

核心概念

`项目初始化`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 44.1.1 创建或选择 Python

项目：先解释它解决的问题，再给出一个可观察的操作。

- 44.1.2 添加测试和文档：先解释它解决的问题，再给出一个可观察的操作。

- 44.1.3 制造可控

Bug：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-52"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-52
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
```

```

    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `项目初始化` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `项目初始化` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

44.2 规则与记忆

学习目标

- 理解 `规则与记忆` 在本章主题中的具体作用。
- 能把 `规则与记忆` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。

- 能说明本节三个重点：编写 `CLAUDE.md`、添加测试规则、添加 Git 规则。

核心概念

规则与记忆

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 44.2.1 编写
`CLAUDE.md`：先解释它解决的问题，再给出一个可观察的操作。
- 44.2.2
添加测试规则：先解释它解决的问题，再给出一个可观察的操作。
- 44.2.3 添加 Git
规则：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C#`。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code" && cd \book\examples\chapter-52
claude " " " "

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-52
claude " " " "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `规则与记忆` 改写成一个包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `规则与记忆` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。

- [] 我能判断 Claude Code 的回答是否基于真实证据。

44.3 Bug 定位与修复

学习目标

- 理解 `Bug 定位与修复` 在本章主题中的具体作用。
- 能把 `Bug 定位与修复` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：读取失败日志、建立假设、先写测试再修复。

核心概念

`Bug 定位与修复`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `44.3.1
读取失败日志`：先解释它解决的问题，再给出一个可观察的操作。
- `44.3.2
建立假设`：先解释它解决的问题，再给出一个可观察的操作。
- `44.3.3 先写测试再修复`：先解释它解决的问题，再给出一个可观察的操作。
- `44.3.4
运行验证`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-52"
claude "    "Bug                "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-52
claude "    "Bug                "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;

public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `Bug 定位与修复` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `Bug 定位与修复` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

44.4 PR 与复盘

学习目标

- 理解 `PR 与复盘` 在本章主题中的具体作用。
- 能把 `PR 与复盘` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：生成 PR 描述、更新知识库、将流程沉淀为命令或 Skill。

核心概念

`PR 与复盘`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资

料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 44.4.1 生成 PR

描述：先解释它解决的问题，再给出一个可观察的操作。

- 44.4.2

更新知识库：先解释它解决的问题，再给出一个可观察的操作。

- 44.4.3 将流程沉淀为命令或

Skill：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C#**。用一个 C# 控制台或 Web API 项目作为观察对象，重点展示项目结构、构建命令和类型约束。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-52"
claude "    "PR    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-52
claude "    "PR    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
using System;
using System.IO;
```

```
public static class ProjectInspector
{
    public static void Summarize(string path)
    {
        var fullPath = Path.GetFullPath(path);
        Console.WriteLine($"Path: {fullPath}");
        Console.WriteLine($"Exists: {File.Exists(fullPath) ||
Directory.Exists(fullPath)}");
    }
}

ProjectInspector.Summarize("README.md");
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `PR 与复盘` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `PR 与复盘` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

|---|---|---|

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 |
缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 |
在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 |
要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到
Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：完成一个 Python CLI 或 API 项目的 Bug 修复 PR。
- 练习：人工审查 Claude Code 生成的 diff。
- 练习：把修复过程写入知识库。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- 一个完整 Bug 修复分支
- `CLAUDE.md`
- 测试文件
- PR 描述
- 复盘文档

本章检查清单

- [] 我已经完成第 44 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。

- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“实战一 - 个人 Python 项目从 Bug 到 PR”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code 在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

下一章将进入“实战二 - 团队 Claude Code 工具包”，继续把本章方法扩展到新的 Claude Code 使用场景。

本章参考

- 官方总览: <https://code.claude.com/docs/zh-CN/overview>
- 记忆系统: <https://code.claude.com/docs/zh-CN/memory>
- Skills: <https://code.claude.com/docs/zh-CN/skills>

第 53 章：实战二 - C# Web API + C++ 库混合项目的团队 Claude Code 工具包

- > 所属篇章：第十篇：完整实战案例
- > 主案例语言：C++
- > 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

通过团队工具包项目，把 Skill、SubAgent、Hook、MCP、CI 和插件化分发串成一个完整系统。

本章是团队级综合案例。读者需要先设计需求，不直接堆功能。工具包要能回答：谁使用、解决什么问题、包含哪些能力、如何安装、如何验证、如何维护。

学习目标

- 理解“实战二 - C# Web API + C++ 库混合项目的团队 Claude Code 工具包”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为一个可交给 Claude Code 执行的小任务。
- 能使用 PowerShell 命令完成主流程，并能读懂 macOS/Linux 对照命令。
- 能用 C++ 示例完成本章案例，并知道如何迁移到其他语言项目。
- 能根据检查清单判断任务是否真正完成。

先做一个小案例

本章先使用 **C++** 项目做一个最小案例。目标不是一次性完成复杂工程，而是训练“提出明确任务 -> 让 Claude Code 探索 -> 获取可验证输出 -> 记录结果”的闭环。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-53"
claude "      "      - C# Web API + C++      Claude Code
      "      "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-53
claude "      "      - C# Web API + C++      Claude Code
      "      "      "
```

案例中的最小代码对象如下：

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

目录

- 45.1 需求设计
 - 45.1.1 团队角色和使用场景
 - 45.1.2 review、test、docs、security 四类能力
 - 45.1.3 工具包边界
- 45.2 构建 Skill 与 SubAgents
 - 45.2.1 code-review Skill
 - 45.2.2 test-runner Agent

- 45.2.3 doc-writer Agent
- 45.2.4 security-scanner Agent
- 45.3 加入 Hooks、MCP 和 CI
- 45.3.1 安全和质量 Hook
- 45.3.2 本地知识库 MCP
- 45.3.3 CI 失败分析流程
- 45.4 插件化分发
- 45.4.1 插件目录结构
- 45.4.2 安装说明
- 45.4.3 版本和升级策略

45.1 需求设计

学习目标

- 理解 `需求设计` 在本章主题中的具体作用。
- 能把 `需求设计` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：团队角色和使用场景、review、test、docs、security 四类能力、工具包边界。

核心概念

`需求设计`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 45.1.1 团队角色和使用场景：先解释它解决的问题，再给出一个可观察的操作。
- 45.1.2 review、test、docs、security 四类能力：先解释它解决的问题，再给出一个可观察的操作。
- 45.1.3 工具包边界：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-53"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-53
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
```

```

";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}

```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `需求设计` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `需求设计` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

45.2 构建 Skill 与 SubAgents

学习目标

- 理解 `构建 Skill 与 SubAgents` 在本章主题中的具体作用。
- 能把 `构建 Skill 与 SubAgents` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：code-review Skill、test-runner Agent、doc-writer Agent。

核心概念

构建 Skill 与 SubAgents

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- 45.2.1 code-review

Skill：先解释它解决的问题，再给出一个可观察的操作。

- 45.2.2 test-runner

Agent：先解释它解决的问题，再给出一个可观察的操作。

- 45.2.3 doc-writer

Agent：先解释它解决的问题，再给出一个可观察的操作。

- 45.2.4 security-scanner

Agent：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `C++`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-53"
claude "    Skill    SubAgents"

# macOS / Linux
```

```
cd ~/claude-code-handbook/book/examples/chapter-53
claude "    " Skill SubAgents"
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `构建 Skill 与 SubAgents` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `构建 Skill 与 SubAgents` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

45.3 加入 Hooks、MCP 和 CI

学习目标

- 理解 `加入 Hooks、MCP 和 CI` 在本章主题中的具体作用。
- 能把 `加入 Hooks、MCP 和 CI` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：安全和质量 Hook、本地知识库 MCP、CI 失败分析流程。

核心概念

`加入 Hooks、MCP 和 CI`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `45.3.1 安全和质量 Hook`：先解释它解决的问题，再给出一个可观察的操作。
- `45.3.2 本地知识库 MCP`：先解释它解决的问题，再给出一个可观察的操作。
- `45.3.3 CI 失败分析流程`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 `**C++**`。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。

2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-53"
claude "    "    Hooks MCP    CI"    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-53
claude "    "    Hooks MCP    CI"    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `加入 Hooks、MCP 和 CI` 改写成包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `加入 Hooks、MCP 和 CI` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

45.4 插件化分发

学习目标

- 理解 `插件化分发` 在本章主题中的具体作用。
- 能把 `插件化分发` 转化为一个可执行、可验证的 Claude Code 任务。
- 能识别这一节常见的误用场景，并知道如何修正。
- 能说明本节三个重点：插件目录结构、安装说明、版本和升级策略。

核心概念

`插件化分发`

的重点不是记住术语，而是把它放回真实工程动作里理解。对于 Claude Code 来说，一个好的任务必须同时说明目标、边界、可用资料、允许执行的动作和验收方式。

本节可以按下面的顺序理解：

- `45.4.1
插件目录结构`：先解释它解决的问题，再给出一个可观察的操作。
- `45.4.2
安装说明`：先解释它解决的问题，再给出一个可观察的操作。

- `45.4.3 版本和升级策略`：先解释它解决的问题，再给出一个可观察的操作。

本节主案例语言是 **C++**。用一个 C++ 命令行或库项目作为观察对象，重点展示构建、边界条件和跨平台命令差异。

操作步骤

1. 明确当前任务只覆盖本节范围，不把后续章节内容提前展开。
2. 让 Claude Code 先读取或搜索与任务相关的最小资料集。
3. 要求 Claude Code 输出它基于哪些文件、命令或文档得出结论。
4. 让 Claude Code 给出可执行步骤，并在执行前说明风险。
5. 完成后用检查清单验证结果，而不是只看回答是否流畅。

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\chapter-53"
claude "    "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/chapter-53
claude "    "
```

示例

下面的示例不是为了展示复杂代码，而是为了给 Claude Code 一个可以读取、运行或解释的最小对象。

```
#include <filesystem>
#include <iostream>

int main() {
    std::filesystem::path target{"README.md"};
    std::cout << "Path: " << std::filesystem::absolute(target) << "
";
    std::cout << "Exists: " << std::filesystem::exists(target) << "
";
    return 0;
}
```

可以把这段代码交给 Claude Code，并要求它只完成一个小目标：解释入口、指出潜在问题、补一个测试，或者生成一段文档。

练习

- 练习 1：把 `插件化分发` 改写成一个包含目标、范围、限制、输出格式的提示词。
- 练习 2：要求 Claude Code 先列计划，不直接修改文件。
- 练习 3：让 Claude Code 给出验证命令，并记录验证结果。

检查清单

- [] 我能用自己的话解释 `插件化分发` 解决什么问题。
- [] 我能写出一个不会让 Claude Code 误解的任务说明。
- [] 我能指出本节任务的输入、输出、风险和验证方式。
- [] 我能判断 Claude Code 的回答是否基于真实证据。

常见坑与排查

| 常见问题 | 表现 | 处理方式 |

| --- | --- | --- |

| 任务描述过宽 | Claude Code 一次读取过多文件或输出泛泛总结 | 缩小范围，明确只处理一个小节或一个文件 |

| 缺少验收标准 | 看起来完成，但无法判断是否正确 | 在提示词中加入测试、检查清单或输出格式 |

| 忽略上下文边界 | 模型引用无关资料或过期规则 | 要求列出依据文件，并清理无关上下文 |

| 没有记录结果 | 下次还要重新解释同一规则 | 将有效流程沉淀到 Markdown、命令或 Skill 中 |

本章案例与练习

- 案例：构建团队 Claude Code toolkit 插件。
- 练习：用一个真实 diff 验证 reviewer、tester、doc-writer 是否工作。
- 练习：写插件发布说明和升级说明。

练习答案不要求唯一，但必须满足三个条件：任务边界清楚、输出可验证、结果能沉淀。

本章交付物

- `team-claude-toolkit/`
- `team-claude-toolkit/README.md`
- `team-claude-toolkit/CHANGELOG.md`
- 团队安装与使用手册

本章检查清单

- [] 我已经完成第 45 章的先导案例。
- [] 我已经记录本章至少 3 条可复用经验。
- [] 我能解释本章每个小节解决的问题。
- [] 我能把本章方法迁移到 Python、C# 或 C++ 项目。
- [] 我知道本章方法什么时候不适用。

本章小结

本章围绕“实战二 - C# Web API + C++ 库混合项目的团队 Claude Code 工具包”建立了一套可执行学习流程。真正的掌握标准不是记住概念，而是能让 Claude Code

在明确边界内完成任务，并通过证据、测试或检查清单验证结果。

下一章衔接

到这里，全书正文已经完成。后续应结合附录、模板和工具清单持续打磨个人或团队的 Claude Code 工作系统。

本章参考

- 官方总览：<https://code.claude.com/docs/zh-CN/overview>
- 记忆系统：<https://code.claude.com/docs/zh-CN/memory>
- Skills：<https://code.claude.com/docs/zh-CN/skills>

深入专题：C# Web API + C++ 库混合项目

团队工具包实战应覆盖跨语言边界。一个典型结构是：C# Web API 负责 HTTP 入口，C++ 库负责核心算法或性能敏感模块。Claude Code 的任务不是“随便改两个项目”，而是先画出边界：

- C#：Controller、DTO、服务层、测试入口。
- C++：头文件、实现文件、构建脚本、单元测试。
- 边界：输入输出结构、错误码、异常映射、性能约束。

练习：让 Claude Code 只读取接口定义和测试，不读取全部实现，先输出跨语言调用链，再决定修改点。

第 54 章：实战三 - 用 Agent SDK 写一个自定义 Agent

> 主案例语言：Python

> 命令示例：Windows PowerShell 为主，macOS/Linux 对照。

本章导读

本章介绍“实战三 - 用 Agent SDK 写一个自定义 Agent”的核心概念，并通过可验证的小任务帮助读者建立实践基础。它不只解释“实战三 - 用 Agent SDK 写一个自定义

Agent是什么”，还要求读者完成一个可复盘的小任务，把经验沉淀到 Markdown、Skill、SubAgent、Hook、MCP 或自动化脚本中。

学习目标

- 理解“实战三 - 用 Agent SDK 写一个自定义 Agent”在 Claude Code 学习路线中的位置。
- 能把本章知识转化为可交给 Claude Code 执行的小任务。
- 能用 PowerShell 完成主流程，并读懂 macOS/Linux 对照命令。
- 能识别本章能力的适用场景、风险和不适用场景。
- 能用检查清单判断任务是否真正完成。

先做一个小案例

```
# Windows PowerShell
Set-Location "D:\AI\claude code    \book\examples\enhanced"
claude "

# macOS / Linux
cd ~/claude-code-handbook/book/examples/enhanced
claude "
```

案例中的最小代码对象如下：

```
from pathlib import Path

target = Path("README.md").resolve()
print({"target": str(target), "exists": target.exists()})
```

目录

- 54.1 选题与边界
 - 54.1.1 设计一个「最小但有用」的 Agent
 - 54.1.2 候选选题：CI 失败 → 假设 → 修复建议 PR 评论；或：每日 PR 摘要机器人；或：自动化文档同步 Agent
 - 54.1.3 选题判断标准：范围小、价值清晰、可演示
 - 54.1.4 写 1 页设计文档
- 54.2 SDK 骨架
 - 54.2.1 用 Python 或 TypeScript SDK 写最小 loop
 - 54.2.2 加 1 个自定义工具（如 `fetch\_pr\_diff`）
 - 54.2.3 加 1 个团队 Skill 复用
 - 54.2.4 程序能跑通一次完整任务
- 54.3 权限、Hook、可观测性
 - 54.3.1 限制工具到最小集合
 - 54.3.2 加审计 Hook 记录每次调用
 - 54.3.3 接 OpenTelemetry 跟踪
 - 54.3.4 加 cost tracking 报表
 - 54.3.5 模拟 3 种风险输入（恶意 PR 描述 / 提示注入 / 工具失败）
- 54.4 部署：让 Agent 跑起来（核心）

- 54.4.1 用 GitHub Actions 或一台小服务器部署
- 54.4.2 写运维 runbook（启动 / 监控 / 故障处理）
- 54.4.3 一次成功的部署演示
- 54.4.4 这是一个真实可用的 Agent，不是 demo
- 54.5 进阶挑战：把 Agent 当产品运营（可选）
- 54.5.1 让 Agent 跑一周，记录所有调用、失败、成本
- 54.5.2 输出第一周运行周报
- 54.5.3 第一次真实 bugfix —— 找到问题并修复
- 54.5.4 持续迭代：v0.2 / v0.3 路线

54.1 选题与边界

学习目标

- 理解 `选题与边界` 解决的具体问题。
- 能把 `选题与边界` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`选题与边界` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 设计一个「最小但有用」的 Agent、候选选题：CI 失败 → 假设 → 修复建议 PR 评论；或：每日 PR 摘要机器人；或：自动化文档同步 Agent、选题判断标准：范围小、价值清晰、可演示、写 1 页设计文档，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“ ”

练习

- 练习 1：把一个与`选题与边界`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-54.md`。

检查清单

- ☐ 任务边界清楚。
- ☐ 有可复现命令。
- ☐ 有证据和验证结果。
- ☐ 有风险与回滚说明。

54.2 SDK 骨架

学习目标

- 理解`SDK 骨架`解决的具体问题。

- 能把 `SDK 骨架` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`SDK 骨架` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕用 Python 或 TypeScript SDK 写最小 loop、加 1 个自定义工具（如 `fetch\_pr\_diff`）、加 1 个团队 Skill 复用、程序能跑通一次完整任务，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“SDK ”

练习

- 练习 1：把一个与 `SDK 骨架` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 `notes/enhanced-54.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

54.3 权限、Hook、可观测性

学习目标

- 理解 `权限、Hook、可观测性` 解决的具体问题。
- 能把 `权限、Hook、可观测性`
 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`权限、Hook、可观测性` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕 限制工具到最小集合、加审计 Hook 记录每次调用、接 OpenTelemetry 跟踪、加 cost tracking 报表、模拟 3 种风险输入（恶意 PR 描述 / 提示注入 / 工具失败），你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。

4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“ Hook ”

练习

- 练习 1：把一个与`权限、Hook、可观测性`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`notes/enhanced-54.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

54.4 部署：让 Agent 跑起来（核心）

学习目标

- 理解`部署：让 Agent 跑起来（核心）`解决的具体问题。
- 能把`部署：让 Agent 跑起来（核心）`写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`部署：让 Agent 跑起来（核心）`

的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕用 GitHub Actions 或一台小服务器部署、写运维 runbook（启动 / 监控 / 故障处理）、一次成功的部署演示、这是一个真实可用的 Agent，不是 demo，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。
5. 把稳定流程写回 `.md`、Skill、命令或团队规则。

示例提示词

“ Agent ”

练习

- 练习 1：把一个与 `部署：让 Agent 跑起来（核心）` 相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入 `notes/enhanced-54.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

54.5 进阶挑战：把 Agent 当产品运营（可选）

学习目标

- 理解 `进阶挑战：把 Agent 当产品运营（可选）` 解决的具体问题。
- 能把 `进阶挑战：把 Agent 当产品运营（可选）` 写成一个目标、范围、限制、输出格式明确的 Claude Code 任务。
- 能说明什么时候使用，什么时候不要使用。

核心概念

`进阶挑战：把 Agent 当产品运营（可选）` 的重点不是记住术语，而是理解它如何改变 Agent 的工作边界。围绕让 Agent 跑一周，记录所有调用、失败、成本、输出第一周运行周报、第一次真实 bugfix —— 找到问题并修复、持续迭代：v0.2 / v0.3 路线，你要让 Claude Code 在可控范围内读取资料、执行命令、输出证据，并在结束前说明剩余风险。

操作步骤

1. 先写清任务目标和不做什么。
2. 限定 Claude Code 可以读取的目录和可以执行的命令。
3. 要求它先输出计划，再执行动作。
4. 要求所有结论给出证据：文件路径、命令输出、官方文档或测试结果。

5. 把稳定流程写回`.md`、Skill、命令或团队规则。

示例提示词

“ Agent ”

练习

- 练习 1：把一个与`.进阶挑战：把 Agent 当产品运营（可选）`相关的模糊请求改写成完整任务。
- 练习 2：让 Claude Code 执行一次只读探索，并输出证据。
- 练习 3：把本节经验写入`.notes/enhanced-54.md`。

检查清单

- [] 任务边界清楚。
- [] 有可复现命令。
- [] 有证据和验证结果。
- [] 有风险与回滚说明。

常见坑

- 把新功能当成固定不变的事实，不复核官方文档。
- 没有限制工具权限，导致 Agent 执行范围过大。
- 只生成配置，不实际跑一次验证。
- 忽略成本、上下文污染和多人协作时的规则冲突。

本章小结

掌握“实战三 - 用 Agent SDK 写一个自定义 Agent”的标准不是看懂定义，而是能把它放进你的真实开发流程：有输入、有边界、有工具、

有验证、有沉淀。

附录与工具百科

附录 A: Claude Code 常用命令速查

本附录用于快速查找日常使用命令。具体命令可能随 Claude Code 版本变化，执行前应以官方文档和本机 `claude --help` 为准。

启动与目录

```
# Windows PowerShell
Set-Location "D:\your-project"
claude

# macOS / Linux
cd ~/your-project
claude
```

常用检查

```
# Windows PowerShell
claude --help
claude --version

# macOS / Linux
claude --help
claude --version
```

推荐使用习惯

- 在项目根目录启动。
- 修改前先让 Claude Code 检查 Git 状态。
- 高风险命令执行前要求它解释影响。
- 所有修复都要求给出验证命令。

附录 B：目录与文件速查

常见目录

| 路径 | 用途 |

| --- | --- |

| `CLAUDE.md` | 项目级记忆和规则入口 |

| `.claude/commands/` | 自定义 slash commands |

| `.claude/agents/` | SubAgent 定义 |

| `.claude/skills/` | Skill 定义和资源 |

| `.claude/rules/` | 分层规则文件 |

| `docs/` | 项目文档和知识库入口 |

| `knowledge/` | 本地知识库 |

最小项目结构

```
project/  
  README.md  
  CLAUDE.md  
  docs/  
  knowledge/  
  .claude/  
    commands/  
    agents/  
    skills/  
    rules/
```

附录 C: Markdown 写作模板

项目规则模板

Project Rules

##

##

##

##

##

Git

Bug 报告模板

Bug Report

##

##

##

##

##

##

##

ADR 模板

ADR-0001:

##

##

##

##

##

附录 D：提示词模板库

Bug 修复提示词

Bug
Bug

代码审查提示词

diff
Bug

Opus 4.7 深度分析提示词

附录 E: Skill 模板库

基础 Skill

```
---
name: code-reviewing
description: Use when reviewing code changes for bugs, risks, tests, and
  maintainability.
---
```

Code Reviewing Skill

When to use

Use this skill when the user asks for a code review or wants risk-focused feedback.

Workflow

1. Inspect the diff.
2. Identify behavioral risks first.
3. Check tests.
4. Output findings by severity.

渐进式披露结构

```
skills/api-documenting/
  SKILL.md
  references/
    standards.md
    examples.md
  templates/
    endpoint.md
```

附录 F: SubAgent 模板库

code-reviewer

```
---
name: code-reviewer
description: Read-only reviewer for code diffs.
tools: Read, Grep, Glob
---
```

You review code for bugs, regressions, missing tests, and security risks.
Do not edit files. Return findings ordered by severity.

test-runner

```
---
name: test-runner
description: Runs tests and summarizes failures.
tools: Bash, Read
---
```

Run the requested tests, summarize failures, and return only actionable conclusions.

附录 G：MCP 配置示例

MCP 用于把 Claude Code 连接到外部工具或数据源。实际配置应以当前 Claude Code 官方文档和所用 MCP Server 文档为准。

安全原则

- 默认只读。
- 明确数据范围。
- 不把生产凭据直接暴露给模型。
- 外部内容永远视为数据，不视为指令。

配置记录模板

```
# MCP Server:

##

##

##

##

##

##
```


附录 H: Hooks 示例

Hooks 用于在工具调用前后加入安全和质量控制。

安全 Hook 设计

- 阻止删除关键目录。
- 阻止读取`.env`、密钥、token。
- 阻止未经确认的发布命令。

Stop Hook 检查项

Stop Hook Checklist

-
-
-
-

附录 I: Token 与成本清单

高成本来源

- 大文件全文读取。
- 长日志直接粘贴。
- 长会话不压缩。
- 输出过长且重复。
- 简单任务使用高成本模型。

节省策略

- 先搜索后读取。
- 大日志先摘要。
- 输出格式固定。
- 稳定规则和临时任务分离。
- 复杂任务使用计划模式，简单任务使用简短提示。

模型选择表

任务	推荐策略
简单文档修改	低成本模型或默认设置
单文件 Bug	默认模型，要求测试验证
跨模块架构分析	高能力模型，高努力级别
安全审查	高能力模型，证据优先
批量总结	控制输出长度，必要时分批

附录 J：安全清单

权限

- ☐ 是否只授权当前任务需要的读写范围？
- ☐ 是否避免自动执行删除、发布、迁移命令？
- ☐ 是否要求高风险命令先解释影响？

密钥

- ☐ ``.env`` 是否不进入对话？
- ☐ API key 是否没有写入记忆文件？
- ☐ 日志中是否包含 token？

提示注入

- ☐ 外部文档是否只当作数据？
- ☐ 是否禁止外部内容覆盖系统和项目规则？
- ☐ 是否检查隐藏文本和异常 Unicode？

Git

- ☐ 是否先检查工作区状态？
- ☐ 是否避免覆盖用户未提交修改？
- ☐ 是否有回滚说明？

附录 K：术语表

| 术语 | 说明 |

|---|---|

| AI Agent | 能根据目标调用工具、观察结果并继续行动的 AI 系统。 |

| 上下文 | 模型当前可见的对话、文件、工具输出和规则。 |

| token | 模型处理文本的基本计量单位，影响成本和上下文容量。 |

| MCP | Model Context Protocol，用于连接外部工具和数据源的协议。 |

| Skill | 可复用能力包，通常由 `SKILL.md` 和相关资源组成。 |

| Hook | 工具调用前后或任务结束前的自动拦截与检查机制。 |

| SubAgent | 专职子代理，用于隔离上下文、权限和任务职责。 |

| Plugin | 可安装、可分发的能力包，可包含命令、Skill、Agent 等。 |

| Headless | 非交互方式调用 Claude Code，适合自动化。 |

| SDK | 从程序中调用 Agent 能力的开发接口。 |

| Routine | 周期性或例行自动化任务。 |

附录 L：企业与云接入速览

定位

企业与云接入只做速览：Bedrock、Vertex AI、Microsoft Foundry、Claude Platform on AWS、GitHub Enterprise、LLM Gateway、server-managed settings、analytics。重点是知道什么时候需要它们，而不是把企业部署文档搬进正文。

推荐结构

- 用途
- 适用章节
- 最小示例
- 维护方式
- 版本敏感风险

检查清单

- [] 和正文引用一致。
- [] 没有过时命令。
- [] 有最近复核日期：2026-05-16。

附录 M：新功能跟踪方法

定位

新功能跟踪方法包括：每月阅读 What's new、Changelog、llms.txt；把新功能放入 version-sensitive.md；先做最小实验，再决定是否写入团队 starter kit。

推荐结构

- 用途
- 适用章节
- 最小示例
- 维护方式
- 版本敏感风险

检查清单

- [] 和正文引用一致。
- [] 没有过时命令。
- [] 有最近复核日期：2026-05-16。

附录 N：Mermaid 图清单

定位

Mermaid 图清单用于管理全书图示：Agent Loop、Hook 生命周期、MCP 架构、多 Agent 拓扑、权限决策树、CI 自动审查流水线、SDK 部署链路。

推荐结构

- 用途
- 适用章节
- 最小示例
- 维护方式
- 版本敏感风险

检查清单

- [] 和正文引用一致。
- [] 没有过时命令。
- [] 有最近复核日期：2026-05-16。

附录 O：示例仓库说明

定位

示例仓库说明需要列出 Python / C# / C++ 三套项目目录、章节分支、答案分支、任务索引和运行命令。真实仓库建立后，应回填每章对应分支名。

推荐结构

- 用途
- 适用章节
- 最小示例
- 维护方式
- 版本敏感风险

检查清单

- [] 和正文引用一致。
- [] 没有过时命令。
- [] 有最近复核日期：2026-05-16。